TECHNICAL REFERENCE MANUAL

Semiconductor Design Infrastructure Handbook

Deployment, Configuration, and Administration of Cadence-based RHEL workstations.

AUTHOR

Anirban Das
BTech (ECE)
Class of 2027

ADVISORS

Dr. Anshika Srivastava
Program Coordinator
Department of Electronics and Communication Engineering

Dr. Prasiddh Trivedi
Assistant Professor
Department of Electrical Engineering

Gati Shakti Vishwavidyalaya

A Central University under Ministry of Railways, Government of India
Vadodara, Gujarat 390004

Abstract

This manual presents a beginner-oriented but infrastructure-conscious guide to building and using a Cadence-based semiconductor design environment on Red Hat Enterprise Linux. It begins by explaining why modern Electronic Design Automation workflows depend on enterprise Linux platforms, why RHEL and RHEL-compatible systems are commonly used in academic and industrial VLSI environments, and how a laboratory workstation must be planned so that the operating system, user identity, licensing, storage, and tool environment remain consistent across machines.

The first part of the document focuses on environment preparation. It covers the practical decisions required before installing RHEL, including hardware expectations, virtualization versus native dual booting, installation media preparation, UEFI and Secure Boot considerations, disk partitioning strategy, and post-installation workstation identity. It then introduces the configuration work required for Cadence usage, including directory organization, environment variables, C-shell setup, license configuration, installation verification, and tool launch procedures. This establishes the workstation as a usable and reproducible EDA platform rather than as a generic Linux desktop.

After the system environment is established, the manual transitions into the digital design flow itself. It introduces Verilog, combinational and sequential RTL design, testbench construction, simulation, waveform analysis, and coverage analysis using Cadence tools such as Xcelium and IMC. The learning path is centered around small laboratory exercises, including adders and counters, so that each concept is connected to a concrete design task rather than presented only as theory.

The later parts of the document extend the same design from RTL toward implementation. The manual outlines logic synthesis using Genus, timing analysis, logical equivalence checking with Conformal, and the major stages of physical design using Innovus, including floorplanning, power planning, placement, clock tree synthesis, routing, and physical verification. It concludes with GDSII generation, tapeout context, the relationship between silicon and PCB-level integration, and a complete review of the RTL-to-GDSII flow.

Overall, the intent of this manual is to give new users a coherent view of both sides of an EDA laboratory: the infrastructure required to run professional design tools reliably, and the step-by-step design flow used to take a simple digital circuit from Verilog source code through simulation, coverage, synthesis, verification, physical implementation, and final layout data generation.

Preface

The ability to design integrated circuits has become one of the defining technological capabilities of the modern world. From communication systems and transportation infrastructure to medical devices, defense electronics, artificial intelligence accelerators, and high-performance computing platforms, nearly every advanced engineering discipline today relies upon semiconductor technology. Yet the process of designing semiconductor devices is not enabled by design software alone. Behind every successful VLSI laboratory exists a carefully constructed computing infrastructure capable of supporting Electronic Design Automation (EDA) workflows reliably, consistently, and at scale.

The creation of such an environment is rarely visible to students. When a laboratory workstation is switched on and a design tool launches successfully, years of operating system development, network engineering, software integration, licensing infrastructure, and administrative planning remain hidden beneath the surface. However, establishing that environment from scratch is often a complex undertaking involving far more than the installation of a single software package.

This document originated from the practical challenges encountered during the establishment of the semiconductor design infrastructure used within the Department of Electronics and Communication Engineering at Gati Shakti Vishwavidyalaya. The work described throughout this manual was carried out as part of a broader effort to deploy a standardized Linux-based CAD environment capable of supporting Cadence design tools under the Chips-to-Startup (C2S) Programme.

The deployment process involved the installation and configuration of Red Hat Enterprise Linux workstations, preparation of user environments, integration of licensing services, workstation provisioning, software deployment, shell configuration, network validation, and the establishment of a repeatable administrative workflow. While many of these tasks appear straightforward when viewed individually, their interaction within a laboratory environment introduces a level of complexity that is often underappreciated until deployment begins.

A significant portion of this work was carried out by the author under the guidance of Dr. Prasiddh Trivedi, Assistant Professor, Department of Electrical Engineering, whose support, technical direction, and institutional coordination made the deployment possible. Throughout the setup process, numerous infrastructure decisions had to be evaluated and justified, ranging from operating system selection and workstation architecture to user naming conventions, deployment methodologies, storage allocation strategies, and license-compliance requirements. The resulting environment was not assembled through trial and error alone, but through a deliberate attempt to create a platform that could be maintained, reproduced, and expanded as the laboratory evolved.

One of the recurring observations during the deployment was the scarcity of documentation describing the complete setup process from an institutional perspective. Vendor manuals typically explain how to install a particular software package, while operating system documentation explains how to configure a particular service. What is often missing is a unified narrative describing how these components fit together within a real laboratory environment. Future administrators are therefore frequently forced to reconstruct deployment decisions by examining existing machines, consulting fragmented notes, or relying upon institutional memory.

This manual was written to address that problem.

Its purpose is not merely to record commands that were executed during installation, but to capture the reasoning behind those decisions. Wherever possible, procedures are accompanied by explanations describing why a particular configuration was chosen, what assumptions were made, and what consequences may arise if those assumptions change. The objective is to transform administrative knowledge from an informal collection of personal experience into a structured and reproducible reference.

The need for such documentation becomes especially important in academic environments. Students graduate, research projects conclude, hardware is replaced, operating systems reach end-of-life, and administrative responsibilities change hands. Without documentation, knowledge accumulated during deployment can disappear remarkably quickly. A workstation that functions perfectly today may become difficult to rebuild several years from now if the decisions that produced it were never recorded. By documenting both procedures and rationale, the laboratory reduces its dependence on individual administrators and preserves

institutional knowledge for future use.

At the time of writing, many of the systems described in this document are already being deployed directly rather than reconstructed from documentation. Consequently, it is entirely possible that no immediate reader will ever need to follow every procedure presented here from beginning to end. In fact, the most successful outcome may be that future administrators rarely need this manual at all because the infrastructure continues to operate smoothly. Nevertheless, the value of technical documentation should not be measured solely by how often it is consulted. Its true purpose is to exist when something eventually changes: when a workstation fails, when a laboratory expands, when software must be reinstalled, or when a future administrator asks a simple but important question—"How was this system originally set up?"

This handbook therefore serves both as an operational reference and as a historical record of the deployment effort undertaken to establish the semiconductor CAD environment at Gati Shakti Vishwavidyalaya. It documents not only the final configuration of the systems, but also the engineering considerations, constraints, and administrative decisions that shaped their design.

The chapters that follow describe the deployment of the workstation environment, operating system preparation, configuration standards, and supporting infrastructure required for Cadence-based semiconductor design workflows. Together, they represent an attempt to ensure that the knowledge acquired during this deployment remains accessible, reproducible, and useful long after the initial installation work has been completed.

Contents

Part I

SYSTEM SETUP

Operating System Deployment and RHEL Workstation Initialization

01

Foundations of the EDA Environment

1.1 Introduction to EDA

Electronic Design Automation (EDA) refers to the collection of software tools, computational frameworks, and engineering workflows used in the design, simulation, verification, and physical implementation of integrated circuits and semiconductor systems. Modern semiconductor devices contain millions to billions of transistors operating at extremely small geometries, making manual design methodologies computationally impractical and operationally impossible. EDA tools therefore serve as the foundational infrastructure enabling engineers to transform conceptual circuit designs into manufacturable silicon devices.

Contemporary EDA environments encompass a wide range of specialized applications including schematic capture systems, Hardware Description Language (HDL) development environments, circuit simulators, synthesis tools, physical layout editors, parasitic extraction frameworks, timing analysis engines, and design-rule verification systems. These tools collectively support the complete semiconductor development lifecycle, from behavioral modeling and functional verification to physical layout generation and fabrication validation.

The increasing complexity of Very Large Scale Integration (VLSI) systems has made EDA indispensable within both academic and industrial semiconductor design workflows. Beyond improving productivity, EDA enables precision, scalability, repeatability, and verification accuracy that would otherwise be unattainable through conventional design methodologies. As a result, modern semiconductor engineering is fundamentally dependent upon robust EDA infrastructure and the computational environments that support it.

1.2 Why Enterprise Linux Anchors Modern EDA?

Modern EDA toolchains impose demanding requirements on the underlying operating system environment, including stability, deterministic behavior, scripting flexibility, network interoperability, multi-user access control, and long-term software compatibility. Consequently, semiconductor design organizations and research laboratories overwhelmingly deploy EDA infrastructure on Linux-based operating systems rather than general-purpose desktop platforms.

Linux provides a highly modular and administratively transparent environment capable of supporting complex engineering workflows involving distributed compute resources, centralized licensing systems, shared storage infrastructure, remote administration, and automated deployment methodologies. Native shell scripting capabilities, package management systems, process control utilities, and network tooling further enable scalable workstation administration and reproducible environment configuration across large laboratory deployments.

Among available Linux distributions, enterprise-grade platforms such as **Red Hat Enterprise Linux (RHEL)** and its derivatives have become the de facto standard within semiconductor design environments. Their widespread adoption is driven by long-term support lifecycles, predictable release stability, extensive vendor certification, and broad compatibility with commercial EDA applications. Most major semiconductor

tool vendors officially validate and support their software primarily on enterprise Linux environments, making RHEL-based systems the preferred foundation for reliable and maintainable CAD infrastructure deployment.

Accordingly, the workstation architecture described throughout this manual is built upon a standardized RHEL-based environment designed to provide operational consistency, scalability, and long-term maintainability for semiconductor design laboratories and VLSI research workflows.

1.3 Red Hat Enterprise Linux as Industrial Infrastructure

The decision to standardize a semiconductor design environment on a specific Linux distribution is not a matter of administrative preference. It is an infrastructure decision with direct consequences for EDA tool compatibility, operational stability, vendor support, and long-term maintainability. Within professional semiconductor environments, Red Hat Enterprise Linux has become a preferred foundation because it provides the stability and compatibility characteristics required by commercial EDA workflows.

Commercial EDA tools are typically distributed as precompiled binary applications that depend on stable system libraries, predictable runtime behavior, and well-defined operating system interfaces. RHEL maintains Application Binary Interface and Application Programming Interface consistency within a major release, allowing validated tool installations to remain reliable across routine system updates. This consistency is especially important for tools that depend on components such as `glibc`, `libX11`, Motif libraries, OpenSSL, shell utilities, and kernel-level behavior.

RHEL also provides long-term lifecycle stability that aligns well with semiconductor design workflows. A laboratory or production design environment may depend on a specific operating system and EDA tool combination for several years. Frequent distribution-level changes can introduce dependency drift and invalidate previously working configurations. By contrast, RHEL's enterprise release model prioritizes predictable maintenance, security updates, and controlled package evolution over rapid upstream change.

Vendor certification is another important factor. Major EDA vendors commonly certify their tools against specific RHEL major versions and closely compatible derivatives. Operating within a certified platform reduces support risk, simplifies troubleshooting, and provides a defensible baseline when vendor escalation is required. While unsupported Linux distributions may sometimes run the same tools, they introduce additional uncertainty into installation, execution, and support workflows.

The RHEL ecosystem also supports maintainable administration practices through standard tools such as `dnf`, `systemd`, `lvm`, `sssd`, and `subscription-manager`. These tools provide a well-documented administrative surface for package management, service control, storage configuration, authentication integration, and system maintenance. In institutional environments where systems must be rebuilt, audited, extended, or handed over between administrators, this consistency is a practical advantage.

Throughout the remainder of this manual, operating system procedures, configuration examples, package names, service management commands, and administrative workflows are specified for a RHEL 9-compatible environment. Unless otherwise noted, the procedures also apply to Rocky Linux 9 and AlmaLinux 9 as binary-compatible RHEL derivatives. References to *RHEL* in the operational context of this document should therefore be understood to include these derivatives unless a Red Hat subscription-specific feature is explicitly required.

1.4 Design Philosophy of the EDA Environment

The semiconductor design infrastructure described in this manual is not assembled opportunistically. It is engineered according to a defined set of operational principles that govern every decision from initial workstation deployment through long-term maintenance. Understanding these principles is necessary context for the procedures that follow, because many configuration choices that might otherwise appear arbitrary are direct expressions of the underlying philosophy.

The environment is built around **standardization**. All workstations in the laboratory are deployed from a common baseline: identical operating system version, identical package selection, identical directory structures, and identical tool installation paths. Deviation from this baseline is treated as an exception requiring justification, not a routine outcome of per-machine administration. Standardization is the prerequisite for everything else.

From standardization follows **reproducibility**. A workstation rebuilt from documented procedures should produce an environment functionally identical to the one it replaces. This property matters both operationally—when a workstation fails and must be restored—and institutionally, as the laboratory's administrative knowledge must remain embedded in its documentation rather than in the accumulated state of individual machines.

The deployment is designed to **scale**. Procedures written for a single workstation apply without modification to the full laboratory fleet. Configuration that must be individually customized per machine is a design defect. Where centralized mechanisms exist—for user authentication, license server connectivity, shared storage, or environment initialization—they are used in preference to local equivalents.

Centralized administration follows from this. User accounts, access permissions, and shell environments are managed at the infrastructure level rather than configured locally on each workstation. This reduces the administrative surface area, eliminates inconsistency across machines, and ensures that changes propagate uniformly rather than requiring repeated per-host intervention.

Underlying all of these principles is the explicit goal of **eliminating configuration drift**. In long-running laboratory environments, workstations tend to diverge from one another over time as ad hoc fixes, local experiments, and undocumented modifications accumulate. The procedures in this manual are written to resist this tendency—through documented baselines, version-controlled configuration, and periodic validation practices that detect and correct deviation before it compounds.

The result is an infrastructure in which any workstation can be administered, diagnosed, or rebuilt by any administrator with access to this document, without dependence on institutional memory or machine-specific knowledge.

1.5 Overview of Laboratory Infrastructure

The laboratory environment should be understood as more than a collection of independent desktop computers. In a semiconductor design lab, each workstation is one part of a larger operating environment that includes user accounts, network identity, license access, shared design data, tool installation paths, and common shell configuration. A student may experience the system as "logging into Linux and opening Cadence", but behind that simple action several infrastructure components must work together reliably.

At the center of the setup are the **RHEL workstations**. These are the machines on which students and users log in, write Verilog, launch simulations, open waveforms, run synthesis, and eventually interact with physical design tools. Each workstation may be installed natively or may initially run RHEL inside a virtual machine, but its role is the same: it acts as the user's EDA compute endpoint. The workstation must therefore have a predictable hostname, a correctly named user account, a stable operating-system baseline, and access to the required Cadence environment.

Surrounding the workstations are the shared services that make the lab behave like a coordinated engineering environment rather than a set of isolated PCs. The **license server** controls access to Cadence tools; the workstation requests permission from this server whenever a licensed tool is launched. Shared storage, when available, provides a common location for project files, PDKs, reference scripts, installation archives, or laboratory examples. The network layer connects these pieces together through IP addresses, DNS names, hostnames, and routing. If these identities are inconsistent, software that depends on licenses, paths, or remote resources may fail even when the local installation appears correct.

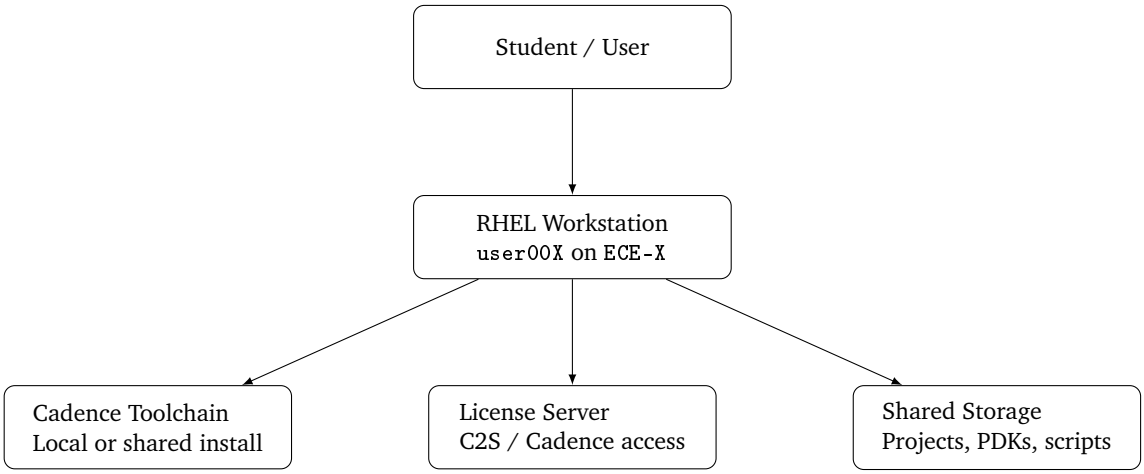


Figure 1.1: High-level view of the laboratory EDA infrastructure

This section is therefore an architectural map, not a detailed installation procedure. It shows what the main pieces are and why they must remain consistent. Later chapters configure these pieces one by one: the operating system is installed, the workstation identity is assigned, the Cadence environment variables are prepared, the license configuration is verified, and the design flow is exercised through RTL design, simulation, synthesis, and physical implementation.

1.6 Workstation Hardware Requirements and Deployment Constraints

Electronic Design Automation tools impose system resource requirements that differ substantially from those of general-purpose desktop software. A typical office productivity environment or software development workstation can operate adequately on modest storage allocations, a few gigabytes of memory, and commodity processor performance. EDA environments do not share these characteristics. The software stacks involved in a complete semiconductor design workflow are architecturally large, computationally intensive, and operationally demanding in ways that must be accounted for at the infrastructure planning stage rather than addressed reactively after deployment.

1.6.1 Storage Requirements

Modern EDA suites consist of numerous interrelated tools, each with its own binaries, libraries, technology files, process design kits, documentation sets, and supporting scripts. A complete installation image for a commercial EDA suite does not occupy gigabytes in the way that a large application suite might. It occupies an order of magnitude more. The Cadence EDA installation deployed in this laboratory environment occupied approximately **480 GB** of storage prior to the creation of any user project data, simulation output, waveform databases, or temporary working files. This figure reflects only the tool installation itself.

In practice, active design work compounds storage consumption considerably. Simulation runs generate large output datasets. Layout databases accumulate across design iterations. Waveform viewers load and cache substantial binary files during interactive sessions. Verification engines write intermediate results to disk throughout their execution. Over the operational lifetime of a workstation, the storage footprint of the design environment grows continuously. Storage planning must therefore account not only for the initial installation image but for a realistic projection of working data volume across the deployment period.

For this reason, workstations intended for sustained EDA use should be provisioned with local storage that provides meaningful headroom beyond the installation baseline. The 480 GB installation figure should be treated as a floor, not a target. Configurations that provision only enough storage to accommodate the initial installation leave no margin for project data, OS overhead, swap allocation, or tool updates, and will require administrative intervention as the environment matures.

1.6.2 Memory Requirements

Semiconductor design workflows routinely involve multiple concurrently active applications. A typical interactive session may involve a layout editor, a schematic capture environment, a circuit simulator, a waveform viewer, and a design rule checking utility operating simultaneously within the same desktop session. Each of these applications maintains its own working set in memory, and several—particularly simulation engines and verification utilities—can expand their memory consumption substantially during active computation.

Insufficient memory allocation produces predictable and disruptive consequences in this context. Operating system memory pressure forces active application data to swap, degrading interactive responsiveness and extending simulation runtimes. In severe cases, the out-of-memory subsystem may terminate running processes, corrupting simulation state and requiring workflow restart. These outcomes are not edge cases in underpowered environments; they are routine operational hazards.

Workstations deployed for full EDA workflows should be provisioned with sufficient physical memory to support concurrent tool operation without inducing swap utilization under normal working conditions. The specific requirement varies by workflow, but **configurations below 16 GB of physical memory are generally unsuitable for sustained multi-tool VLSI design sessions**. Workstations supporting heavier simulation workloads or concurrent multi-user access benefit from higher memory allocations accordingly.

1.6.3 Deployment Methodology: Native Installation versus Virtualization

The manner in which RHEL is deployed on a workstation has direct consequences for the resource availability of the EDA environment. Two deployment models are relevant to this laboratory context: direct installation on dedicated hardware, and virtualized deployment within a hypervisor environment running on an existing host operating system.

In a native installation—whether on a dedicated workstation or in a dual-boot configuration alongside an existing operating system—RHEL has unrestricted access to all physical hardware resources. The full complement of CPU cores, physical memory, storage throughput, and graphics capability is available to the operating system and the EDA tools running within it. This is the preferred deployment model for sustained engineering workflows, as it eliminates the resource-sharing constraints and performance overhead inherent to virtualized environments.

Virtualization, however, represents a practically useful alternative under specific deployment conditions. During the initial workshop phase of this laboratory deployment, Microsoft Hyper-V was used to provision RHEL virtual machines on workstations running existing Windows installations. This approach was selected because many of the available systems contained active Windows environments that could not be immediately repartitioned, and the operational priority was to make the EDA environment accessible within a constrained preparation timeline.

The resource tradeoffs introduced by this approach are significant and should be understood clearly. The reference deployment system carried 32 GB of physical memory. Of this, 16 GB was allocated to the RHEL virtual machine, with the remaining capacity reserved for the Windows host and the memory overhead of the Hyper-V hypervisor layer itself. Under this configuration, the EDA environment operates within a resource envelope that is materially smaller than the physical hardware could otherwise provide. CPU scheduling is mediated by the hypervisor, storage I/O is subject to virtualization overhead, and memory allocation is fixed at provisioning time rather than dynamically available from the full physical pool.

Virtualized deployment is appropriate for introductory workshops, environment familiarization, and short-duration instructional use cases in which the full performance envelope of the EDA tools is not required. It is not the preferred model for production semiconductor design workflows. As workstation preparation schedules permit, systems initially deployed under Hyper-V should be transitioned to native RHEL installations in order to provide the EDA environment with unrestricted access to the underlying hardware resources. All procedures documented in this manual apply to both deployment models unless a specific section explicitly notes otherwise.

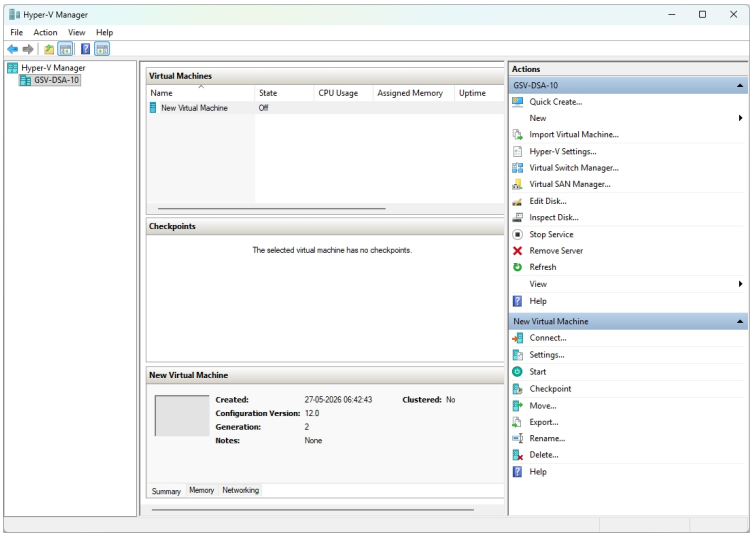


Figure 1.2: Screenshot of Hyper-V

1.7 Summary

This chapter introduced the foundational concepts underlying modern semiconductor EDA infrastructure and established the operational context for the deployment procedures described throughout this manual. The discussion examined the role of Electronic Design Automation within contemporary VLSI workflows, the infrastructure requirements imposed by commercial EDA toolchains, and the reasons enterprise Linux environments have become the standard platform for semiconductor engineering systems.

The chapter further justified the adoption of Red Hat Enterprise Linux as the primary operating system environment for workstation deployment, emphasizing considerations such as long-term stability, vendor certification, maintainability, and compatibility with commercial EDA applications. In addition, the architectural philosophy of the laboratory environment, including standardization, reproducibility, centralized administration, and scalability, was introduced as the guiding framework for all subsequent deployment and maintenance procedures.

Finally, the chapter discussed the hardware and deployment considerations associated with EDA environments, including storage requirements, memory allocation strategies, and the operational tradeoffs between native and virtualized deployment methodologies. These concepts collectively establish the technical foundation required for the operating system installation and workstation provisioning procedures described in the following chapter.

02

RHEL Installation and Setup

Note: This guide assumes you are familiar with Linux environments to a basic degree, are comfortable with the terminal, and have an active internet connection. We will not discuss the entire installation process, only the core infrastructure decisions that need to be made for successful setup of the base system.

That being said however, every decision and command will be discussed in elaborate detail – justifying why it was made, and what it does.

2.1 Installation Planning and Deployment Strategy

Before a workstation is installed, the deployment method must be selected deliberately. The choice is not merely a matter of convenience; it determines how the machine will consume hardware resources, how reliably EDA tools will interact with the operating system, how easily the workstation can be maintained, and how closely the final environment will resemble a production semiconductor design platform. In the context of this manual, RHEL is treated as the operating system baseline for a controlled EDA workstation rather than as a general-purpose desktop installation.

There are two practical ways to deploy RHEL on a laboratory workstation:

1. **Virtualization** – installing RHEL inside a virtual machine hosted by an existing operating system.
2. **Dual booting** – installing RHEL directly on the physical machine alongside an existing operating system by using bootable installation media.

Virtualization is useful for training, experimentation, documentation, and low-risk validation of administrative procedures. It allows the installer to be tested without repartitioning the physical disk and makes it easy to take snapshots before major configuration changes. However, virtual machines introduce an additional abstraction layer between the EDA workload and the hardware. This can limit graphics performance, memory availability, disk throughput, USB access, license-dongle support, and other device-level behavior that may matter during tool deployment or troubleshooting.

Dual booting installs RHEL natively on the workstation hardware. This approach gives the Linux environment direct access to the CPU, memory, graphics adapter, storage devices, and network interfaces, making it the preferred method when the machine is expected to function as a serious EDA workstation. Native installation also more closely matches the assumptions made by commercial EDA vendors, whose installation and support documentation typically targets physical or enterprise-managed Linux systems rather than desktop virtual machines.

The decision made at this stage affects the entire downstream configuration: disk layout, package selection, driver support, user account preparation, license-server connectivity, shared storage mounts, backup strategy, and future system recovery. For that reason, the installation procedure should be selected before any

partitioning or media preparation begins. This chapter documents the installation workflow with attention to university licensing constraints, C2S operational guidelines, and the need for a reproducible workstation baseline.

2.2 Hardware Requirements and Workstation Specifications

A modern semiconductor CAD workstation intended for full-scale Cadence deployment must be provisioned with significantly higher hardware resources than a conventional desktop system. Contemporary EDA workflows involve computationally intensive applications for schematic capture, mixed-signal simulation, physical layout, parasitic extraction, verification, and waveform analysis, many of which operate on large design databases and generate substantial intermediate data during execution. In addition to the operating system itself, a complete Cadence installation together with process design kits (PDKs), technology libraries, simulation environments, temporary build data, and user project directories can consume several hundred gigabytes of storage.

Workstations intended for academic laboratory deployment should therefore prioritize memory capacity, storage performance, multi-core CPU capability, thermal stability, and long-term maintainability over consumer-oriented graphical features. While smaller configurations may support introductory workflows, a poorly provisioned workstation can significantly degrade simulation performance, layout responsiveness, and overall user productivity during large-scale VLSI design tasks.

At the time of writing of this document, the current system available to us had the following specs: (*so you can make a fair estimate of the requirements based on this*)

Component	**Detected Specification**
Workstation Model	HP Z4 G5 Workstation Desktop PC
System Type	64-bit x86 Workstation Platform
Processor (CPU)	Intel Xeon w3-2423 Processor 6 Physical Cores, 12 Logical Processors Base Frequency: 2.11 GHz
Installed Memory (RAM)	32 GB DDR Memory Installed
Firmware / BIOS Mode	UEFI Boot Mode Enabled
Secure Boot Status	Enabled
Virtualization Support	Hardware Virtualization Enabled Hypervisor and Virtualization-Based Security Active
Motherboard Platform	HP System Board 8962
Operating System	Microsoft Windows 11 Pro for Workstations Build 26200 (64-bit)
Platform Classification	Enterprise Workstation-Class System
Storage Configuration	Not fully listed in exported report Recommended verification of SSD/NVMe capacity before EDA deployment
Graphics Processing	Not listed in current report extract Dedicated GPU verification recommended for OpenGL-based layout workflows

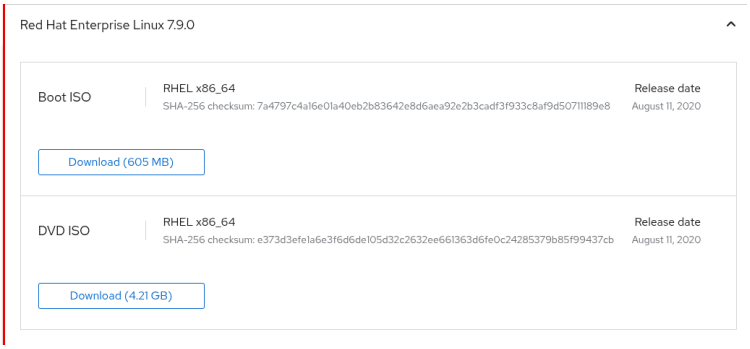
Table 2.1: Host Workstation Specifications Prior to RHEL Deployment

2.3 Obtaining the RHEL Installation Media

The installation workflow begins with obtaining the correct RHEL installation image from Red Hat. A Red Hat Developer account can be used for non-production developer access, subject to Red Hat's current subscription terms and the institution's licensing policy. After signing in, the installer image should be downloaded from the Red Hat Developer portal:

`https://developers.redhat.com/products/rhel/download`

For this workstation deployment, we are using **Red Hat Enterprise Linux 8.7**. This version is selected because many EDA tools and supporting libraries are validated against older enterprise Linux releases, and RHEL 8.7 provides a mature, well-documented platform with broad community and vendor support. In an EDA environment, compatibility and predictable behavior are usually more important than using the newest available operating-system release.



Make sure to download the DVD not the Boot version. The downloaded ISO should match the architecture of the workstation, normally `x86_64` for standard laboratory PCs. The version should also match the operating-system baseline approved for the EDA toolchain being deployed. Once the ISO has been downloaded, its integrity should be verified when checksum information is available, and the file should be stored in a documented location so that future workstation rebuilds use the same installation source.

Before writing the ISO to a USB drive or modifying the workstation disk, record the machine's current boot mode, storage configuration, and existing operating-system layout. In particular, confirm whether the system is using UEFI or legacy BIOS boot, whether Secure Boot is enabled, how much unallocated disk space is available, and whether the disk uses GPT or MBR partitioning. These details determine the correct installation path and reduce the risk of inconsistent dual-boot behavior across machines.

2.4 BIOS, UEFI and Secure Boot Configuration

Before installing RHEL, it is important to understand the firmware layer that controls how the machine starts. The firmware initializes hardware, detects bootable devices, and hands control to the operating-system bootloader. Incorrect firmware settings can prevent the installer from booting or can cause the installed operating system to disappear from the boot menu.

BIOS refers to the older firmware interface used by traditional PCs. It performs basic hardware initialization and boots from disks using the legacy boot process, commonly associated with MBR partitioning. Modern workstations generally no longer depend on legacy BIOS, but the term is still commonly used to refer to the system firmware setup screen.

UEFI is the modern replacement for legacy BIOS. It supports GPT partitioning, an EFI System Partition, graphical firmware menus, larger disks, and more flexible boot management. For current workstation deployments, RHEL should normally be installed in UEFI mode so that it matches the boot mode used by Windows and avoids mixed-mode dual-boot problems.

Secure Boot is a UEFI security feature that allows only trusted, signed bootloaders and kernel components to start during the boot process. It helps protect the system from unauthorized boot-time code, but it can also affect Linux installation, third-party drivers, and some virtualization workflows if not configured correctly.

> **Note:** On the HP workstation used in this deployment, the firmware setup screen can be entered by pressing **F2** during startup. On other systems, the key may be different; common alternatives include **F10**, **F12**, **Esc**, or **Delete**, depending on the manufacturer.

Secure Boot will be discussed in more detail in the next section, separately for native dual-boot installation and virtualization-based deployment. The correct setting depends on the installation model, the bootloader path, and whether the deployment requires vendor drivers or kernel-level components that may not be signed for Secure Boot.

2.5 Virtualized RHEL Installation

A virtualized RHEL installation is useful for evaluating the installer, documenting setup procedures, testing package installations, and training users without modifying the physical workstation disk. In this approach, RHEL runs as a guest operating system inside a hypervisor such as VMware Workstation, VirtualBox, or Hyper-V, while Windows remains the host system, making the process low risk because the virtual disk exists only as a file that can be backed up, restored, or removed easily.

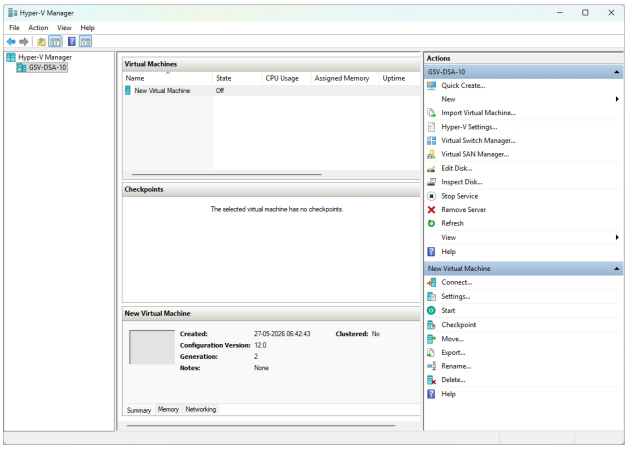
Although virtualization accurately reproduces the RHEL userspace, shell environment, package management, and administrative workflow, it does not fully replicate native hardware behavior, particularly for graphics acceleration, USB passthrough, storage performance, license dongles, and memory-intensive EDA workloads. Therefore, virtualization is best suited for validation, learning, and preliminary testing, while native dual booting remains the preferred choice for production-scale Cadence usage. Before creating the VM, ensure that hardware virtualization support (Intel VT-x or AMD-V/SVM) is enabled in the system firmware.

2.5.1 Preparing the Virtual Machine

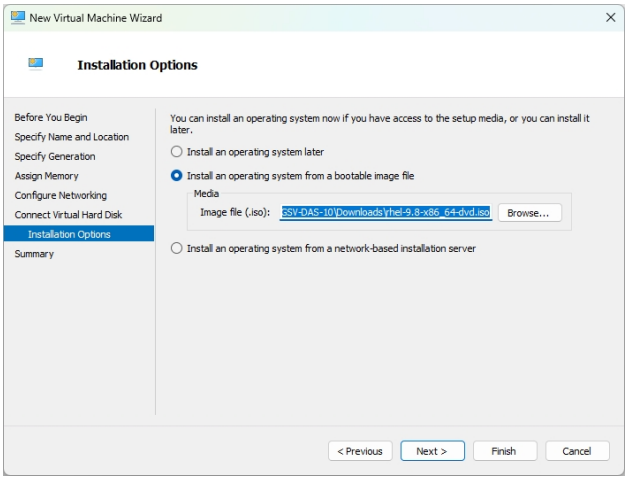
We will need to enable Microsoft Hyper-V for this. Click **Start -> Turn Windows Features On or Off**, scroll till you find the Hyper-V icon and check the box. Restart the system once.

> **Note:** You need Windows Pro or Enterprise to be able to enable Hyper-V. On Windows-Home Hyper-V is not natively supported, but can be installed via a script provided at the end of this chapter.

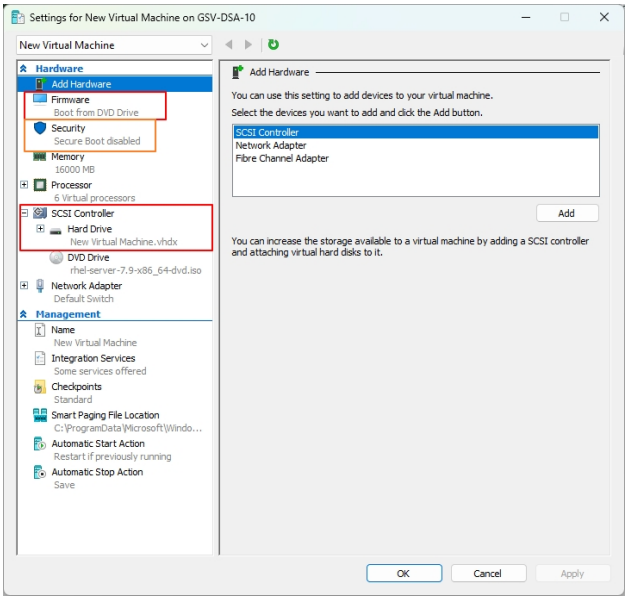
Once you have restarted the system once, open Hyper-V, you will be met with the following screen:



Click "**New**" → "**Virtual Machine**" and follow the prompts further. Make sure you choose the correct Installation Media – the ISO file we installed earlier.

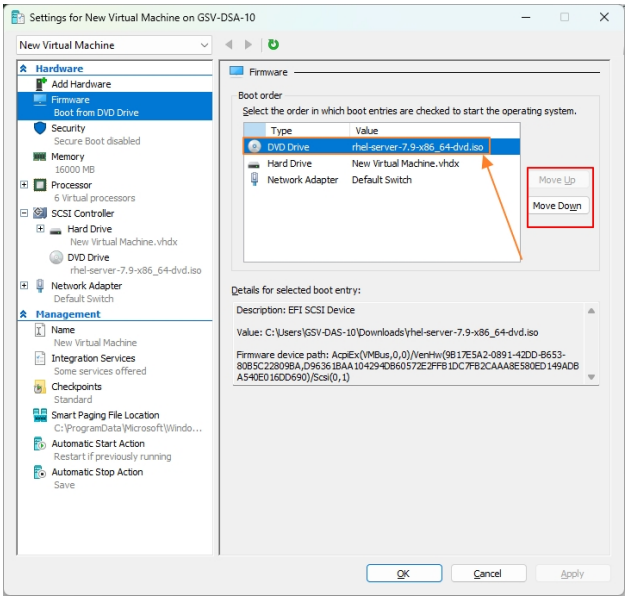


Most of the steps in the prompt are straight-forward. There are a few key configurations you need to keep in mind after setting up the virtual Machine. **Right Click** on the virtual machine, and then click **Settings** to open the following window:

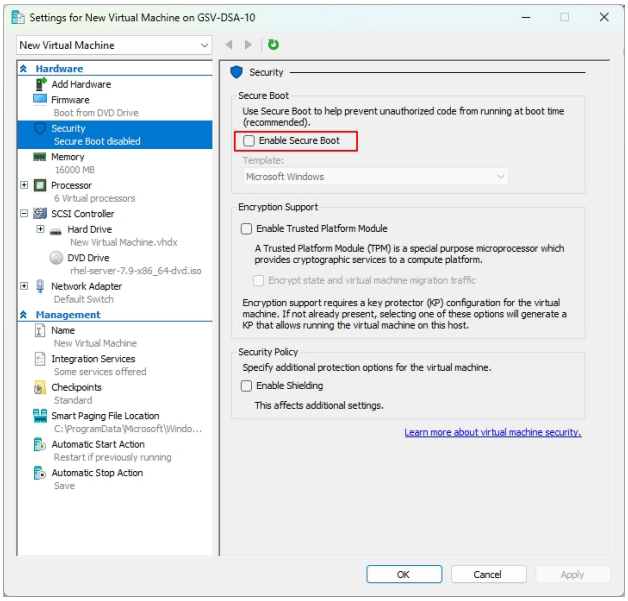


We're interested in only the following three topics, namely:

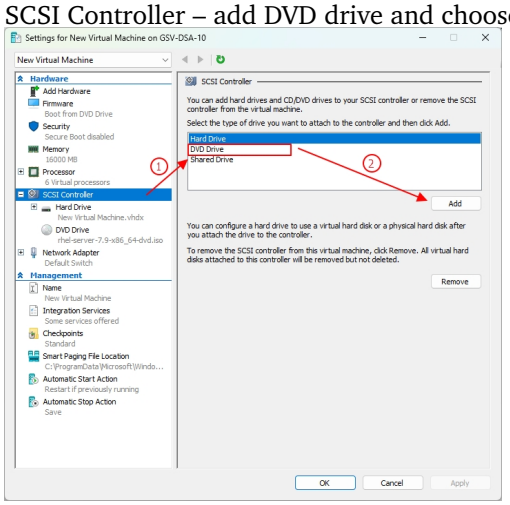
1. Firmware – make sure that DVD drive is at the top of the hierarchy.

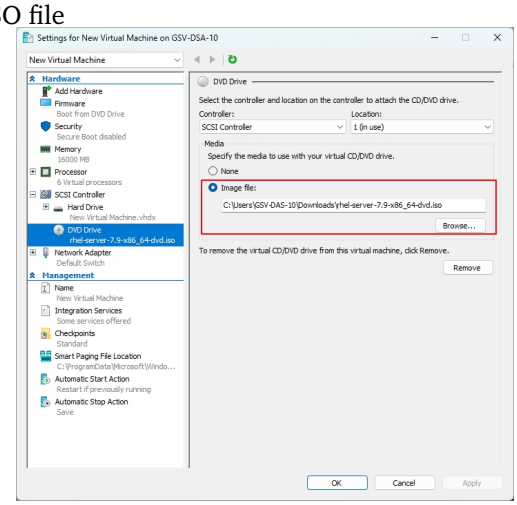


2. Security – disable Secure Boot

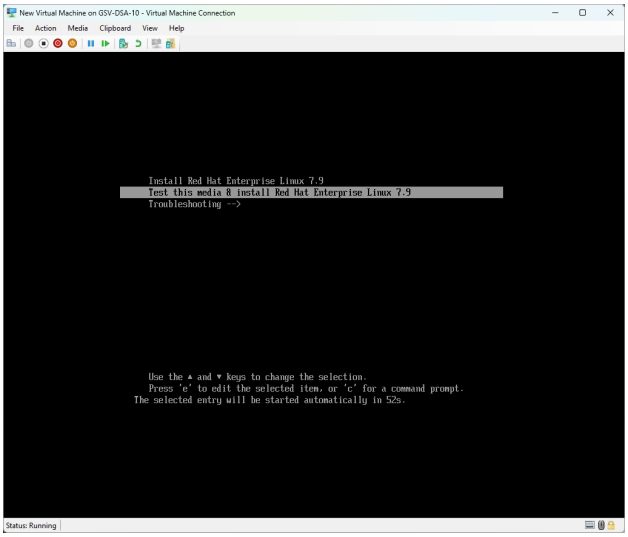


3. SCSI Controller – add DVD drive and choose the RHEL ISO file

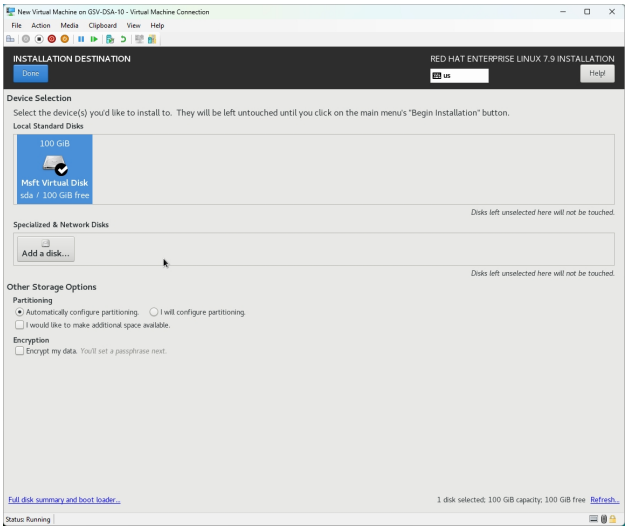




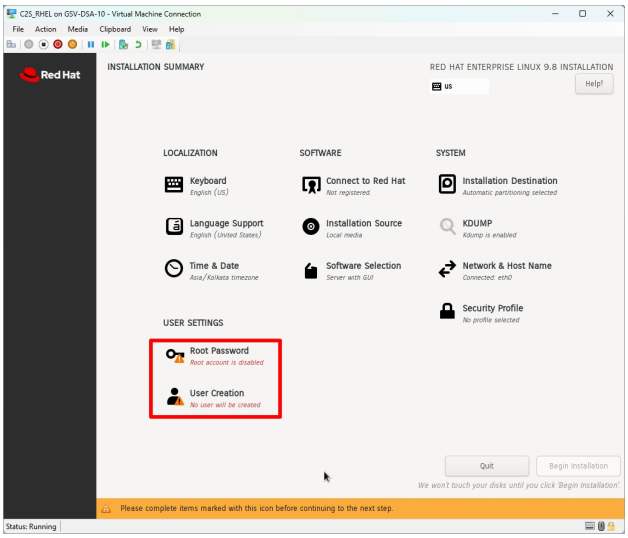
After that close the window and click **Start** and then **Connect**. You should see the following screen:



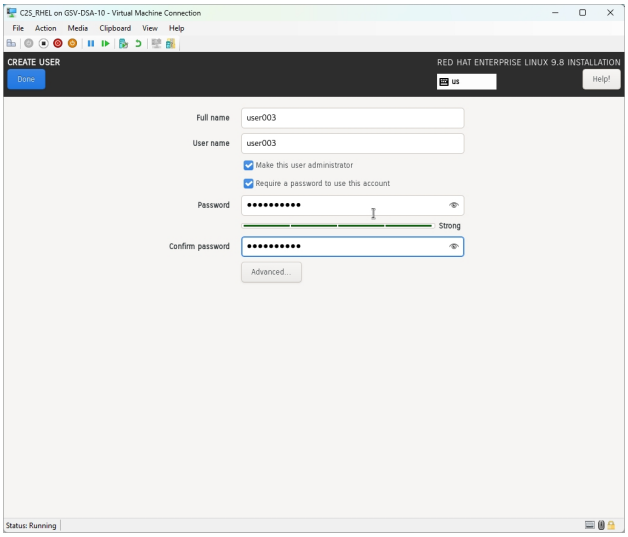
Click "**Install Red Hat Enterprise Linux 8.7**", and follow the setup prompts. For the "Installation Destination" on the Virtual Machine, it should automatically select the Virtual Disk Space (.vhdx) you set while setting up the Virtual Machine.



After all this, just one crucial thing is remaining – the username. Click the following cards in the red box, shown on the image on the left below:

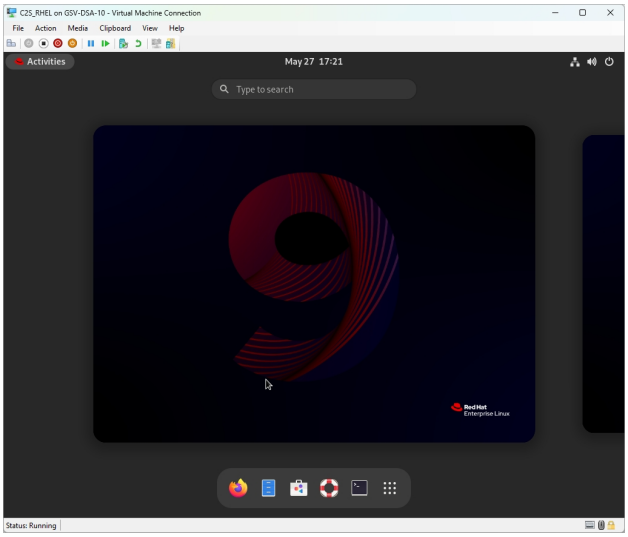


For our use case, we're using a placeholder user named ***user003***. This username has been chosen for a specific reason, as will be explained in elaborate detail later in this chapter:



It is generally wise to keep different root and local user passwords, but since we're setting up a workstation for a Laboratory, we keep the passwords the same.

The installation is ready to begin. After the final installation you will be prompted to reboot the virtual Machine. Finally you should see a screen similar to this: (*this is the GNOME desktop environment*)



2.6 Native Dual Booting RHEL 8.7

Dual booting allows RHEL to run directly on the workstation while preserving an existing operating system on a separate partition. This is especially useful in academic laboratories where a machine may need to support both Linux-based EDA workflows and existing instructional or administrative software. The dual-boot model must be planned carefully, because mistakes during partitioning or bootloader configuration can make the existing operating system temporarily unbootable or, in the worst case, destroy data. Important user files should therefore be backed up before installation begins.

2.6.1 Disk Partitioning Strategy

This is a critical stage in the overall system deployment pipeline and must be approached with precision and caution. Any misconfiguration at this level can result in irreversible data loss, including corruption or

removal of existing operating systems such as Windows.

Disk partitioning is the process of logically dividing a physical storage device (SSD or HDD) into multiple independent regions, known as partitions. Each partition behaves as a separate storage volume and is assigned a specific role within the operating system architecture.

In Linux-based system design, partitioning is not arbitrary. It is a structured layout intended to separate system-critical components, improve stability, simplify recovery, and support long-term maintainability of the environment.

A standard Linux workstation deployment typically includes the following core partitions:

1. EFI Partition – *mounted at /boot/efi/*
   This is a mandatory boot partition on UEFI-based systems. It stores bootloaders, firmware interfaces, and essential startup binaries required for system initialization. Without this partition, the system cannot boot in a modern UEFI environment.

2. Root Partition – *mounted at /*
   This is the primary filesystem hierarchy. It contains the operating system, installed packages, system binaries, configuration files, and most runtime environments. It effectively forms the core of the Linux installation.

3. Swap Partition
   This is a dedicated memory extension area used when physical RAM is fully utilized. It supports memory paging, system hibernation (if enabled), and prevents abrupt failures under memory pressure. While modern systems often use swap files, a dedicated swap partition is preferred in controlled workstation environments for predictability and performance consistency.

You may need to manually create a partition based on your system configuration. Press **Windows Key** and then type **Disk Management**. You should see the following:

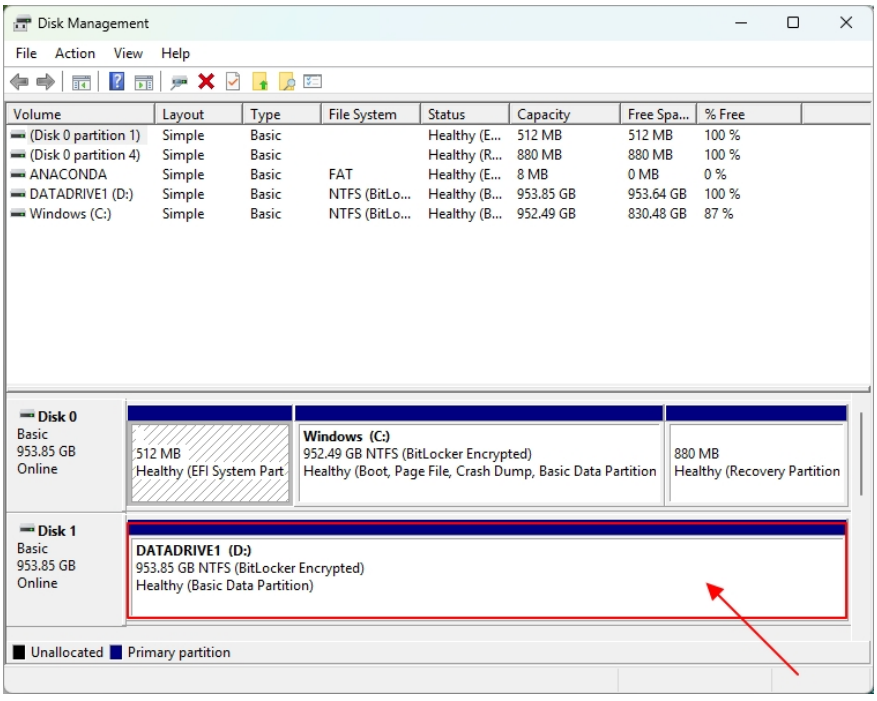


Figure 2.1: Disk Management

From the screenshot, we already have a separate disk — **Disk 1**, with a capacity of **1 TB**. In this case, we can directly use the entire disk without creating additional partitions. However, in the case of a single shared disk, it may be necessary to shrink existing unallocated space in order to create a new partition for installation or data separation.

A detailed walkthrough of this process can be found here: `https://www.youtube.com/watch?v=WvOId5CRmw8`.

For this demonstration, we will skip the partitioning step. The key point of interest here is **disk nomenclature in Linux**. Unlike Windows, which uses drive letters such as `C:` or `D:`, Linux represents storage devices using a structured naming convention. For NVMe SSDs, the naming format is:

`nvmeXnYpZ`

where:

- `X` denotes the NVMe controller number (i.e., the physical interface index),
- `Y` denotes the namespace number (a firmware-defined logical storage unit),
- `Z` denotes the partition number (a software-defined subdivision of the namespace).

Thus, from the observed system output, we can infer the following mappings:

Disk 0 $\rightarrow$ **nvme0n1**
Disk 1 $\rightarrow$ **nvme1n1**

Furthermore, partitions within these disks are represented as extensions of the namespace identifier. For example:

Disk 0, Partition 1 $\rightarrow$ **nvme0n1p1**
Disk 0, Partition 2 $\rightarrow$ **nvme0n1p2**
Disk 1, Partition 1 $\rightarrow$ **nvme1n1p1**

Here, the namespace (`nY`) represents a firmware-defined logical block device exposed by the NVMe controller, while partitions (`pZ`) are operating system-defined subdivisions within that namespace used for filesystem organization and data separation.

> **Note:** The important thing here to remember the name of the disk we want to install RHEL on. In our case, its ***nvme1n1***. The mount-point will be at ***/dev/nvme1n1***

2.6.2 Creating the Installation Media

For dual-booting, we need to create something known as a "**bootable-drive**". The process is straight-forward. All you need are a USB with atleast 8GB storage capacity, and the ISO we downloaded earlier. To create a bootable-drive, you need a flashing tool.

For Windows, the most popular choice is Rufus; can be installed here: `https://rufus.ie/en/`. If you're on Linux, the most popular choice is Balena Etcher; can be installed here: `https://etcher.balena.io/`.

Plug in your USB and make sure it's mounted. Rufus automatically detects it.

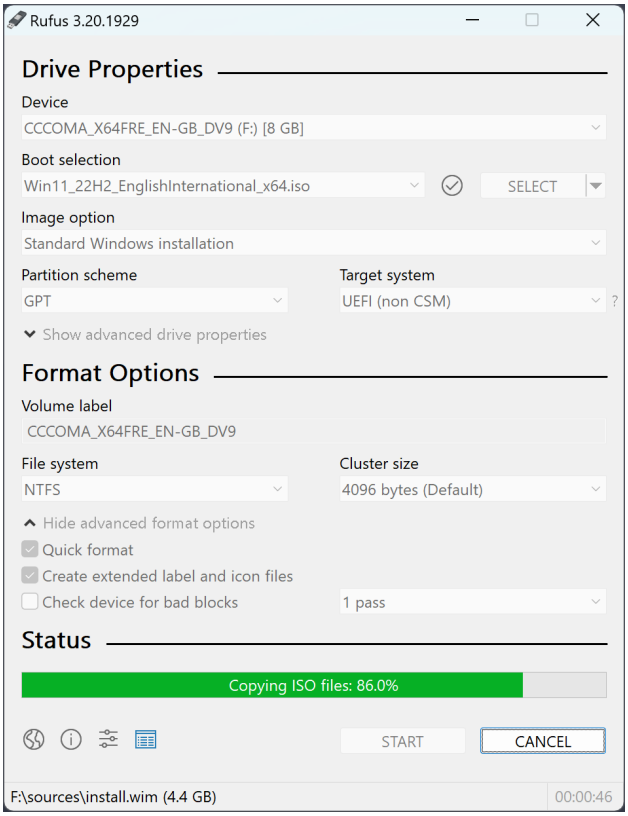


Figure 2.2: Example Screenshot of Rufus

This is a screenshot of the Rufus Tool. The *Device* is usually automatically selected as your USB drive; if not, you can manually select it. Set the *Boot selection* to the RHEL 8.7 ISO we just downloaded. For a modern workstation, use **GPT** as the partition scheme and **UEFI** as the target system. GPT is preferred because current Windows installations and enterprise workstations normally boot in UEFI mode, and using the same boot mode for both Windows and RHEL avoids mixed-mode bootloader issues. Use **MBR** only if the machine is confirmed to be using legacy BIOS boot. On Windows, this can be checked from **System Information** by looking at **BIOS Mode**: choose UEFI if it says *UEFI*, and choose legacy/MBR only if it says *Legacy*.

For the file system, keep the Rufus default unless there is a specific reason to change it. **FAT32** is the safest choice for UEFI boot media because it is directly supported by UEFI firmware; **NTFS** may be used if Rufus requires it because of ISO file-size constraints. Do not choose **EXT4** for the installation USB, since it is not reliably readable by PC firmware during boot and is meant for Linux partitions after installation. Leave the cluster size at **Default**, typically **4096 bytes**, since changing it provides no practical benefit for this installer.

Once this completes, the USB is now ready for installation. Reboot your system, and ***enter the Boot Menu*** by pressing the relevant key. In our case, we're working on an HP workstation, so we're pressing **F9**.

2.6.3 Installation Destination Stage

This is the main step for dual booting, and usually the place where most novices mess up. Most of the steps are the same as the virtualization setup. The only thing we're going to have to do differently here is selecting the disk where the base system will be installed. Click on the "**Installation Destination**" option from the image below, and select the disk we chose in Subsection 2.6.1 – ***nvme1n1***

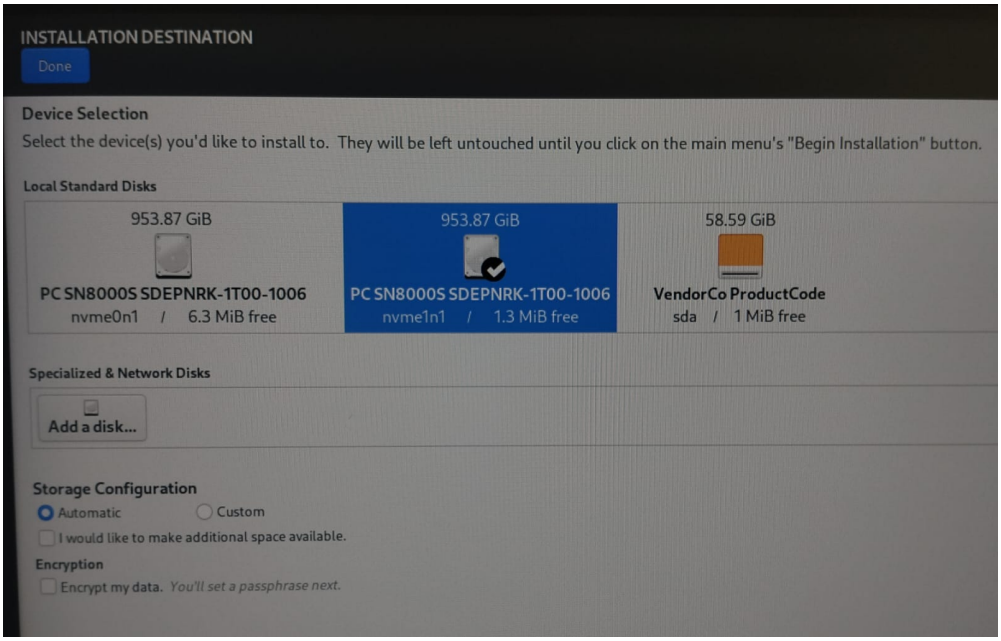


Figure 2.3: Installation Media Selection

Once the base system installs, you should remove the installation media and wait for the system to reboot.

2.7 Chapter Summary

This chapter established the preparation steps required to deploy RHEL 8.7 for Cadence and other EDA workflows. It compared virtualization and native dual booting, noting that virtualization is useful for testing and training, while native installation provides direct access to hardware resources and is therefore preferred for production EDA environments. The chapter also outlined the workstation hardware requirements, documented the target HP workstation, and explained the selection of the RHEL 8.7 DVD installation image for compatibility with enterprise toolchains.

The chapter then reviewed the firmware and boot environment, including the differences between BIOS, UEFI, and Secure Boot. Emphasis was placed on maintaining a consistent boot mode between Windows and RHEL to avoid bootloader and recovery issues. For virtualized deployments, a Hyper-V-based installation procedure was presented, including virtual machine creation, ISO attachment, firmware configuration, Secure Boot settings, and completion of the base operating system installation.

For native dual-boot deployment, the chapter introduced the required Linux partition layout and explained the mapping between Windows disk numbering and Linux NVMe device names to prevent accidental data loss. Finally, a bootable RHEL installation USB was created using Rufus with GPT and UEFI settings appropriate for the target workstation. At this stage, the system is prepared to boot the installer and proceed to disk selection and operating system installation.

03

Post Install Configurations

System identity, licensing, and workstation preparation

In this chapter we will discuss the post-installation configurations required for the deployed workstations. This includes setting up the appropriate hostname and username nomenclature aligned with the license provided by C2S, preparing the system identity used by the Cadence environment, and installing the relevant software packages required for the Cadence Suite.

3.1 Username/Hostname Configuration

After the base RHEL installation is complete, the first administrative task is to standardize the identity of each workstation. In an EDA laboratory environment, usernames and hostnames should not be chosen casually. They are used by the operating system, network services, license checks, remote administration scripts, shared storage mounts, and Cadence startup environments. If different machines follow different naming patterns, the environment becomes difficult to audit and errors in license or path configuration become much harder to diagnose.

According to the license our university has received, the workstation identity should follow the following naming convention:

user0_X @ ECE-X.gsv.ac.in

Here, `user0X` represents the local Linux username assigned to the workstation user, while `ECE-X.gsv.ac.in` represents the fully qualified hostname assigned to that workstation on the department network. The value of `X` should be replaced by the workstation number or seat number allocated by the laboratory administrator. For example, workstation 3 may use the username `user003` and the hostname `ECE-3.gsv.ac.in`. The important point is that the same numbering scheme must be used consistently across the user account, hostname records, license configuration, and any administrative documentation.

This convention gives every machine a predictable identity. A system administrator can immediately identify which physical workstation corresponds to which Linux account and hostname, and the Cadence license environment can be configured against a stable naming scheme. Before installing or launching Cadence tools, confirm that the selected username and hostname match the C2S license documentation and the laboratory workstation inventory. If a machine is renamed later, related configuration files, license references, and shell startup scripts may also need to be updated.

> At the time of writing this document, there are officially upto 100 registered workstations for GSV students under the C2S License. The first 70 systems are reserved for the ECE department, while the remaining 30 are reserved for EE department.

This very first priority is ensuring the groupname and username/hostnames are as per the respective workstations. File ownership depends heavily on the groupname and GID (group ID). In Linux, the default file permissions and ownership records are verified using GID and UID (user ID), hence ensuring they agree,

resolves problems down the line.

On a fresh install, usually the default group ID is set to **1000** for the default user. In our case it is **user003**. You can verify this with the following commands:

terminal

```
id user003  && whoami && hostname
```

The output should display the expected username, User ID (UID), group name, Group ID (GID), and hostname associated with the workstation. These values should be verified before proceeding with the Cadence installation, as several configuration files and environment settings may reference the user account, host identity, or group ownership. If any of these values are incorrect, they should be corrected prior to continuing with the software installation process.

Once the workstation identity has been verified, the next step is to refresh the package metadata and update the operating system packages. On RHEL 9.8, this is typically performed using the `dnf` package manager. The following command updates all installed packages to their latest available versions:

terminal

```
sudo dnf update -y
```

After the update process is complete, the workstation should be rebooted if any kernel, driver, or system library updates have been installed. This ensures that the running operating system matches the updated package state before any Cadence-specific dependencies or environment configurations are applied.

The primary configuration tasks at this stage involve setting up the required environment variables, configuring executable search paths, and ensuring that the appropriate file permissions are assigned to the intended user account. Since multiple workstations must be configured with identical software environments, a system imaging and cloning approach is employed using Clonezilla.

After a workstation image has been cloned using Clonezilla, the target machine inherits several machine-specific attributes from the source workstation, including the username, hostname, home directory structure, and file ownership information. These parameters must be customized before the workstation can be deployed for regular use.

Traditionally, this process would require the administrator to manually perform the following tasks:

- Renaming the user account
- Renaming the primary group
- Updating the user's display name
- Updating the hostname
- Updating /etc/hostname
- Updating /etc/hosts
- Correcting ownership of user directories

Performing these operations manually on every cloned workstation is repetitive and increases the likelihood of configuration errors. To streamline the deployment process and ensure consistency across all laboratory systems, these tasks have been automated through a dedicated C-shell (`csh`) configuration script. The script prompts the administrator for the required workstation-specific parameters and automatically applies the necessary user, group, hostname, and ownership modifications, thereby reducing deployment time and minimizing the risk of human error.

The full script is present in Appendix A

3.2 Installing and Configuring Cadence

3.2.1 Cadence Directory Structure

Cadence expects everything to be present in the `/home/installs` directory, so we need to make that:

terminal

```
sudo mkdir -p /home/installs
```

This is the place where we install all the tools in the Cadence Suite and unzip them.

For a laboratory deployment, the `/home/installs` directory should be treated as a centralized local software repository rather than as an arbitrary extraction location. All Cadence releases, auxiliary setup files, and vendor-provided directory trees should be placed under this common root so that every workstation follows the same filesystem layout. This standardization is important for system imaging, recovery, and later maintenance. If one workstation stores the Cadence tree in a different location, the shell startup file, environment variables, license diagnostics, and teaching scripts may need to be modified only for that machine, defeating the purpose of a common laboratory image.

The Cadence installation archives used for the C2S laboratory setup are obtained from the repository provided for the deployment. Administrators should download the packages from the following source and preserve the original archive structure supplied there:

`https://entupletech-my.sharepoint.com/personal/cadence_support_entuple_com/_layouts/15/onedrive.aspx?id=%2Fpersonal%2Fcadence%5Fsupport%5Fentuple%5Fcom%2FDocuments%2FDATA%2FCadence%5FRHEL%5F8%5Fversion%2FIndividual%5Flinks&ga=1`

In normal workstation preparation, the archive is first downloaded into the user's `~/Downloads` directory. This keeps the downloaded source material separate from the final installation location and allows the administrator to inspect the archive before extraction. After verification, the archive must be extracted directly into `/home/installs/`. No additional wrapper directory should be created during extraction. The contents of the archive should land directly inside `/home/installs/` so that the Cadence directory hierarchy expected by the later environment-variable, C-shell initialization, and license setup steps is preserved.

The exact archive name may differ depending on the release downloaded from the repository. The following example therefore uses a clearly marked placeholder for the archive filename:

terminal

```
cd ~/Downloads
ls -lh

# Replace this with the actual archive downloaded from the repository.
ls -lh <CADENCE_ARCHIVE_NAME>

sudo tar -xf <CADENCE_ARCHIVE_NAME> -C /home/installs/

ls -lh /home/installs
find /home/installs -maxdepth 2 -type d | sort
```

If the downloaded file is provided in a different compressed format, the extraction command should be adjusted accordingly while keeping the destination unchanged. For example, a `.zip` archive should still be extracted with `/home/installs/` as the target directory. The administrative rule remains the same: do not create an extra directory layer between `/home/installs/` and the Cadence tree supplied by the archive.

Preserving the original hierarchy is critical because subsequent configuration steps refer to predictable paths below `/home/installs`. The `.cshrc` file, Cadence environment variables, executable search path, runtime library path, and license diagnostic commands all assume that the installation tree has not been rearranged

manually. A small change made during extraction can later appear as a shell configuration failure or a license startup problem, even though the root cause is simply that the expected directory structure was not preserved. For this reason, the extraction layout should be verified immediately after unpacking the archive and before any environment variables are written.

3.2.2 Environment Variables

Before launching any Cadence tool, the user's shell environment must point to the correct installation directories and license server. If a working `.cshrc` file is already present and contains the required paths, it does not need to be recreated. The important requirement is that the active shell can locate the Cadence executables, libraries, setup files, and license configuration.

The following values should be checked for every workstation:

- The Cadence installation root under `/home/installs`
- The executable paths for Cadence tools, usually the `bin` and `tools/bin` directories inside the Cadence installation
- The runtime library path required by the installed Cadence release
- The license variables, such as `CDS_LIC_FILE` and `LM_LICENSE_FILE`, pointing to the correct license server and port
- Any release-specific setup paths required by the installed Cadence package

The complete C-shell startup configuration is provided in the appendix: B. After confirming or updating the startup file, log out and log back in, or source the file manually, and verify the active environment using:

terminal

```
echo $CDS_INST_DIR
echo $CDS_LIC_FILE
which genus
```

If `which genus` does not return a path under the Cadence installation directory, the executable path has not been configured correctly. The environment variables should be corrected before continuing with license checks or tool validation.

3.2.3 C-Shell Configuration

The C-shell startup file, usually stored as `~/.cshrc`, is one of the most important configuration files in a Cadence workstation deployment. It is read whenever an interactive C-shell session is started, and it defines the environment that the user receives before any EDA tool is launched. In a single-user Linux installation this file may only contain a few aliases or display preferences, but in an EDA laboratory it becomes part of the workstation infrastructure. It controls which tool release is visible, which license server is contacted, which library paths are searched, and whether the shell behaves consistently across cloned systems.

Cadence installations have traditionally relied on C-shell based initialization because many vendor-provided setup scripts, examples, and legacy flows assume `csh` or `tcsh` syntax. Even when the operating system default shell is Bash, laboratory users may still be instructed to start Cadence tools from a C-shell session so that the same environment logic is applied on every workstation. This reduces variation between machines and prevents accidental differences between users who may otherwise maintain separate Bash, Zsh, or graphical terminal profiles. For a C2S laboratory, where the same toolchain must be reproduced across many student systems, this consistency is more important than personal shell preference.

The `.cshrc` file should therefore be treated as a controlled configuration file. Administrators should avoid scattering Cadence variables across multiple unrelated startup files, because such a layout makes troubleshooting difficult after cloning or system repair. A maintainable configuration should clearly define the

installation root, append the required executable directories to the shell path, define any required runtime library locations, and set the licensing variables used by Cadence and FlexLM. If several Cadence releases are installed under `/home/installs`, the selected release should be documented in the file so that future administrators can identify which version is active without inspecting every directory manually.

A minimal administrative check is to confirm that the user account is actually reading the expected C-shell startup file. This should be tested from the same account that will run the tools, not only from the root account, because file ownership, home directory changes, and cloned user profiles can easily produce different results for different users. The following commands may be used after logging in as the workstation user:

terminal

```
echo $shell
ls -l ~/.cshrc
source ~/.cshrc
rehash
```

The `source` command reloads the startup file in the current shell, while `rehash` refreshes the C-shell command lookup table after new executable directories have been added to the path. This is useful during installation because the administrator can edit the startup file and test it immediately without logging out after every change. However, final validation should still be performed in a new login session, because that confirms that the settings are loaded automatically and do not depend on manual commands typed during installation.

The C-shell configuration should not contain workstation-specific values unless those values are intentionally different for each machine. The tool installation path and license configuration should normally be identical across all cloned workstations, while the username and hostname are handled separately during the system identity configuration described earlier in this chapter. This separation improves maintainability: a cloned machine can receive its own hostname and user identity without requiring a different Cadence startup file for every seat in the laboratory.

3.2.4 License Configuration

Cadence tools are protected by network licensing, commonly implemented through the FlexLM licensing system. In this model, the workstation does not independently decide whether a tool may run. Instead, when a Cadence executable starts, it contacts the configured license server and requests the required license feature. The server checks whether the feature exists, whether the request is allowed, and whether enough license tokens are available. Only after this exchange succeeds does the tool proceed to the normal graphical or command-line startup sequence.

License configuration is therefore a core infrastructure dependency, not a cosmetic environment setting. A workstation may have a complete Cadence installation and a correct executable path, but the tools will still fail to launch if the license variable is missing, the server hostname cannot be resolved, the port is incorrect, or the network path to the server is unavailable. In a teaching laboratory this can affect many users at the same time, so license settings should be kept centralized, documented, and reproducible. The same license configuration should be applied to every workstation unless the C2S license documentation explicitly requires separate values for separate systems.

The two variables most commonly involved are `CDS_LIC_FILE` and `LM_LICENSE_FILE`. The Cadence-specific variable points Cadence tools to the license source, while `LM_LICENSE_FILE` is understood by FlexLM-based utilities and may also be used by other licensed software. The value normally follows the form `PORT@LICENSE_SERVER`, where `PORT` is the FlexLM server port and `LICENSE_SERVER` is the hostname or IP address of the license server. The actual value must be taken from the C2S license documentation or from the laboratory license administrator. If the value is not yet known during documentation preparation, it should be left as a clearly marked placeholder rather than replaced by an assumed address:

```csh
setenv CDS_LIC_FILE <PORT>@<LICENSE_SERVER_HOSTNAME>
setenv LM_LICENSE_FILE $CDS_LIC_FILE
```

Hostname resolution should be checked before blaming the Cadence installation itself. If the license variable uses a hostname, that hostname must resolve through DNS or through a controlled entry in `/etc/hosts`. If the laboratory network is segmented, administrators should also confirm that the workstation can reach the license server from the student VLAN or laboratory subnet. Using an IP address can avoid name resolution problems, but it also makes future server migration less transparent. For maintainability, a stable hostname is usually preferable when the network infrastructure supports it correctly.

Licensing failures commonly occur because the startup file was not sourced, the wrong shell was used, the license server name was mistyped, the FlexLM port was changed, or a firewall blocked the connection. Another frequent cause is a mismatch between the installed Cadence release and the features available in the license file. For this reason, license validation should be performed after every installation, after every workstation clone, and after any license server maintenance window. The following checks are useful when the FlexLM utility is available in the active environment:

```terminal
echo $CDS_LIC_FILE
echo $LM_LICENSE_FILE
lmutil lmstat -a -c $CDS_LIC_FILE
```

If `lmutil` is not available directly in the shell path, the administrator should use the copy provided by the installed Cadence release, or replace the command with the site-approved license diagnostic utility. The important point is not the exact utility name, but the validation principle: the workstation must be able to contact the same license source that Cadence will use at runtime.

3.2.5 Verifying Installation

Verification should be performed systematically before a workstation is released for student use. The goal is not merely to prove that one command opens once, but to confirm that the deployed machine matches the intended laboratory baseline. This includes the operating system identity, the Cadence installation directory, the C-shell startup configuration, the executable search path, the runtime library environment, and access to the license server. A structured validation step is especially important after Clonezilla deployment, because cloned machines can inherit paths and user configuration from the source image while still requiring workstation-specific identity changes.

The first validation step is to check that the environment visible to the user is the same environment described in the configuration files. This must be done from a fresh terminal session as the intended user account. Administrators should avoid relying only on commands run immediately after manual sourcing, because that can hide startup errors that will reappear when students log in normally. The following commands provide a basic view of the active shell and Cadence environment:

```terminal
whoami
hostname
echo $shell
echo $CDS_INST_DIR
echo $CDS_LIC_FILE
echo $LM_LICENSE_FILE
```

The values should match the workstation identity convention and the Cadence configuration approved for the laboratory. Empty output for a required variable usually indicates that the startup file was not read, the user is not running the expected shell, or the variable was defined in a file that is not part of the current login

sequence. Incorrect output is more serious than empty output, because it may point the user to an outdated release or to an obsolete license server without making the failure obvious at first.

The next step is to confirm executable visibility. Cadence tools are large application suites, and their front-end commands must be reachable through the C-shell path. If the shell cannot find the executable, the installation may still exist on disk, but the user environment is not ready. The administrator should check both command discovery and version reporting where the tool supports it:

```terminal
which genus
which xrun
which innovus

genus -version
xrun -version
innovus -version
```

Not every laboratory installation will include every Cadence product. A missing command is therefore not automatically an error if that product was not installed or was not included in the C2S entitlement. However, every tool expected for the course flow should resolve to a path under the approved Cadence installation directory, and its version should correspond to the documented release. If different machines report different versions, the clone image or startup file should be reviewed before the laboratory is used for instruction.

License access should be validated after executable visibility, because a tool may be installed correctly but still fail during feature checkout. If FlexLM diagnostics are available, the license server can be queried before opening a full graphical application:

```terminal
lmutil lmstat -a -c $CDS_LIC_FILE
```

Finally, the administrator should perform a practical launch test using at least one graphical tool and one command-line or batch-capable tool from the expected design flow. This catches problems that simple path checks may miss, such as missing shared libraries, display forwarding issues, incompatible system packages, or permission problems in the user's working directory. The workstation should only be considered ready when these checks succeed from the normal student account without requiring root privileges.

3.2.6 Launching Cadence Tools

Cadence tools should be launched from a prepared user environment, not from an arbitrary terminal state. In normal laboratory use this means that the user logs in to the workstation, opens a C-shell-compatible terminal session, and starts the required tool from a project working directory. The working directory matters because many Cadence applications create local run files, logs, lock files, simulation outputs, or design database directories during startup. Launching tools from a temporary or root-owned directory can create permission problems that are difficult for students to diagnose later.

During the first execution, the tool performs several checks before the user reaches the main interface. It reads the shell environment, locates the installation tree, loads shared libraries, checks release-specific setup files, contacts the license server, and initializes user configuration data. A delay during the first launch is therefore not unusual, especially on a newly cloned workstation or after a user profile has been created. However, repeated failures at this stage usually indicate an infrastructure problem: an incorrect path, a missing library, an unavailable license feature, or an environment mismatch between the selected release and the active shell.

A typical launch test for the synthesis environment is performed with `genus`. The administrator should run it from a writable directory owned by the user, observe whether the application starts without license errors, and then close it normally after the startup sequence is complete:

terminal

```
mkdir -p ~/cadence_test
cd ~/cadence_test
genus &
```

Successful startup confirms more than the presence of the `genus` executable. It confirms that the shell path is active, the required runtime libraries can be loaded, the license server is reachable, and the release selected in the shell matches the installation tree on disk. If the tool reports license warnings or missing setup files, the workstation should not be handed over as ready until those messages are understood. Warning messages that appear harmless during installation often become recurring support issues once a full class begins using the machines.

Digital design and implementation tools may also be checked from the command line. Version queries are useful for confirming the active release without starting a full project, while a later course-specific flow can be used for deeper validation. The following commands provide a lightweight launch-level check for commonly used Cadence tools, subject to the products actually installed under the C2S deployment:

terminal

```
xrun -version
genus -version
innovus -version
```

A successful launch or version response should be recorded as part of the workstation handover checklist. In a laboratory setting, this record helps distinguish between installation problems and later user-level project errors. Once the major tools start from the normal user account, the expected license source is reachable, and the reported versions match the documented release, the Cadence environment can be considered ready for instructional use.

Part II

RTL DESIGN FUNDAMENTALS

Introduction to Verilog and basic RTL design

04

Introduction to Verilog

System identity, licensing, and workstation preparation

4.1 Why Hardware Description Languages Exist

As digital systems grew beyond small collections of gates, schematic-only design became unsuitable for serious integrated-circuit development. A modern chip must be described, simulated, revised, and verified long before it reaches physical implementation, and this requires a representation that EDA tools can process consistently. **Hardware Description Languages (HDLs)** provide this representation. They allow engineers to describe logic, registers, interconnections, timing intent, and hierarchy in text, while still representing hardware rather than software instructions executed by a processor.

Verilog is one of the primary HDLs used for digital design and verification. It can describe simple combinational circuits, sequential elements, counters, multiplexers, and complete RTL blocks that later pass through simulation, synthesis, and implementation. Its main value in this flow is that the same design can be organized hierarchically, tested before fabrication, and reused across larger systems. This chapter introduces the basic Verilog program structure, modules, ports, data types, constants, and operators needed for the RTL design and verification work used in later chapters.

4.2 Verilog Program Structure

A Verilog source file is organized around one or more modules, and each module acts as a self-contained hardware block with its own ports, declarations, and internal logic. This structure is not a stylistic convention; it is what allows EDA tools to parse the design, elaborate the hierarchy, and determine which parts of the description are meant to behave as combinational logic, sequential logic, or hierarchical instances of other blocks. A complete Verilog design therefore begins with the module header, continues with the declaration of ports and internal objects, and ends with one or more behavioral or structural descriptions that define how the block operates.

The general pattern is consistent across small laboratory examples and larger RTL blocks. The module header names the design unit and lists its external interface. The body then defines whether each port is an input, output, or inout, assigns widths where required, declares nets and variables, and introduces continuous assignments or procedural blocks. A module may also contain parameters, submodule instances, tasks, functions, and generate constructs when the design grows in complexity. The following skeleton shows the layout used throughout this manual:

```verilog
module module_name (port_list);
    // port directions and widths
    // internal declarations
```

```
    // continuous assignments
    // procedural blocks
endmodule
```

In practice, this structure allows the same description to be read by both simulator and synthesis tools. Simulation uses the procedural and concurrent statements to model behavior over time, while synthesis extracts the logic implied by the supported constructs. For that reason, the organization of the code matters as much as the syntax itself. A clean module layout makes a design easier to review, easier to debug, and easier to hand over to another engineer or laboratory administrator later in the flow.

4.3 Modules

The module is the basic unit of composition in Verilog. Every hardware description is built from one or more modules, whether the design represents a simple gate-level block or a complete subsystem with multiple submodules. A module may be reused many times within a larger design, which is why it is useful to think of it as a hardware component rather than as a software procedure. When one module instantiates another, the relationship is fixed at elaboration time; the resulting hierarchy mirrors the structure of the hardware that will be simulated or synthesized.

The module body can include instances of other modules, primitive gates, continuous assignments, tasks, functions, and procedural blocks. A small module may contain only a few lines of declaration and a single assignment, while a larger module may include several nested blocks and many internal signals. The important point is that the interface remains explicit. For example, a full adder can be described with only a few statements:

verilog

```
module full_adder(A, B, Cin, S, Cout);
    input A, B, Cin;
    output S, Cout;

    assign {Cout, S} = A + B + Cin;
endmodule
```

The same block may later be reused as part of a larger arithmetic unit without changing its interface. This reuse is one of the practical reasons Verilog became central to RTL design. It allows an engineer to define a small verified building block once, then connect that block into a wider system by instantiating it repeatedly. The module boundary therefore serves both as a design boundary and as a documentation boundary.

4.4 Ports

Ports define the external interface of a module. They are the connection points through which the module exchanges signals with the rest of the design, and they must be declared explicitly as input, output, or inout. A port list alone is not enough to define the signal type; the direction and, when necessary, the data type and bit range must also be stated inside the module body. In many cases a port is treated as a wire by default, but wider buses and registered outputs require additional declaration.

The port list becomes especially important when a module is parameterized. Widths can be described using parameters so that the same source file may be reused for different data widths without rewriting the logic. This is common in laboratory designs, where a small example may later be scaled to a wider bus for experimentation. The syntax below shows the basic pattern:

verilog

```
module module_name
    #(parameter WORD = 32)
```

```
    (addr, sum, a, b, sel, data_bus);

    input  [WORD-1:0] addr;
    output reg signed [32:1] sum;
    input  a, b, sel;
    input  [0:15] data_bus;
endmodule
```

The declaration order matters when the design is read by tools and by human reviewers. Signals in the port list should be easy to map back to the declarations, and the direction of each signal should be obvious before the body of the module is examined. A clear port definition is one of the simplest ways to keep a Verilog project maintainable as it grows.

4.5 Nets and Registers

Verilog separates physical connections from storage elements. Nets, such as `wire`, are used to model connections between components, while registers such as `reg` are used when the value must be held or assigned inside procedural code. This distinction is important because it reflects the hardware interpretation of the source. A net is driven by some source and used as a connection in the design, whereas a register is typically updated by an `always` or `initial` block during simulation.

Verilog also models four-valued logic, which is necessary when a design is not fully driven or when a signal cannot yet be resolved during simulation. The values are `0`, `1`, `x`, and `z`. The `z` state represents a high- impedance or undriven condition, while `x` represents an unknown or unresolved value. These states are useful when checking reset behavior, tri-state buses, or incomplete initialization. The following examples show the basic declaration style used in practice:

verilog

```
wire A, X, Y, Z, Cin, Cout;
wire [3:0] B, S;
wire [15:0] C;

reg q_out;
reg [7:0] counter;
```

Nets are not assigned inside procedural blocks, and registers are not used as direct continuous-connection targets in the same way as a wire. That separation keeps the model consistent with the intended hardware behavior and helps the simulator distinguish between continuous hardware connections and values that are updated under procedural control.

4.6 Numbers and Constants

Verilog numbers are written using a compact notation that encodes the size, radix, and value in a single literal. The general form is `<size>'<radix><value>`, where the radix indicates binary, octal, decimal, or hexadecimal representation. This syntax is used throughout RTL coding because it makes the intended width explicit and avoids ambiguity about sign extension or default integer size. The standard radix characters are `b`, `o`, `d`, and `h`.

Sized constants are preferable whenever the width matters, especially in module interfaces, state machines, and testbench stimulus. Verilog also allows unknown and high-impedance digits inside constants, which is useful when modeling incomplete bus values or uninitialized conditions during simulation. The examples below match the style used in the notes:

```verilog
4'b1010
8'h3F
16'd255
4'b01xx
1'bz
```

Constants are not simply a syntactic detail. In a hardware description, they determine how the simulator interprets a value and how a synthesis tool treats a mask, width, or reset state. A value written without an explicit size may behave differently from the same value written with a fixed bit width, so laboratory code should state widths clearly whenever the design depends on them.

4.7 Operators

Verilog operators follow the same broad categories used in digital logic, but they are applied directly to hardware signals rather than to software variables. Bitwise operators act on individual bits, logical operators act on the truth value of an expression, shift operators move data left or right, and the conditional operator provides a compact form of multiplexing. Understanding this division is essential because a mistake in operator choice can change the synthesized hardware even when the simulation appears to run.

Bitwise operators operate on each bit of their operands. Unary reduction operators, by contrast, collapse a vector into a single result by combining all bits together. Logical operators return a true or false result and are typically used in conditions. The shift operators `«` and `»` move the bit pattern by a fixed number of positions, while the conditional operator `?:` is commonly used to express a multiplexer in a compact form. The basic usage is illustrated below:

```verilog
assign A = X | (Y & ~Z);
assign B = &data_bus;
assign C = |mask_bus;
assign D = sel ? in1 : in0;

a = 4'b1010;
d = a >> 2;
c = a << 1;
```

In laboratory RTL, these operators are used constantly because they map directly onto the hardware view of the design. A conditional expression often describes the same logic that would be drawn as a multiplexer in a schematic, while bitwise and reduction operators are useful for masks, parity checks, and grouped control logic. The operator chosen in the source should therefore match the intended hardware structure, not merely the result visible in a quick simulation.

4.8 Laboratory Exercise – Half Adder

4.8.1 Objective

The purpose of this exercise is to design, simulate, and verify a Half Adder using Verilog HDL. The exercise introduces a complete but minimal digital design flow, beginning with the RTL description and continuing through testbench development, compilation, simulation, and waveform inspection in the Cadence environment.

A Half Adder is a suitable starting point because it is small enough to understand in a single session, yet complete enough to demonstrate the relationship between Boolean equations, Verilog code, stimulus generation, and simulation results. The same workflow used here will be used again in later chapters for larger combinational and sequential designs.

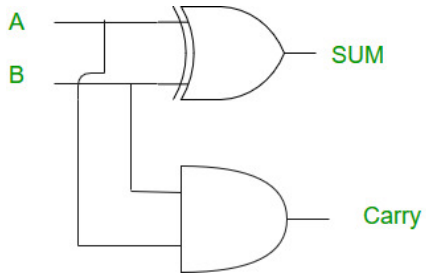
4.8.2 Theory

A Half Adder is a combinational circuit that adds two one-bit inputs, `A` and `B`, and produces two one-bit outputs: the *Sum* and the *Carry*. The Sum bit represents the least significant result of the addition, while the Carry bit indicates whether both inputs were high at the same time. Since the circuit has no storage element, its outputs depend only on the current input combination.

In Boolean form, the Half Adder is expressed as:

$$\text{Sum} = A \oplus B$$

$$\text{Carry} = A \cdot B$$



The Sum output is obtained using the exclusive-OR operation because the result is high only when exactly one input is high. The Carry output is obtained using the AND operation because a carry occurs only when both inputs are asserted simultaneously.

4.8.3 Truth Table

The truth table below lists all possible input combinations for a Half Adder and the corresponding Sum and Carry outputs. This table serves as the reference against which the Verilog model and the testbench results are checked.

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Table 4.1: Truth table of the Half Adder.

4.8.4 Verilog Implementation

The Verilog description of a Half Adder is intentionally short, but it should still follow the same discipline used for larger RTL blocks. The module name identifies the design unit, the port list exposes the external interface, and the continuous assignments map directly to the Boolean expressions derived from the truth table. This makes the source easy to read and easy to verify against the expected hardware behavior.

```verilog
module half_adder(A, B, Sum, Carry);
    input A, B;
    output Sum, Carry;

    assign Sum = A ^ B;
    assign Carry = A & B;
endmodule
```

The module declaration names the design and defines the port order. The input declarations identify the two one-bit operands. The output declarations identify the result signals that will be driven by the logic inside the module. The `assign` statement for `Sum` implements the exclusive-OR function, while the `assign` statement for `Carry` implements the AND function. The `endmodule` keyword closes the design unit and allows it to be instantiated inside a testbench or a larger circuit.

4.8.5 Testbench

A testbench is required because the design itself does not generate stimuli. The Half Adder must be exercised with all valid input combinations so that the simulation can confirm both outputs against the truth table. In a laboratory setting, the testbench also establishes a repeatable procedure for running the design inside the simulator, which is essential when the same flow must later be repeated for larger circuits.

```verilog
`timescale 1ns/1ps

module half_adder_tb;
    reg A, B;
    wire Sum, Carry;

    half_adder dut (
        .A(A),
        .B(B),
        .Sum(Sum),
        .Carry(Carry)
    );

    initial begin
        $monitor($time, " A=%b B=%b Sum=%b Carry=%b", A, B, Sum, Carry);

        A = 0; B = 0; #10;
        A = 0; B = 1; #10;
        A = 1; B = 0; #10;
        A = 1; B = 1; #10;

        $finish;
    end
endmodule
```

The stimulus sequence applies each of the four possible input combinations in turn, with a short delay between each state so that the waveform viewer can capture the transitions clearly. The `$monitor` statement records the active values during simulation, making it easy to compare the observed output against the truth table. Once the final input combination has been applied, the simulation terminates with `$finish`.

4.8.6 Simulation Procedure

The simulation workflow should be treated as a standard laboratory procedure rather than as an ad hoc sequence of tool clicks. A clean project directory, a correct set of source files, and a known simulation entry point make it much easier to reproduce results later. The exact window layout may vary slightly between Cadence releases, but the workflow remains the same.

1. Create a new Cadence project or working directory for the exercise.
2. Add the design file containing the Half Adder RTL description.
3. Add the testbench file and ensure it references the design module correctly.
4. Compile the design and confirm that no syntax errors are reported.
5. Run the simulation and allow the testbench to apply all input combinations.
6. Open the waveform viewer and inspect the Sum and Carry signals.
7. Save the simulation session if the laboratory workflow requires a record of the results.

In practice, the compilation stage confirms that both files are readable by the simulator and that the module names, port names, and signal declarations agree. The simulation stage then checks that the testbench drives the design in the intended order. The waveform viewer is used at the end because it provides the clearest visual confirmation that the output transitions match the truth table.

4.8.7 Expected Results

The expected waveform is straightforward. When both inputs are low, both outputs remain low. When exactly one input is high, the Sum output becomes high while the Carry output remains low. When both inputs are high, the Sum output returns low and the Carry output becomes high. This is the defining behavior of a Half Adder and should be visible directly in the waveform viewer.

TODO: Insert Half Adder waveform screenshot

Figure 4.1: Expected Half Adder waveform showing Sum and Carry behavior for all input combinations.

The waveform should therefore show `Sum` following the XOR of `A` and `B`, while `Carry` follows the AND of the same inputs. If the observed outputs do not match the truth table, the design file and testbench should be checked first, followed by the simulation setup and signal connections.

4.8.8 Conclusion

This exercise demonstrates the complete Verilog verification flow on a simple combinational circuit. The Half Adder is small, but it introduces the essential sequence used throughout digital design: derive the logic, write the RTL, apply test stimulus, simulate the model, and verify the waveform against the expected behavior. That procedure will be reused in later chapters when the designs become larger and the simulation setup becomes more involved.

By completing the exercise, the reader gains a practical starting point for working with Cadence tools in a reproducible laboratory environment. The same method of checking the Boolean equations, validating the truth table, and confirming the waveform response will remain useful when the designs move from simple combinational logic to sequential circuits and larger RTL blocks.

05

Combinational Logic Design

System identity, licensing, and workstation preparation

5.1 Continuous Assignments

Combinational circuits produce outputs that depend only on the present values of their inputs. In Verilog, the most direct way to describe this kind of hardware is by using a continuous assignment. A continuous assignment represents a permanent connection from an expression to a net, similar to a wire driven by a gate or a small network of gates in a schematic.

The keyword used for this purpose is `assign`. It is written inside a module, but outside procedural blocks such as `initial` and `always`. The left-hand side of a continuous assignment is normally a net such as `wire`, and the right-hand side is an expression constructed from input signals, constants, and Verilog operators. Whenever any signal on the right-hand side changes, the expression is re-evaluated and the driven output is updated by the simulator. This continuous evaluation behavior matches the physical behavior of combinational logic, where gate outputs respond to changes at their inputs after propagation delay.

The simplest example is a two-input AND gate:

```verilog
module and_gate(A, B, Y);
    input A, B;
    output Y;

    assign Y = A & B;
endmodule
```

The output `Y` is driven by the bitwise AND of `A` and `B`. If either input changes during simulation, `Y` is automatically updated. No explicit event control or procedural statement is required because the assignment is concurrent with the rest of the module.

An OR gate follows the same pattern:

```verilog
module or_gate(A, B, Y);
    input A, B;
    output Y;

    assign Y = A | B;
endmodule
```

In this case, the operator `|` maps to a bitwise OR function. For one-bit signals it behaves as the OR gate used in a logic diagram. For vectors, the same operator applies bit by bit across corresponding positions of the operands.

Continuous assignments may also contain compound expressions. The following module drives three outputs from the same set of inputs:

```verilog
module simple_logic(A, B, C, Y1, Y2, Y3);
    input A, B, C;
    output Y1, Y2, Y3;

    assign Y1 = A & B;
    assign Y2 = A | C;
    assign Y3 = (A & B) | (~C);
endmodule
```

Each `assign` statement describes a separate concurrent hardware connection. The expression for `Y3` contains an AND operation, an inversion, and an OR operation; a synthesis tool interprets this as the corresponding combinational network. The order of the statements in the source does not imply execution order. All continuous assignments are active at the same time, which is why they are commonly used for RTL descriptions of combinational logic.

5.2 Dataflow Modeling

Dataflow modeling describes a circuit by specifying how data values are transformed as they move from inputs to outputs. Instead of instantiating individual gates, the designer writes Boolean or arithmetic expressions that define the required output behavior. The synthesis tool then constructs the equivalent logic hardware from those expressions.

This is different from a structural description. In structural modeling, the source code explicitly connects gates or lower-level modules, much like a textual schematic. In dataflow modeling, the source code focuses on the relationship between signals. For example, a designer may write the expression for an output directly rather than listing every AND, OR, and NOT gate needed to realize it. This makes dataflow modeling concise and well suited to RTL design, especially for combinational blocks whose behavior is naturally expressed by equations.

A simple dataflow example is the implementation of an inverter and a two-input gate:

```verilog
module basic_dataflow(A, B, Y_not, Y_and);
    input A, B;
    output Y_not, Y_and;

    assign Y_not = ~A;
    assign Y_and = A & B;
endmodule
```

The unary operator `~` describes bitwise inversion, while `&` describes bitwise AND. Since all signals are one bit wide in this example, each operator maps directly to the corresponding logic gate.

More useful combinational functions are usually written as expressions that combine several operators:

```verilog
module combinational_network(A, B, C, D, F);
    input A, B, C, D;
    output F;

    assign F = (A & B) | (C & ~D);
endmodule
```

The output `F` becomes high when `A` and `B` are both high, or when `C` is high while `D` is low. Parentheses are used to make the intended grouping clear. Although Verilog has defined operator precedence rules, explicit grouping is preferred in laboratory code because it reduces ambiguity during review.

Dataflow descriptions also work naturally with buses. In the next example, the same operation is applied across four bits:

```verilog
module vector_logic(A, B, mask, Y);
    input [3:0] A, B, mask;
    output [3:0] Y;

    assign Y = (A & mask) | (B & ~mask);
endmodule
```

Here, `A`, `B`, `mask`, and `Y` are four-bit vectors. The bitwise operators act independently on each bit position. This style is common in RTL because it describes a repeated logic pattern without writing a separate equation for every bit.

The following table summarizes the operators most frequently used in dataflow models of combinational logic.

Operator	Name	Typical Hardware Interpretation
`~`	Bitwise NOT	Inverter on each bit of the operand
`&`	Bitwise AND	AND gate for each bit position
`|`	Bitwise OR	OR gate for each bit position
`^`	Bitwise XOR	XOR gate for each bit position
`+`	Addition	Combinational adder network
`?:`	Conditional	Multiplexer selection logic

Table 5.1: Common Verilog operators used in dataflow descriptions.

Dataflow modeling is widely used because it is compact, synthesizable, and close to the Boolean equations used during circuit design. Its main limitation is that very complex expressions can become difficult to read and debug. For larger designs, dataflow statements are often combined with hierarchy: small combinational functions are written as equations, and larger systems are built by instantiating those verified modules.

5.3 Conditional Operators

The conditional operator provides a compact way to select one of two expressions based on a condition. It is also called the ternary operator because it uses three operands: the condition, the expression selected when the condition is true, and the expression selected when the condition is false. Its general form is:

```verilog
condition ? expression1 : expression2;
```

If `condition` evaluates to true, the result of the whole expression is `expression1`. If the condition evaluates to false, the result is `expression2`. In synthesizable combinational RTL, this operator is normally interpreted as selection logic. For one-bit selection, that hardware is a multiplexer.

A simple output selection example is shown below:

```verilog
module conditional_example(A, B, sel, Y);
    input A, B, sel;
    output Y;

    assign Y = sel ? A : B;
endmodule
```

When `sel` is high, `Y` follows `A`. When `sel` is low, `Y` follows `B`. The statement is continuously evaluated, so any change on `sel`, `A`, or `B` is reflected at the output.

The selected expressions may also be logic expressions rather than simple signals:

```verilog
module conditional_logic(A, B, C, mode, Y);
    input A, B, C, mode;
    output Y;

    assign Y = mode ? (A & B) : (A | C);
endmodule
```

In this example, `mode` selects between two different logic functions. When `mode` is high, the output is the AND of `A` and `B`. When `mode` is low, the output is the OR of `A` and `C`. This is useful when a circuit must route one of several possible results to a common output.

Conditional operators are also useful for signal routing on buses:

```verilog
module bus_select(data_a, data_b, sel, data_out);
    input [3:0] data_a, data_b;
    input sel;
    output [3:0] data_out;

    assign data_out = sel ? data_a : data_b;
endmodule
```

The selection is applied to the complete four-bit vector. Hardware synthesis produces four parallel one-bit selection circuits controlled by the same `sel` signal. This pattern is the natural Verilog form of a multiplexer, which is developed in detail in the next section.

5.4 Multiplexers

A multiplexer, commonly abbreviated as MUX, is a combinational circuit that selects one input from several available data inputs and forwards the selected value to a single output. The selection is controlled by one or more select lines. Multiplexers are fundamental building blocks in digital systems because they implement controlled routing: one data source is chosen while the others are ignored.

A multiplexer has three main parts: data inputs, select inputs, and an output. The number of select lines determines how many data inputs can be addressed. With one select line, two inputs can be selected. With two select lines, four inputs can be selected. In general, `n` select lines can address `2^n` input choices. Since the output is determined only by the current values of the data and select inputs, a multiplexer is a combinational circuit and is well suited to continuous assignments and dataflow modeling.

5.4.1 2:1 Multiplexer

A 2:1 multiplexer has two data inputs, one select input, and one output. When the select input is low, the output follows `I0`. When the select input is high, the output follows `I1`. The complete truth table is shown in Table 5.2. The selected input determines the output value, while the unselected input has no effect on the output.

The Boolean expression for a 2:1 multiplexer is:

$$Y = \overline{sel} \cdot I0 + sel \cdot I1$$

The first product term enables `I0` when `sel = 0`, and the second product term enables `I1` when `sel = 1`.

sel	I0	I1	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Table 5.2: Full truth table of a 2:1 multiplexer.

The conditional operator gives the clearest Verilog description of this selection:

```verilog
module mux2_conditional(I0, I1, sel, Y);
    input I0, I1, sel;
    output Y;

    assign Y = sel ? I1 : I0;
endmodule
```

The expression reads directly from the truth table. If `sel` is high, `I1` is selected; otherwise `I0` is selected. This is a dataflow description because the output is defined by an expression rather than by explicit gate instances.

The same 2:1 multiplexer can also be written using its Boolean equation:

```verilog
module mux2_dataflow(I0, I1, sel, Y);
    input I0, I1, sel;
    output Y;

    assign Y = (~sel & I0) | (sel & I1);
endmodule
```

This version exposes the underlying AND-OR structure. The term `~sel & I0` enables `I0` when the select signal is low, and the term `sel & I1` enables `I1` when the select signal is high. The OR operation combines the enabled path into the output.

5.4.2 4:1 Multiplexer

A 4:1 multiplexer extends the same idea to four data inputs. It requires two select lines because four choices must be encoded. The select input is usually represented as a two-bit vector `sel[1:0]`, where `sel[1]` is the most significant select bit and `sel[0]` is the least significant select bit. The output follows exactly one of the four data inputs for each select combination.

The Boolean expression for a 4:1 multiplexer is:

$$Y = \overline{sel[1]} \cdot \overline{sel[0]} \cdot I0 + \overline{sel[1]} \cdot sel[0] \cdot I1 + sel[1] \cdot \overline{sel[0]} \cdot I2 + sel[1] \cdot sel[0] \cdot I3$$

Each product term corresponds to one decoded select condition. Only the selected data input is allowed to contribute to `Y`; the other data inputs are masked by the select logic.

sel[1]	sel[0]	I0	I1	I2	I3	Y
0	0	0	X	X	X	0
0	0	1	X	X	X	1
0	1	X	0	X	X	0
0	1	X	1	X	X	1
1	0	X	X	0	X	0
1	0	X	X	1	X	1
1	1	X	X	X	0	0
1	1	X	X	X	1	1

Table 5.3: Full truth table of a 4:1 multiplexer.

A compact dataflow implementation can be written by nesting conditional operators:

```verilog
module mux4_conditional(I0, I1, I2, I3, sel, Y);
    input I0, I1, I2, I3;
    input [1:0] sel;
    output Y;

    assign Y = sel[1] ? (sel[0] ? I3 : I2) : (sel[0] ? I1 : I0);
endmodule
```

The outer conditional checks `sel[1]`. If `sel[1]` is low, the selection is between `I0` and `I1`; if `sel[1]` is high, the selection is between `I2` and `I3`. The inner conditional in each branch then uses `sel[0]` to choose the final input. This corresponds to a tree of 2:1 multiplexers.

The same function can be expressed using minterms:

```verilog
module mux4_dataflow(I0, I1, I2, I3, sel, Y);
    input I0, I1, I2, I3;
    input [1:0] sel;
    output Y;

    assign Y = (~sel[1] & ~sel[0] & I0) |
               (~sel[1] &  sel[0] & I1) |
               ( sel[1] & ~sel[0] & I2) |
               ( sel[1] &  sel[0] & I3);
endmodule
```

This form is longer, but it makes the Boolean decoding of the select lines explicit. Each product term corresponds to one row of the truth table, and only one data input is enabled for each binary value of `sel`.

Multiplexers are used extensively in digital systems. They select operands for arithmetic units, choose between register outputs, route data onto internal buses, select next-state values in control logic, and implement configurable datapaths. The conditional operator and dataflow assignments introduced earlier provide the most direct Verilog notation for these functions.

5.5 Full Adder Design

A Half Adder adds two one-bit inputs and produces a Sum and Carry output. It is useful for introducing combinational design, but it cannot accept a carry from a lower-order bit position. Multi-bit addition requires that each bit position include the carry generated by the previous position. This is the purpose of a Full Adder.

A Full Adder has three one-bit inputs: `A`, `B`, and `Cin`. The input `Cin` represents the carry-in from a previous stage. It has two outputs: `Sum`, which is the one-bit addition result, and `Cout`, which is the carry-out passed to the next stage. The complete behavior is shown in Table 5.4.

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 5.4: Truth table of the Full Adder.

From the truth table, the Sum output is high when an odd number of inputs are high. The Carry output is high when at least two of the three inputs are high. The Boolean equations are:

$$\text{Sum} = A \oplus B \oplus Cin$$

$$\text{Cout} = (A \cdot B) + (B \cdot Cin) + (A \cdot Cin)$$

The carry equation shows why the Full Adder is suitable for multi-bit arithmetic. A carry is produced either because `A` and `B` are both high, or because one operand bit is high while the incoming carry is also high. In a ripple carry adder, this `Cout` becomes the `Cin` of the next more significant Full Adder stage. This stage-to-stage dependency is called carry propagation.

A Full Adder can also be understood as two Half Adders and an OR gate. The first Half Adder adds `A` and `B`, producing an intermediate sum and carry. The second Half Adder adds the intermediate sum to `Cin`.

The two carry outputs are then ORed to produce `Cout`. In RTL, the same behavior can be written more directly using continuous assignments:

```verilog
module full_adder(A, B, Cin, Sum, Cout);
    input A, B, Cin;
    output Sum, Cout;

    assign Sum = A ^ B ^ Cin;
    assign Cout = (A & B) | (B & Cin) | (A & Cin);
endmodule
```

The module declaration defines the Full Adder as a reusable design block. The input declaration lists the three one-bit operands, including the carry-in. The output declaration lists the two result signals. The first continuous assignment implements the three-input XOR function for `Sum`. The second continuous assignment implements the carry equation using AND terms for the possible carry-generating input pairs and an OR operation to combine them. Finally, `endmodule` closes the module so that it can be compiled independently or instantiated inside a larger design.

An alternative compact implementation uses Verilog addition and concatenation:

```verilog
module full_adder_compact(A, B, Cin, Sum, Cout);
    input A, B, Cin;
    output Sum, Cout;

    assign {Cout, Sum} = A + B + Cin;
endmodule
```

The concatenation `{Cout, Sum}` forms a two-bit result. Since the maximum value of `A + B + Cin` is three, two bits are sufficient to represent both the carry and the sum. This form is concise and synthesizable, but the explicit Boolean version is often preferred when first studying the circuit because it shows the logic structure directly.

The Full Adder is the basic cell used to construct wider adders. In the following laboratory exercise, several Full Adder stages will be connected so that the carry-out of each stage feeds the carry-in of the next stage, forming a 4-bit ripple carry adder.

5.6 Laboratory Exercise – 4-Bit Ripple Carry Adder

5.6.1 Objective

The objective of this laboratory exercise is to design, simulate, and verify a 4-bit Ripple Carry Adder using Verilog HDL. The exercise combines the combinational design techniques introduced in this chapter and applies them to a practical arithmetic circuit.

By completing this exercise, the reader will demonstrate hierarchical design, module instantiation, carry propagation between adder stages, and functional verification through simulation. The design will be written as reusable Verilog modules and verified in the Cadence simulation environment using a separate testbench.

5.6.2 Theory

Binary addition is performed bit by bit, beginning with the least significant bit and progressing toward the most significant bit. At each bit position, the circuit must add the corresponding bits of the two operands and also include any carry generated by the previous lower-order position. A Half Adder is insufficient for this

task because it has no carry-in input.

A Full Adder solves this limitation by accepting three inputs: `A`, `B`, and `Cin`. It produces a `Sum` bit and a carry-out signal `Cout`. When several Full Adders are connected together, each stage handles one bit of a multi-bit addition. The carry-out of one stage becomes the carry-in of the next stage. This carry-chain structure is called a ripple carry adder because the carry information ripples from the least significant stage to the most significant stage.

The ripple carry adder is simple and well suited for laboratory study. It clearly shows module reuse, signal interconnection, and the relationship between a one-bit arithmetic cell and a wider arithmetic circuit.

5.6.3 Ripple Carry Adder Architecture

A 4-bit Ripple Carry Adder is constructed from four one-bit Full Adder modules connected in series. The first Full Adder adds `A[0]`, `B[0]`, and the external `Cin`. Its carry-out is connected to the carry-in of the second Full Adder. This pattern continues until the fourth stage produces the final carry-out `Cout`.

The input operands are `A[3:0]` and `B[3:0]`. The least significant bits are `A[0]` and `B[0]`, while the most significant bits are `A[3]` and `B[3]`. The output `Sum[3:0]` contains the four result bits, and `Cout` indicates whether the addition generated a carry beyond the most significant bit.

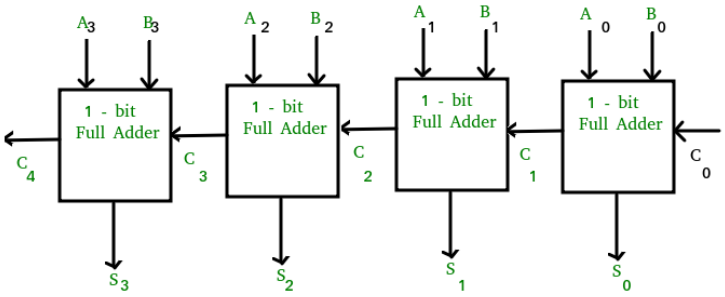


Figure 5.1: Conceptual block diagram of a 4-bit Ripple Carry Adder constructed from four Full Adder stages.

Data enters all four stages through the operand buses, but the carry path is sequential through the combinational chain. The first stage computes `Sum[0]` and an internal carry `c1`. The second stage uses `c1` to compute `Sum[1]` and `c2`. The third stage produces `Sum[2]` and `c3`. The fourth stage uses `c3` to compute `Sum[3]` and the final `Cout`. Although there is no clock in this circuit, the output cannot settle fully until the carry has propagated through the required stages.

5.6.4 Full Adder Module

The Full Adder module used in this exercise is the same one-bit combinational block discussed earlier in the chapter. It is written using continuous assignments so that the Boolean equations are represented directly in Verilog.

```verilog
module full_adder(A, B, Cin, Sum, Cout);
    input A, B, Cin;
    output Sum, Cout;

    assign Sum = A ^ B ^ Cin;
    assign Cout = (A & B) | (B & Cin) | (A & Cin);
endmodule
```

The inputs `A` and `B` are the operand bits for the current stage, and `Cin` is the carry entering that stage. The output `Sum` is generated by the XOR of the three inputs. The output `Cout` is generated when any two or more inputs are high, as expressed by the three AND terms combined by OR operations. This module is designed as a reusable cell and will be instantiated four times in the 4-bit adder.

5.6.5 4-Bit Ripple Carry Adder Implementation

The top-level Ripple Carry Adder module demonstrates hierarchical combinational design. Instead of rewriting the Full Adder equations four times, the design instantiates four copies of the `full_adder` module and connects them through intermediate carry wires.

```verilog
module ripple_carry_adder_4bit(A, B, Cin, Sum, Cout);
    input [3:0] A, B;
    input Cin;
    output [3:0] Sum;
    output Cout;

    wire c1, c2, c3;

    full_adder FA0 (
        .A(A[0]),
        .B(B[0]),
        .Cin(Cin),
        .Sum(Sum[0]),
        .Cout(c1)
    );

    full_adder FA1 (
        .A(A[1]),
        .B(B[1]),
        .Cin(c1),
        .Sum(Sum[1]),
        .Cout(c2)
    );

    full_adder FA2 (
        .A(A[2]),
        .B(B[2]),
        .Cin(c2),
        .Sum(Sum[2]),
        .Cout(c3)
    );

    full_adder FA3 (
        .A(A[3]),
        .B(B[3]),
        .Cin(c3),
        .Sum(Sum[3]),
        .Cout(Cout)
    );
endmodule
```

The module interface declares two four-bit input operands, one external carry-in, a four-bit sum output, and one final carry-out. The internal wires `c1`, `c2`, and `c3` carry information between adjacent Full Adder stages.

Instance `FA0` is the least significant stage. It adds `A[0]`, `B[0]`, and `Cin`, then produces `Sum[0]` and internal carry `c1`. Instance `FA1` uses `c1` as its carry-in and produces `c2`. Instance `FA2` uses `c2` and produces `c3`. Finally, instance `FA3` uses `c3` and produces the most significant sum bit and final

`Cout`. This carry chaining is the defining feature of the ripple carry architecture.

5.6.6 Testbench Development

A testbench is required because the Ripple Carry Adder module does not generate its own input stimulus. The testbench instantiates the design under test, applies representative operand values, waits for the combinational outputs to settle, and records the results for inspection in the simulator and waveform viewer.

The following testbench checks additions without carry generation, additions that produce internal carries, and additions that overflow beyond four bits.

```verilog
`timescale 1ns/1ps

module ripple_carry_adder_4bit_tb;
    reg [3:0] A, B;
    reg Cin;
    wire [3:0] Sum;
    wire Cout;

    ripple_carry_adder_4bit dut (
        .A(A),
        .B(B),
        .Cin(Cin),
        .Sum(Sum),
        .Cout(Cout)
    );

    initial begin
        $monitor($time, " A=%b B=%b Cin=%b Cout=%b Sum=%b",
                 A, B, Cin, Cout, Sum);

        A = 4'b0000; B = 4'b0000; Cin = 1'b0; #10;
        A = 4'b0011; B = 4'b0100; Cin = 1'b0; #10;
        A = 4'b0111; B = 4'b0001; Cin = 1'b0; #10;
        A = 4'b0101; B = 4'b0011; Cin = 1'b1; #10;
        A = 4'b1111; B = 4'b0001; Cin = 1'b0; #10;
        A = 4'b1111; B = 4'b1111; Cin = 1'b1; #10;

        $finish;
    end
endmodule
```

The first test applies zero operands and verifies the reset-like baseline behavior of the combinational circuit. The second test performs addition without overflow. The third and fourth tests require carry propagation through one or more stages. The fifth test produces a carry beyond the four-bit sum field, and the final test verifies the maximum input condition with an asserted carry-in. The `##10` delays separate each stimulus vector in simulation time so that the waveform viewer clearly shows the response to each input combination. The `$finish` system task terminates the simulation after all vectors have been applied.

5.6.7 Simulation Procedure

The simulation should be performed as a controlled laboratory workflow. Keep the Full Adder source, Ripple Carry Adder source, and testbench source in the same project directory or add all three files explicitly to the Cadence simulation setup.

1. Create a new Cadence project or working directory for the Ripple Carry Adder exercise.
2. Create the Verilog source file containing the `full_adder` module.
3. Create the Verilog source file containing the `ripple_carry_adder_4bit` module.

4. Create the testbench file containing `ripple_carry_adder_4bit_tb`.
5. Add all design and testbench files to the simulation environment.
6. Compile the source files and confirm that there are no syntax or elaboration errors.
7. Launch the simulation with `ripple_carry_adder_4bit_tb` as the top-level module.
8. Add `A`, `B`, `Cin`, `Sum`, and `Cout` to the waveform viewer.
9. Run the simulation long enough for all test vectors to execute.
10. Compare the waveform and console output against the expected binary addition results.

During compilation, errors are most commonly caused by mismatched module names, missing source files, or incorrect port names during instantiation. During simulation, incorrect results are usually caused by a broken carry connection between stages or a mismatch between the expected and actual bit ordering of the buses.

5.6.8 Expected Results

The simulation should show that `Sum[3:0]` contains the lower four bits of the binary addition and that `Cout` contains the carry beyond the most significant bit. For example, `0011 + 0100` should produce `Sum = 0111` and `Cout = 0`. The case `1111 + 0001` should produce `Sum = 0000` and `Cout = 1`, showing overflow beyond the four-bit output range.

Carry propagation should be visible in the functional behavior of the result. When a lower-order stage generates a carry, the next stage must include that carry in its own sum calculation. Although the internal carry wires may not be top-level ports, they can be added to the waveform viewer from the design hierarchy if the simulator setup allows internal signal probing.

TODO: Insert Ripple Carry Adder waveform

Figure 5.2: Expected waveform for the 4-bit Ripple Carry Adder showing operand inputs, carry-in, sum output, and carry-out.

The waveform verifies correct operation when each applied input vector produces the corresponding binary sum after the simulator advances past the stimulus delay. A mismatch in `Sum` indicates an error in the Full Adder equation or bit connection. A mismatch in `Cout` usually indicates an error in the final carry connection or in one of the intermediate carry wires.

5.6.9 Conclusion

This exercise demonstrated the design and verification of a 4-bit Ripple Carry Adder using Verilog HDL. The circuit was built hierarchically by defining a reusable Full Adder module and instantiating it four times inside a top-level combinational design. The intermediate carry wires illustrated how carry propagation links individual one-bit stages into a wider arithmetic circuit.

The testbench verified the design by applying representative input combinations and checking the resulting sum and carry-out behavior through simulation. This workflow reinforces the standard RTL design method used throughout digital systems engineering: describe a reusable module, connect modules hierarchically, compile the design, simulate with controlled stimulus, and verify the waveform against expected behavior. The same discipline will be used in later chapters when the manual moves from combinational circuits to sequential logic and more complex digital systems.

06

Sequential Logic Design

System identity, licensing, and workstation preparation

6.1 Why Memory is Required

The combinational circuits studied in the previous chapter produce outputs that depend only on the present values of their inputs. A multiplexer, Full Adder, or dataflow Boolean network has no internal record of what happened earlier. If the same inputs are applied again, the same outputs are produced, regardless of the previous input sequence. This property is useful for arithmetic and selection logic, but it is not sufficient for digital systems that must remember events, hold data, or move through a sequence of states.

Practical digital systems require *state*. A counter must remember its present count before it can generate the next count. A register must hold a binary value until the system is ready to load a new one. A finite state machine must remember its current state so that it can respond correctly to the next input condition. Digital controllers use this stored state to sequence operations, enable datapath blocks, and coordinate communication with other modules.

A circuit that includes memory is called a sequential circuit. Its output may depend on both the current inputs and the stored state. The stored state is normally updated at well-defined clock events so that the system changes in a controlled and predictable manner. This distinction is summarized in Table 6.1.

Feature	**Combinational Logic**	**Sequential Logic**
Output dependence	Present input values only	Present inputs and stored state
Memory	No internal memory	Contains storage elements
Typical examples	Adders, multiplexers, decoders	Registers, counters, finite state machines
Verilog style	Continuous assignments and combinational blocks	Clocked `always` blocks and non-blocking assignments

Table 6.1: Conceptual comparison between combinational and sequential logic.

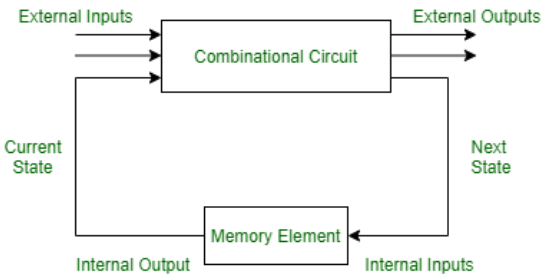


Figure 6.1: Conceptual of Sequential logic with stored state.

In RTL design, combinational logic and sequential logic are usually used together. Combinational logic computes next values, while storage elements capture those values at clock events. This chapter introduces the clocked storage elements required to build registers and counters, leading directly to the 4-bit synchronous counter exercise.

6.2 Clock Signals

A clock signal is a periodic digital signal used to coordinate when storage elements update their values. In a synchronous digital system, flip-flops and registers usually sample their inputs on a specific clock transition. This shared timing reference allows many storage elements to update together and makes the behavior of the system easier to simulate, verify, and synthesize.

The *clock period* is the time required for one complete clock cycle. The *clock frequency* is the number of cycles per second and is the reciprocal of the period. A clock transition from low to high is called a rising edge, while a transition from high to low is called a falling edge. In Verilog RTL, these transitions are written using `posedge clk` and `negedge clk`.

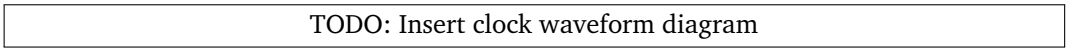


Figure 6.2: Clock waveform showing period, frequency, rising edge, and falling edge.

A clock can be generated in a testbench using an `always` block and a delay. The following example toggles `clk` every 5 ns, producing a clock period of 10 ns.

```verilog
`timescale 1ns/1ps

module clock_example_tb;
    reg clk;

    initial begin
        clk = 1'b0;
    end

    always #5 clk = ~clk;
endmodule
```

This form is intended for simulation testbenches, not for synthesizable design logic. It creates a simple periodic signal that can drive clocked modules during verification.

A storage element responds to a clock edge by listing that edge in the sensitivity list of an `always` block:

```verilog
always @(posedge clk) begin
    q <= d;
end
```

The statement above means that `q` is updated only on the rising edge of `clk`. If a falling-edge triggered design is required, `negedge clk` is used instead. Synchronous systems are preferred in most RTL designs because the state updates occur at known clock boundaries, which simplifies simulation, timing analysis, and debugging.

6.3 D Flip-Flops

A D Flip-Flop is the basic edge-triggered storage element used in synchronous RTL design. It has a data input, usually named `D`, a clock input, and an output, usually named `Q`. On the active clock edge, the value present at `D` is captured and stored at `Q`. Between active clock edges, `Q` holds its previous value even if `D` changes.

This behavior is what gives sequential circuits memory. The flip-flop does not continuously pass its input to the output. Instead, it samples the input at a clock transition and holds that sampled value until the next active edge. Table 6.2 summarizes the functional behavior for a rising-edge triggered D Flip-Flop.

Clock Event	D	Next Q
Rising edge	0	0
Rising edge	1	1
No rising edge	X	Previous Q

Table 6.2: Functional behavior of a rising-edge triggered D Flip-Flop.

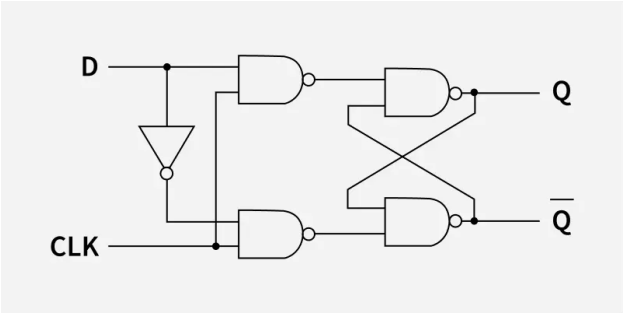


Figure 6.3: D Flip Flop Circuit Diagram

The timing behavior should be interpreted carefully in simulation. The input `D` may change many times between clock edges, but only the value present at the active edge is stored. After the edge, `Q` remains stable until the next active edge, except when a reset signal is included and asserted.

```verilog
module d_flip_flop(d, clk, q);
    input d, clk;
    output reg q;

    always @(posedge clk) begin
        q <= d;
    end
endmodule
```

The `always` block describes logic that is evaluated when the event in its sensitivity list occurs. In this example, the event is the rising edge of `clk`. The output `q` is declared as `reg` because it is assigned inside a procedural block. The assignment operator `<=` is a non-blocking assignment and is the standard form used for clocked sequential logic. It allows all registers triggered by the same clock edge to be updated together in simulation, matching the behavior expected from synchronous hardware.

Most practical flip-flops include a reset signal so that the circuit can be placed into a known state. A synchronous reset is checked inside the clocked block and affects the output only on the active clock edge.

```verilog
module dff_sync_reset(d, clk, reset, q);
    input d, clk, reset;
    output reg q;

    always @(posedge clk) begin
        if (reset)
            q <= 1'b0;
        else
            q <= d;
    end
endmodule
```

In this module, `reset` is sampled on the rising edge of `clk`. If `reset` is high at that edge, `q` is cleared to zero. Otherwise, `q` captures `d`.

An asynchronous reset is included in the sensitivity list and can clear the output without waiting for the next clock edge. This is common when a design must enter a known state immediately after reset is asserted.

```verilog
module dff_async_reset(d, clk, resetn, q);
    input d, clk, resetn;
    output reg q;

    always @(posedge clk or negedge resetn) begin
        if (!resetn)
            q <= 1'b0;
        else
            q <= d;
    end
endmodule
```

The signal `resetn` is active low, as indicated by the suffix `n` and by the condition `!resetn`. The block executes either on a rising clock edge or on a falling edge of reset. If reset is asserted, the output clears immediately in simulation. If reset is not asserted, the flip-flop behaves like the basic clocked D Flip-Flop.

6.4 Registers

A register is a collection of flip-flops used to store a multi-bit binary value. Each bit of the register is held by one flip-flop, and all bits are normally loaded on the same active clock edge. For example, a 4-bit register stores four bits, an 8-bit register stores eight bits, and an `n`-bit register stores `n` bits.

Registers are used throughout digital systems. They hold operands in datapaths, store intermediate results, capture status flags, preserve configuration values, and form the state registers of finite state machines. In a synchronous RTL design, registers define the boundaries between cycles: combinational logic computes a value during one cycle, and a register captures that value at the next active clock edge.

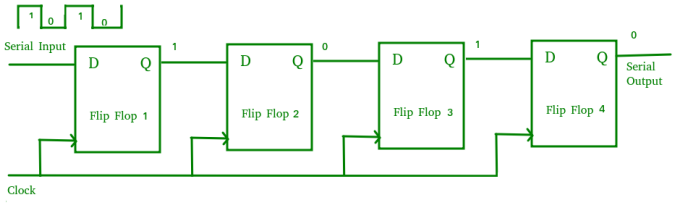


Figure 6.4: Register architecture showing multiple D Flip-Flops sharing a common clock and reset.

In Verilog, a register output assigned inside a clocked `always` block is declared using `reg`. The width of the stored value is written using a vector range.

```verilog
reg [3:0] data_reg;
reg [7:0] status_reg;
```

A simple 4-bit register with synchronous reset and clocked loading is shown below.

```verilog
module register_4bit(d, clk, reset, q);
    input [3:0] d;
    input clk, reset;
    output reg [3:0] q;

    always @(posedge clk) begin
        if (reset)
            q <= 4'b0000;
        else
            q <= d;
    end
endmodule
```

On each rising edge of `clk`, the register either clears to zero or captures the four-bit input `d`. The output `q` then holds that value until the next clock edge. The same pattern can be extended to wider registers by changing the vector width.

```verilog
module register_8bit(d, clk, resetn, q);
    input [7:0] d;
    input clk, resetn;
    output reg [7:0] q;

    always @(posedge clk or negedge resetn) begin
        if (!resetn)
            q <= 8'b00000000;
        else
            q <= d;
    end
endmodule
```

This 8-bit register uses an active-low asynchronous reset. The design style is the same as the D Flip-Flop example, but the stored signal is now a vector. In datapath design, such registers are commonly placed before and after combinational blocks so that values move through the system one clock cycle at a time.

6.5 Counters

A counter is a sequential circuit that moves through a defined sequence of binary states. The most common counter increments by one on each active clock edge, but counters may also decrement, load a specified value, enable or disable counting, or count through a custom sequence. Since a counter must remember its current count before computing the next count, it is built from registers and combinational next-state logic.

Counters are used for timing generation, event counting, address sequencing, loop control, clock division, and control-state tracking. A binary up-counter with `n` bits can represent `2^n` states. When the counter reaches its maximum value and increments again, it wraps around to zero unless additional logic is provided.

Asynchronous Counters
In an asynchronous counter, also called a ripple counter, only the first flip-flop is driven by the external clock. The output of one flip-flop clocks the next stage. This creates a ripple effect: later bits change only after earlier bits have toggled. The circuit is simple, but the accumulated propagation delay makes timing less predictable, especially as the counter width increases. For this reason, asynchronous counters are generally avoided in synthesizable RTL intended for modern synchronous digital systems.

Synchronous Counters
In a synchronous counter, all flip-flops share the same clock. The next count value is computed by combinational logic and captured by all counter bits on the same active clock edge. This makes the timing behavior easier to analyze and verify. Synchronous counters are therefore the preferred style for RTL design and are the style used in the following laboratory exercise.

Table 6.3 shows the state sequence for a simple 3-bit binary up-counter.

Present Count	Next Count
000	001
001	010
010	011
011	100
100	101
101	110
110	111
111	000

Table 6.3: State transition sequence of a 3-bit binary up-counter.

A basic synchronous counter can be written by registering the count value and adding one on each clock edge. The increment operation is a combinational addition whose result is captured by the register at the active edge.

```verilog
module counter_3bit(clk, reset, count);
    input clk, reset;
    output reg [2:0] count;

    always @(posedge clk) begin
        if (reset)
            count <= 3'b000;
        else
            count <= count + 1'b1;
    end
endmodule
```

When `reset` is high at the rising edge of `clk`, the counter returns to zero. Otherwise, the current value of `count` is incremented by one. Since `count` is three bits wide, the value after `111` wraps around to `000`. This wrap-around behavior is a natural result of fixed-width binary arithmetic in Verilog.

The next laboratory exercise applies these ideas to a complete 4-bit synchronous counter. The design will use a clocked register, reset behavior, and binary increment logic, then verify the counting sequence using simulation waveforms in the Cadence environment.

6.6 Laboratory Exercise – 4-Bit Synchronous Counter

6.6.1 Objective

The objective of this experiment is to design, simulate, and verify a 4-bit synchronous counter using Verilog HDL. The exercise applies the sequential logic concepts introduced in this chapter and demonstrates how a clocked RTL block stores and updates state over time.

By completing this exercise, the reader will implement a state-based digital system, observe clocked operation, use sequential logic design practices, and verify the counter behavior through simulation. The completed design will be suitable for compilation and waveform inspection in the Cadence simulation environment.

6.6.2 Theory

A binary counter advances through a sequence of binary states. For a 4-bit up-counter, the output can represent sixteen values, from `0000` to `1111`. On each active clock edge, the counter increments by one and stores the new value in its internal register. Since the counter must remember its present value before producing the next value, it is a sequential circuit rather than a purely combinational circuit.

In a synchronous counter, all flip-flops share the same clock signal. This means that all bits of the counter update together on the active clock edge. The count sequence progresses as:

```terminal
0000
0001
0010
0011
...
1111
0000
```

After the maximum 4-bit value `1111`, the next increment produces `0000`. This is called overflow or wrap-around behavior. It occurs because the counter register has a fixed width of four bits; any carry beyond the most significant bit is discarded unless it is explicitly captured by additional logic.

6.6.3 Counter Architecture

The 4-bit synchronous counter contains a clock input, a reset input, and a 4-bit output bus. The output bus represents the present count value. Internally, the counter behaves as a 4-bit register whose next value is either zero during reset or the present count plus one during normal operation.

All storage bits receive the same clock. On each rising edge of the clock, the counter checks the reset signal. If reset is asserted, the count is cleared to `0000`. If reset is not asserted, the counter advances to the next binary value.

TODO: Insert 4-Bit Synchronous Counter block diagram

Figure 6.5: Block diagram of a 4-bit synchronous counter with clock, reset, and count output.

Table 6.4 shows several successive states of the 4-bit counter.

Present Count	Next Count
0000	0001
0001	0010
0010	0011
0011	0100
0100	0101
1110	1111
1111	0000

Table 6.4: Selected state transitions of a 4-bit synchronous up-counter.

6.6.4 Verilog Implementation

The RTL implementation uses one clocked `always` block. The counter output is declared as a 4-bit `reg` because it is assigned inside the procedural block. Non-blocking assignments are used so that the update behavior matches edge-triggered hardware.

```verilog
module synchronous_counter_4bit(clk, reset, count);
    input clk, reset;
    output reg [3:0] count;

    always @(posedge clk) begin
        if (reset)
            count <= 4'b0000;
        else
            count <= count + 1'b1;
    end
endmodule
```

The module interface contains the clock input `clk`, the reset input `reset`, and the 4-bit output `count`. The `always` block executes on each rising edge of `clk`. If `reset` is high at that edge, the counter is loaded with `0000`. Otherwise, the present value of `count` is incremented by one. Since `count` is four bits wide, the increment after `1111` wraps around to `0000`.

6.6.5 Testbench Development

Verification is required to confirm that the counter initializes correctly, increments on clock edges, and responds properly to reset. The testbench generates a clock, applies reset, lets the counter run for several cycles, applies reset again, and then terminates the simulation.

```verilog
`timescale 1ns/1ps

module synchronous_counter_4bit_tb;
    reg clk, reset;
    wire [3:0] count;

    synchronous_counter_4bit dut (
        .clk(clk),
        .reset(reset),
        .count(count)
    );

    initial begin
```

```
        clk = 1'b0;
    end

    always #5 clk = ~clk;

    initial begin
        $monitor($time, " reset=%b count=%b", reset, count);

        reset = 1'b1;
        #12;

        reset = 1'b0;
        #180;

        reset = 1'b1;
        #10;

        reset = 1'b0;
        #40;

        $finish;
    end
endmodule
```

The `always ##5` statement generates a 10 ns clock period for simulation. The first reset interval initializes the counter to zero. After reset is released, the counter advances on each rising clock edge. The second reset interval verifies that the counter can be returned to `0000` after it has already been counting. The `$monitor` statement prints the reset and count values during simulation, while the waveform viewer provides the clearest visual confirmation of the state transitions.

6.6.6 Simulation Procedure

Use the following workflow to perform the experiment in the Cadence simulation environment:

1. Create a new working directory or Cadence project for the synchronous counter experiment.
2. Create the Verilog source file containing `synchronous_counter_4bit`.
3. Create the testbench file containing `synchronous_counter_4bit_tb`.
4. Add both Verilog files to the simulation setup.
5. Compile the design and confirm that no syntax or elaboration errors are reported.
6. Run the simulation with `synchronous_counter_4bit_tb` as the top-level module.
7. Open the waveform viewer and add `clk`, `reset`, and `count`.
8. Verify that the counter resets to zero and increments on rising clock edges.

If the design does not compile, check the module names, port names, and file order. If the simulation runs but the waveform is incorrect, inspect the reset polarity, the clock generation statement, and the use of non-blocking assignments in the counter.

6.6.7 Expected Results

During simulation, the counter output should become `0000` whenever reset is asserted and a rising clock edge occurs. After reset is released, the output should increment by one on each rising edge of `clk`. The sequence should progress through the normal binary count values from `0000` to `1111`. After `1111`, the next increment should wrap around to `0000`.

The waveform should show that changes in `count` occur only at rising clock edges. This confirms that the design is sequential and clock controlled, rather than continuously driven like combinational logic.

TODO: Insert synchronous counter waveform

Figure 6.6: Expected waveform of the 4-bit synchronous counter showing reset behavior, binary incrementing, and wrap-around.

Correct operation is verified when the waveform shows initialization to `0000`, steady binary progression at clock edges, reset returning the count to zero, and overflow from `1111` back to `0000`.

6.6.8 Conclusion

This experiment demonstrated the design and verification of a 4-bit synchronous counter using Verilog HDL. The counter was implemented as a clocked sequential circuit using an `always @(posedge clk)` block, non-blocking assignments, reset logic, and fixed-width binary incrementing.

The simulation verified clocked state transitions, reset behavior, normal counter operation, and wrap-around after the maximum 4-bit value. These concepts form the basis for more advanced sequential systems, including finite state machines, digital controllers, and processing datapaths that will be explored in later chapters.

Part III

VERIFICATION

Testbenches, simulation, waveform analysis, and coverage

07

Verilog Testbenches

System identity, licensing, and workstation preparation

7.1 What is Verification?

Every design discussed so far in this manual — the Half Adder, the Ripple Carry Adder, and the 4-bit Synchronous Counter — was written, compiled, and exercised through a testbench before its behavior was accepted as correct. This chapter formalizes that practice. Verification is the process of confirming that a design behaves according to its intended specification before it is committed to the next stage of the design flow.

In the RTL-to-GDSII flow described throughout this manual, simulation is performed before synthesis for a deliberate reason. Once an RTL description has been synthesized, the design exists as a gate-level netlist tied to a specific standard cell library. Any functional error discovered at that stage requires the designer to return to the RTL, correct the logic, and repeat synthesis, timing closure, and any downstream physical design work that depended on the earlier netlist. The further a defect travels through the flow before it is caught, the more expensive it becomes to fix — not only in engineering time, but in the number of dependent steps that must be redone. Simulation at the RTL stage is the least expensive point at which a functional error can be identified, which is why a design is never considered complete simply because it compiles or synthesizes without errors.

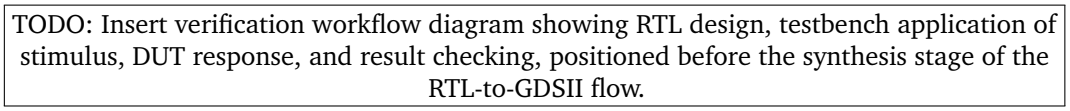


Figure 7.1: Conceptual position of verification within the RTL-to-GDSII flow, showing the DUT receiving stimulus from a testbench and producing outputs that are checked before synthesis begins.

The design being verified is referred to as the Design Under Test, or DUT. The DUT is the module whose correctness is in question — it may be a single combinational block such as a multiplexer, or a hierarchical design such as the Ripple Carry Adder, where individual Full Adder instances are themselves part of a larger structure being verified as a whole. The DUT does not verify itself; it is a passive block of RTL that simply responds to whatever signals are applied to its inputs.

Verification is carried out by applying a controlled sequence of input values to the DUT and observing the resulting outputs. This is the same principle used in the Half Adder exercise in Chapter 4 and the Ripple Carry Adder exercise in Chapter 5: known input combinations are applied, and the resulting Sum and Carry values are compared against the expected truth table. What differs in this chapter is not the principle, but the formality and structure with which stimulus is applied and results are checked.

It is useful to think of verification as a process of building confidence rather than as a single pass/fail event. A single simulation run that produces the expected output for one input combination tells the designer very little on its own. Confidence increases as the design is exercised across a representative range of operating

conditions — typical values, boundary values, reset conditions, and sequences that are likely to expose timing-related or state-dependent errors. A design that has been verified across such a range can be carried into synthesis and physical implementation with reasonable assurance that its logical behavior is correct, leaving the remaining stages of the flow to focus on timing, area, and physical constraints rather than functional correctness.

7.2 Testbench Architecture

A testbench is a Verilog module written specifically to exercise another module – the DUT – and to observe its behavior during simulation. Every testbench used in this manual so far has followed the same underlying structure, even though earlier chapters did not name the individual pieces explicitly. This section identifies those pieces so that they can be reused deliberately when writing new testbenches.

A complete testbench is generally organized around the following elements:

1. **Testbench Module** – a top-level module with no ports. Unlike the DUT, a testbench does not need to expose an external interface, because it is never instantiated inside another design. It exists only for simulation.
2. **DUT Instantiation** – an instance of the design being verified, connected to signals declared inside the testbench. This instance is often referred to by a short identifier such as `dut`, matching the convention used in earlier chapters.
3. **Input Stimulus** – signals declared as `reg` inside the testbench and connected to the input ports of the DUT. These signals are driven by the testbench to apply test conditions.
4. **Output Monitoring** – signals declared as `wire` inside the testbench and connected to the output ports of the DUT. These signals are observed, either visually in the waveform viewer or programmatically by the testbench itself.
5. **Clock Generation** – for sequential designs, a periodic signal that drives the DUT's clock input. Combinational designs such as the Half Adder and Ripple Carry Adder do not require this element, but sequential designs such as the Synchronous Counter do.
6. **Result Checking** – logic within the testbench that compares the observed outputs against expected values and reports whether the DUT behaved correctly.

The general skeleton of a testbench module follows the same pattern used for ordinary RTL modules, with one important difference in how it is treated by the tools:

```verilog
`timescale 1ns/1ps

module dut_tb;

    // input stimulus signals (reg)
    // output monitoring signals (wire)

    dut_module dut (
        // port connections
    );

    // clock generation, if required

    initial begin
        // stimulus sequence
    end

endmodule
```

A testbench is a simulation construct only. It is never passed through synthesis, and it is never represented in the final hardware. Constructs that are perfectly normal in a testbench — unbounded delays, system tasks such as `$display` and `$finish`, and procedural blocks with no corresponding hardware structure — would either be rejected by a synthesis tool or would simply be ignored. For this reason, the separation between the DUT and the testbench is not just an organizational convenience; it reflects a real distinction in how the two pieces of code are processed by the Cadence toolchain. The DUT must remain synthesizable, while the testbench exists purely to drive and observe it during simulation.

This separation also explains why the DUT is instantiated inside the testbench, and not the other way around. The testbench forms the outermost layer of the simulation hierarchy: it generates the clock, drives the inputs, and observes the outputs of the design beneath it, but it has no ports of its own and is never itself instantiated.

TODO: Insert testbench architecture diagram showing the testbench module as an outer block containing the DUT instance, with stimulus signals flowing into the DUT, output signals flowing out to monitoring logic, and a clock generator feeding the DUT's clock input.

Figure 7.2: Structural view of a Verilog testbench, showing the DUT instance surrounded by stimulus generation, clock generation, and result-checking logic, all contained within the testbench module.

With the overall structure established, the remaining sections of this chapter examine each of the procedural constructs used to build these elements – `initial` blocks for one-time stimulus sequences, `always` blocks for repeating behavior such as clocks, and the system tasks used for monitoring and checking results.

7.3 Initial Blocks

An `initial` block describes a sequence of statements that begins executing at the start of simulation and runs exactly once. This makes it the natural construct for two purposes that appear in almost every testbench: establishing the initial state of signals before the DUT begins responding, and applying a sequence of stimulus values over the course of the simulation.

The general form of an `initial` block is:

```verilog
initial begin
    // statements executed once, in order
end
```

The statements inside an `initial` block execute in the order they are written, exactly like a sequential procedure. This is different from continuous assignments, where, as discussed in Chapter 5, the order of the statements in the source does not imply an execution order. Inside an `initial` block, order matters directly: each statement is completed before the next one begins, subject to any simulation delays that are written into the sequence.

The simplest use of an `initial` block is to assign starting values to signals before the simulation proceeds further. This was used implicitly in the Synchronous Counter testbench in Chapter 6, where the clock signal was given a defined starting value:

```verilog
`timescale 1ns/1ps

module init_example_tb;

    reg clk;
```

```
    initial begin
        clk = 1'b0;
    end

endmodule
end
```

Without this assignment, `clk` would begin simulation in the unknown state `x`, and any logic depending on its value would also begin in an unknown state. Establishing a known starting condition for every signal driven by the testbench is good practice, since an unresolved `x` value can propagate through combinational logic and obscure the actual behavior under test.

The second, and more common, use of an `initial` block is to apply a sequence of input values over simulation time. This is done using the `##` delay control, which pauses execution of the block for a specified number of simulation time units before the next statement runs. The unit of time is determined by the `‘timescale` directive at the top of the file, which was introduced in Chapter 4.

The following example applies four different input combinations to a pair of signals, separated by a fixed delay:

verilog

```
`timescale 1ns/1ps

module stimulus_example_tb;

    reg A, B;

    initial begin
        A = 0; B = 0; #10;
        A = 0; B = 1; #10;
        A = 1; B = 0; #10;
        A = 1; B = 1; #10;
        $finish;
    end

endmodule
```

This sequence is functionally identical to the stimulus used in the Half Adder testbench in Chapter 4. Execution begins with `A = 0; B = 0;`, both of which take effect immediately, at simulation time 0. The `##10` that follows then pauses the block for 10 time units before the next pair of assignments executes. This pattern — assign values, then delay — repeats for each row of the intended truth table, after which `$finish` ends the simulation.

It is important to recognize that the delays inside an `initial` block apply to the testbench’s own sequence of statements, not to the DUT. The DUT has no awareness of the testbench’s timing structure; it simply responds to whatever values appear on its input ports at whatever simulation time they change. The delays exist so that the waveform viewer has a clear, evenly spaced record of each applied condition, and so that combinational logic has time to settle before the next input change is applied.

A single testbench module may contain more than one `initial` block. When this is done, all `initial` blocks begin execution at simulation time 0 and proceed independently and concurrently. This is useful when one block is responsible for generating stimulus while another is responsible for reporting results, since the two concerns can be kept separate in the source code while still executing over the same simulation timeline. This pattern is used again later in this chapter when self-checking behavior is introduced.

7.4 Always Blocks

An `always` block describes behavior that repeats for the duration of the simulation, in response to events specified in its sensitivity list. Where an `initial` block executes its statements exactly once, an `always` block re-executes its statements every time the conditions in its sensitivity list are met. This makes `always` blocks suitable for two distinct purposes that appear throughout this manual: describing clocked sequential logic in RTL design, and generating repeating signals — most commonly a clock — inside a testbench.

The general form of an `always` block is:

```verilog
always @(sensitivity_list)
    // statement or block of statements
```

The sensitivity list specifies the events that cause the block to execute. In RTL design, the sensitivity list is normally an edge of a clock signal, written using `posedge` or `negedge`, as introduced in Chapter 6. Each time the specified edge occurs, the statements inside the block execute once and then wait for the next occurrence of that edge. The D Flip-Flop and counter modules from Chapter 6 are built entirely on this principle:

```verilog
always @(posedge clk) begin
    if (reset)
        count <= 4'b0000;
    else
        count <= count + 1'b1;
end
```

This block does not run once and stop; it runs every time `clk` transitions from low to high, for as long as the simulation continues. Each execution either resets `count` or increments it, depending on the value of `reset` at that edge. This is the event-driven behavior that gives sequential circuits their defining property: the output changes only in response to a specific event, not continuously and not on a fixed schedule of its own choosing.

Inside a testbench, `always` blocks are used differently. Rather than describing how a register should respond to a clock, the testbench uses an `always` block to generate the clock itself. A clock signal is, by definition, a value that toggles repeatedly for the entire simulation — which is exactly the kind of repeating behavior an `always` block is suited to describe. This usage was introduced briefly in Chapter 6 and is examined in full in the next section.

A second testbench use of `always` blocks is continuous monitoring of a signal as it changes, without waiting for a clock edge. An `always` block sensitive to a change in any signal — written using `always @(*)` or by listing the relevant signals explicitly — executes its body every time one of those signals changes value:

```verilog
always @(A or B or Sum or Carry)
    $display($time, " A=%b B=%b Sum=%b Carry=%b", A, B, Sum, Carry);
```

This block produces a printed line every time any of `A, B, Sum,` or `Carry` changes, which can be useful for tracing the exact order in which signals settle during a single delay step. In practice, the `$monitor` system task used in earlier chapters provides a more convenient way to achieve a similar result, since it

automatically reports whenever any of its arguments change without requiring an explicit sensitivity list. `$monitor` is used extensively later in this chapter and is revisited in the self-checking testbenches discussed in Section 7.7.

The distinction to carry forward from this section is straightforward: an `initial` block describes something that happens once, at the start of simulation, while an `always` block describes something that happens repeatedly, in response to events that recur for as long as the simulation runs. Both constructs are procedural — both use the same `begin` / `end` block structure and the same assignment operators — but they serve fundamentally different roles in a testbench: `initial` blocks apply stimulus, while `always` blocks generate ongoing signals such as the clock, which is the subject of the next section.

7.5 Clock Generation

Every sequential design discussed so far — the D Flip-Flop, the registers, and the Synchronous Counter — depends on a clock signal to determine when its stored state is updated. In simulation, this clock does not come from an external oscillator or a physical clock distribution network; it must be generated by the testbench itself, using the `always` block construct introduced in the previous section.

A clock signal in simulation is characterized by the same properties as a clock signal in hardware, introduced in Chapter 6: a clock period, a clock frequency, and a duty cycle. The clock period is the length of time required for one complete cycle — one rising edge to the next rising edge. The clock frequency is the reciprocal of the period. The duty cycle is the proportion of the period during which the signal is high, expressed as a percentage. A duty cycle of 50% means the signal spends equal time high and low during each period, which is the most common configuration for digital testbenches and the one used throughout this manual.

The standard method for generating a 50% duty cycle clock in a testbench is a single `always` block that toggles the clock signal after a fixed delay:

```verilog
`timescale 1ns/1ps

module clock_gen_tb;

    reg clk;

    initial begin
        clk = 1'b0;
    end

    always #5 clk = ~clk;

endmodule
```

This is the same construct used in the Synchronous Counter testbench in Chapter 6. Its operation can be read in two parts. The `initial` block establishes a known starting value for `clk` at simulation time 0, as described in Section 7.3. The `always` block then takes over: every 5 time units, it inverts the current value of `clk`. Starting from `0`, the signal becomes `1` at time 5, returns to 0 at time 10, becomes `1` again at time 15, and so on for the remainder of the simulation.

Because the signal toggles every 5 time units, one complete period — low, then high, then back to low — takes 10 time units. With the `` `timescale 1ns/1ps `` directive in effect, this corresponds to a clock period of 10 ns, or equivalently a clock frequency of 100 MHz. The duty cycle is 50%, since the signal spends exactly 5 ns high and 5 ns low within each 10 ns period. Changing the delay value changes the period proportionally:

`always ##10 clk = ~clk;` would produce a 20 ns period, or 50 MHz, while still maintaining a 50% duty cycle, since both the high and low phases are controlled by the same delay value.

A duty cycle other than 50% can be produced by using different delay values for the high and low phases, though this is rarely required for the digital designs covered in this manual:

```verilog
always begin
    clk = 1'b1; #3;
    clk = 1'b0; #7;
end
```

This produces a 10 ns period in which the clock is high for 3 ns and low for 7 ns — a 30% duty cycle. Designs in this manual use the standard 50% duty cycle clock generator shown earlier unless a specific exercise states otherwise.

One detail is worth emphasizing for laboratory work: the `always ##5 clk = ~clk;` statement, once started, never stops on its own. It continues toggling clk for as long as the simulation runs. Simulation ends only when a separate part of the testbench — typically the stimulus-generating `initial` block — calls `$finish`. If a testbench contains a clock generator but no statement that calls `$finish`, the simulation will continue indefinitely, or until it is stopped manually or by a tool-imposed time limit. For this reason, every testbench that includes a clock generator must also include a stimulus sequence that determines how long the simulation should run and brings it to an end.

TODO: Insert clock waveform diagram showing the generated clk signal over several periods, with the period, the high and low phases, and the rising and falling edges labeled.

Figure 7.3: Waveform produced by the standard clock generator, showing a 10 ns period with a 50% duty cycle.

With a clock available, the testbench can now drive sequential designs through a defined number of clock cycles. The next section addresses how stimulus — for both combinational and sequential designs — is organized into a coherent sequence of test conditions.

7.6 Stimulus Generation

The stimulus applied by a testbench is organized as a sequence of test vectors — specific combinations of input values applied at specific points in simulation time. The purpose of a test vector is to place the DUT into a known operating condition so that its response can be observed and compared against the expected behavior. A single test vector establishes one data point; a well-constructed sequence of test vectors establishes confidence across the range of conditions the design is expected to handle.

For combinational designs, stimulus generation is comparatively simple, because the DUT's output depends only on the present inputs. Each test vector can be applied and allowed to settle independently of any other vector. The Ripple Carry Adder testbench from Chapter 5 illustrates this pattern:

```verilog
initial begin
    $monitor($time, " A=%b B=%b Cin=%b Cout=%b Sum=%b",
             A, B, Cin, Cout, Sum);

    A = 4'b0000; B = 4'b0000; Cin = 1'b0; #10;
    A = 4'b0011; B = 4'b0100; Cin = 1'b0; #10;
    A = 4'b0111; B = 4'b0001; Cin = 1'b0; #10;
```

```
    A = 4'b0101; B = 4'b0011; Cin = 1'b1; #10;
    A = 4'b1111; B = 4'b0001; Cin = 1'b0; #10;
    A = 4'b1111; B = 4'b1111; Cin = 1'b1; #10;
    $finish;
end
```

Each line applies a complete set of operand and carry-in values and then waits 10 ns before moving to the next. The delay is not required by the combinational logic itself — the Ripple Carry Adder's outputs settle almost immediately after its inputs change — but it provides the waveform viewer with a clear, evenly spaced record of each applied condition, and it gives each test vector its own identifiable region of the simulation timeline. As discussed in Chapter 5, the chosen vectors deliberately cover several categories of behavior: an all-zero baseline, addition without carry propagation, addition with carry propagation through one or more stages, and addition that overflows the 4-bit result.

For sequential designs, stimulus generation must additionally account for the clock. Because a sequential design only updates its state on a clock edge, a test vector applied to a sequential DUT is not fully "absorbed" until the next active edge occurs. Stimulus for sequential designs is therefore expressed not only in terms of what values are applied, but how many clock cycles those values are held for. The Synchronous Counter testbench from Chapter 6 demonstrates this:

verilog

```
initial begin
    $monitor($time, " reset=%b count=%b", reset, count);

    reset = 1'b1;
    #12;
    reset = 1'b0;
    #180;
    reset = 1'b1;
    #10;
    reset = 1'b0;
    #40;
    $finish;
end
```

Here, the delays are chosen relative to the clock period rather than as arbitrary spacing. With a 10 ns clock period, holding `reset` high for 12 ns guarantees that at least one rising edge occurs while `reset` is asserted, so that the counter is observed entering its `reset` state. Releasing `reset` for 180 ns allows eighteen clock cycles to elapse — more than enough for a 4-bit counter to count from `0000` through `1111` and wrap around at least once, which is the overflow behavior described in Chapter 6. The second `reset` pulse and the final delay confirm that the counter can be returned to a known state after it has already been running, rather than only immediately after power-up.

This illustrates the general principle behind stimulus generation: every test vector should be chosen for a reason connected to the behavior it is intended to exercise, and the duration for which it is held should be sufficient for that behavior to become observable — whether that means allowing combinational logic to settle, or allowing enough clock cycles for sequential state to evolve through the conditions of interest.

A stimulus sequence that only exercises a small, convenient subset of conditions — for example, only ever testing a counter near `0000` and never near its maximum value — risks leaving design errors undetected simply because the conditions that would expose them were never applied. Comprehensive stimulus does not necessarily mean exhaustive stimulus; for a 4-bit counter, applying all sixteen possible states is practical, but for wider buses or more complex designs, exhaustive testing quickly becomes impractical. In those cases, stimulus is instead chosen to represent the categories of behavior the design must handle correctly — normal operation, boundary conditions, reset behavior, and overflow or wrap-around — which is the same approach already used in the Ripple Carry Adder and Synchronous Counter examples.

7.7 Self-Checking Testbenches

Every testbench presented so far in this manual relies on the `$monitor` system task to print signal values during simulation, leaving the task of judging correctness to the person running the simulation. The designer compares the printed values, or the waveform viewer's display, against the expected truth table or state sequence, and decides by inspection whether the DUT behaved correctly.

This approach is adequate for the small designs used in the laboratory exercises so far, where the expected outputs for each test vector can be checked by eye in a few seconds. It does not scale well. As designs grow larger, as stimulus sequences grow longer, and as the same testbench is reused repeatedly — after every RTL change, after every workstation clone, or as part of a larger regression — manually inspecting printed values for every run becomes slow, tedious, and error-prone. A self-checking testbench addresses this by having the testbench itself compare the DUT's actual output against the expected output, and report a pass or fail result without requiring the designer to inspect every line of output.

The basic mechanism for a self-checking testbench is the `if` statement, used to compare an observed output against an expected value immediately after a test vector has had time to take effect. The following example extends the Half Adder testbench from Chapter 4 with a check after each applied input combination:

verilog

```
`timescale 1ns/1ps

module half_adder_selfcheck_tb;

    reg A, B;
    wire Sum, Carry;
    reg exp_sum, exp_carry;

    half_adder dut (
        .A(A),
        .B(B),
        .Sum(Sum),
        .Carry(Carry)
    );

    initial begin
        A = 0; B = 0; exp_sum = 0; exp_carry = 0; #10;
        A = 0; B = 1; exp_sum = 1; exp_carry = 0; #10;
        A = 1; B = 0; exp_sum = 1; exp_carry = 0; #10;
        A = 1; B = 1; exp_sum = 0; exp_carry = 1; #10;
        $finish;
    end

    always @(A or B) begin
        #1;
        if (Sum !== exp_sum || Carry !== exp_carry)
            $error("Mismatch: A=%b B=%b Sum=%b (exp %b) Carry=%b (exp %b)",
                   A, B, Sum, exp_sum, Carry, exp_carry);
        else
            $display($time, " PASS: A=%b B=%b Sum=%b Carry=%b", A, B, Sum, Carry);
    end

endmodule
```

The structure of this testbench follows directly from the constructs introduced earlier in this chapter. The `initial` block applies the same four input combinations used in Chapter 4, but each assignment is now accompanied by the corresponding expected values, `exp_sum` and `exp_carry`, taken directly from the Half Adder truth table. A second block, sensitive to changes in `A` or `B`, waits 1 ns for the combinational outputs to settle and then compares `Sum` and `Carry` against the expected values using `if`. If the

comparison fails, `$error` reports a mismatch, including the actual and expected values for both outputs. If the comparison succeeds, `$display` reports a pass for that input combination.

The comparison uses the `!==` operator rather than `!=`. As discussed in Chapter 4, Verilog signals can take the values `0, 1, x,` and `z`. The `!==` operator performs an exact four-state comparison, treating `x` and `z` as distinct values rather than attempting to resolve them. This is important in a checking context: if a design defect causes an output to remain at `x` when it should be a defined `0` or `1`, a four-state comparison will correctly flag this as a mismatch, whereas a two-state comparison might not.

The same principle applies to sequential designs, with the check performed relative to the clock rather than relative to a change in the inputs. For the Synchronous Counter, the expected count value can be tracked by the testbench itself and compared against the DUT's output on each clock edge:

```verilog
reg [3:0] expected_count;

always @(posedge clk) begin
    if (reset)
        expected_count <= 4'b0000;
    else
        expected_count <= expected_count + 1'b1;
end

always @(posedge clk) begin
    #1;
    if (count !== expected_count)
        $error("Mismatch at time %0t: count=%b expected=%b",
               $time, count, expected_count);
end
```

Here, the testbench maintains its own copy of the expected count using exactly the same reset and increment logic as the DUT, described in Chapter 6. A second `always` block, also triggered on `posedge clk`, waits briefly for the DUT's output to update and then compares it against the testbench's own expected value. Because the expected-value logic is written independently of the DUT, this check is meaningful: it is not simply comparing the DUT against itself, but against a separately written model of the correct behavior.

The distinction between this approach and the `$monitor`-based testbenches used earlier in this manual is one of effort and scalability rather than of fundamental technique. A self-checking testbench requires more work to write, because the expected behavior of the design must be expressed explicitly in the testbench rather than left for the designer to judge by eye. In exchange, the testbench becomes repeatable: it can be run after every change to the RTL, every workstation clone, or every tool update, and it will report `PASS` or `$error` without requiring anyone to re-examine a waveform. As designs grow beyond the small combinational and sequential blocks used in this manual, this repeatability becomes the primary reason self-checking testbenches are used in preference to manual inspection — the verification effort invested once in writing the checking logic is reused automatically on every subsequent simulation run.

The laboratory exercise that follows applies these constructs together — clock generation, sequential stimulus, and self-checking comparison — to the 4-bit Synchronous Counter introduced in Chapter 6, producing a complete and automatically verified testbench for that design.

7.8 Laboratory Exercise – Counter Testbench

7.8.1 Objective

The objective of this laboratory exercise is to develop, execute, and verify a complete Verilog testbench for the 4-bit Synchronous Counter designed in Chapter 6. The exercise builds directly on the constructs introduced earlier in this chapter — initial blocks, always blocks, clock generation, and self-checking comparison — and

applies them in a single, integrated verification environment.

By completing this exercise, the reader will demonstrate DUT instantiation, clock generation, reset sequencing, stimulus generation, functional verification, waveform observation, and simulation-based debugging. The resulting testbench will confirm that the counter initializes correctly, responds to reset, increments through its full range of states, and exhibits the wrap-around behavior described in Chapter 6, all observed through the Cadence simulation environment.

7.8.2 Theory

A testbench is a Verilog module written to exercise another module — the Design Under Test, or DUT — and to observe its response during simulation. As discussed earlier in this chapter, the testbench applies a sequence of input stimulus to the DUT, allows the design to respond according to its RTL description, and either presents the resulting output for inspection or compares it directly against an expected value.

For a sequential design such as the Synchronous Counter, verification requires three coordinated elements: a clock signal that drives the counter's internal state updates, a reset signal that establishes a known starting condition, and a stimulus sequence long enough to observe the counter progressing through its complete state space. The counter is verified by holding reset asserted for at least one clock edge, releasing it, and then allowing the clock to run for enough cycles that the count value can be seen advancing from `0000` through `1111` and wrapping back to `0000`.

This exercise is performed in simulation, before any synthesis activity, for the reason established in Section 7.1: a functional error in the counter's reset or increment logic is far less costly to identify and correct at the RTL stage than after the design has been synthesized into a gate-level netlist.

7.8.3 Design Under Test

The Design Under Test for this exercise is the `synchronous_counter_4bit` module developed in Chapter 6. Its interface consists of three ports: a clock input `clk`, a reset input `reset`, and a 4-bit registered output `count`.

The expected operational behavior of the counter is as follows. On each rising edge of `clk`, the counter checks the value of `reset`. If `reset` is asserted, `count` is cleared to `4'b0000` on that edge. If `reset` is not asserted, `count` is incremented by one. Because `count` is four bits wide, an increment applied to `1111` wraps around to `0000`, which is the overflow behavior described in Chapter 6.

TODO: Insert DUT block diagram

Figure 7.4: Block diagram of the 4-bit Synchronous Counter, showing the `clk` and `reset` inputs and the `count` output.

The RTL for this module is unchanged from Chapter 6 and is reproduced here for reference:

```verilog
module synchronous_counter_4bit(clk, reset, count);
    input clk, reset;
    output reg [3:0] count;

    always @(posedge clk) begin
        if (reset)
            count <= 4'b0000;
        else
            count <= count + 1'b1;
    end
endmodule
```

7.8.4 Testbench Architecture

The verification environment for this exercise follows the architecture introduced in Section 7.2. A top-level testbench module, `synchronous_counter_4bit_tb`, contains no ports of its own and is responsible for instantiating the DUT, generating the clock, generating the reset sequence, applying stimulus, and monitoring the output.

The DUT is instantiated as `dut`, with its `clk`, `reset`, and `count` ports connected to corresponding signals declared inside the testbench. A clock generator, built using the `always` block construct from Section 7.5, produces a continuous 10 ns-period clock for the duration of the simulation. A separate initial block generates the reset sequence and the overall timing of the test, asserting `reset` at the start of simulation, releasing it to allow the counter to run freely, and asserting it again partway through the simulation to confirm that the counter can be returned to a known state after it has already been counting. Output monitoring is performed using `$monitor`, which prints the value of `reset` and `count` whenever either signal changes.

Figure 7.5: Verification environment for the 4-bit Synchronous Counter, showing the testbench module containing the DUT instance, clock generator, reset and stimulus generation, and output monitoring.

Each of these components plays a distinct role. The clock generator establishes the timing reference against which all state updates occur. The reset sequence establishes and re-establishes a known starting condition. The stimulus duration determines how many clock cycles the counter is allowed to run, which must be sufficient to observe the full count sequence and at least one wrap-around. The monitoring statement provides a textual, time-stamped record of the counter's behavior that can be compared directly against the expected state sequence from Table 6.3 and Table 6.4.

7.8.5 Testbench Implementation

The complete testbench is constructed in stages, beginning with the module declaration, signal definitions, and DUT instantiation.

```verilog
`timescale 1ns/1ps

module synchronous_counter_4bit_tb;

    reg clk, reset;
    wire [3:0] count;

    synchronous_counter_4bit dut (
        .clk(clk),
        .reset(reset),
        .count(count)
    );

endmodule
```

The `‘timescale` directive establishes the simulation time unit and precision, consistent with the testbenches used in earlier chapters. The signals `clk` and `reset` are declared as `reg`, since they are driven by the testbench, while `count` is declared as `wire`, since it is driven by the DUT. The DUT is instantiated as `dut`, with each of its ports connected by name to the corresponding testbench signal — the same connection style used for the Full Adder instances in the Ripple Carry Adder testbench in Chapter 5.

The next addition is the clock generator, using the standard construct introduced in Section 7.5:

```verilog
    initial begin
        clk = 1'b0;
    end
```

```
    always #5 clk = ~clk;
```

The `initial` block establishes a known starting value of `0` for `clk` at simulation time 0. The `always` block then toggles `clk` every 5 ns, producing a clock signal with a 10 ns period and a 50% duty cycle. As discussed in Section 7.5, this block runs continuously for the remainder of the simulation and does not, by itself, determine when the simulation ends.

The reset sequence and overall stimulus timing are provided by a second `initial` block:

verilog

```
    initial begin
        $monitor($time, " reset=%b count=%b", reset, count);

        reset = 1'b1;
        #12;
        reset = 1'b0;
        #180;
        reset = 1'b1;
        #10;
        reset = 1'b0;
        #40;
        $finish;
    end
```

This sequence proceeds as follows. The `$monitor` statement is registered first, so that every subsequent change to `reset` or `count` is printed with its associated simulation time. `reset` is then asserted at time 0 and held for 12 ns. Because the clock period is 10 ns, this guarantees that at least one rising edge of `clk` occurs while `reset` is asserted, forcing `count` to `4'b0000` on that edge. `reset` is then released and held low for 180 ns — eighteen clock cycles — which is more than enough for the counter to advance from `0000` through `1111` and wrap around to `0000` at least once. `reset` is asserted a second time for 10 ns, confirming that the counter can be returned to `0000` after it has already been running, and is then released for a final 40 ns before `$finish` ends the simulation.

The complete testbench, combining all of the segments above, is as follows:

verilog

```
`timescale 1ns/1ps

module synchronous_counter_4bit_tb;

    reg clk, reset;
    wire [3:0] count;

    synchronous_counter_4bit dut (
        .clk(clk),
        .reset(reset),
        .count(count)
    );

    initial begin
        clk = 1'b0;
    end

    always #5 clk = ~clk;

    initial begin
        $monitor($time, " reset=%b count=%b", reset, count);
```

```
        reset = 1'b1;
        #12;
        reset = 1'b0;
        #180;
        reset = 1'b1;
        #10;
        reset = 1'b0;
        #40;
        $finish;
    end

endmodule
```

This testbench satisfies each of the requirements for the exercise. The DUT is instantiated and connected to a generated clock and a controlled reset sequence. The counter is initialized through the first reset pulse, allowed to increment freely through enough cycles to wrap around at least once, and then returned to a known state through the second reset pulse. The `$monitor` statement provides a continuous textual record of `reset` and `count`, while the waveform viewer, opened during the simulation procedure that follows, provides a graphical record of the same behavior.

7.8.6 Simulation Procedure

The simulation should be carried out using the same workflow established in Chapters 4 through 6.

1. Create a new working directory or Cadence project for the Synchronous Counter testbench exercise.
2. Create the Verilog source file containing the `synchronous_counter_4bit` module, using the implementation from Chapter 6.
3. Create the testbench file containing `synchronous_counter_4bit_tb`, using the implementation from Section 7.8.5.
4. Add both Verilog files to the simulation environment.
5. Compile the design and confirm that no syntax or elaboration errors are reported.
6. Launch the simulation with `synchronous_counter_4bit_tb` as the top-level module.
7. Open the waveform viewer and add `clk`, `reset`, and `count` to the display.
8. Run the simulation to completion and verify the counter's behavior against the expected sequence described in the following section.

During compilation, errors are most commonly caused by mismatched module or port names between the DUT and the testbench, or by a missing source file in the simulation setup. During simulation, incorrect behavior in the printed `$monitor` output or in the waveform usually indicates either an incorrect reset polarity or an error in the increment logic of the DUT, and the relevant RTL should be reviewed against the implementation in Chapter 6.

7.8.7 Expected Results

During simulation, `clk` should be observed toggling continuously with a 10 ns period for the entire duration of the run, beginning from its initialized value of `0`. At simulation time 0, `reset` should be high, and by the first rising edge of `clk`, `count` should become `4'b0000`.

After `reset` is released, `count` should increment by one on each subsequent rising edge of `clk`, progressing through the binary sequence `0001`, `0010`, `0011`, and so on. The transitions in `count` should occur only at rising clock edges and should remain stable between edges, confirming the synchronous, clock-controlled behavior described in Chapter 6. As the simulation continues, `count` should reach `1111`

and, on the next rising edge, wrap around to `0000`, demonstrating the overflow behavior of the 4-bit register.

When `reset` is asserted a second time, `count` should return to `0000` on the next rising edge, regardless of its value at that point in the sequence, confirming that reset behaves correctly even after the counter has been running for an extended period.

Figure 7.6: Verification waveform for the 4-bit Synchronous Counter, showing `clk`, `reset`, and `count`, with the initial reset, binary count progression, wrap-around from `1111` to `0000`, and the second reset pulse all visible.

The waveform confirms correct functionality when each of these behaviors is visible in the order described: initialization to `0000` under the first reset, uninterrupted binary counting at each rising clock edge, a single wrap-around from `1111` back to `0000`, and a return to `0000` under the second reset. Any deviation from this sequence — such as `count` changing between clock edges, failing to reset, or failing to wrap around — indicates an error in either the DUT or the testbench and should be investigated using the `$monitor` output as a time-stamped reference before the waveform is examined in detail.

7.8.8 Conclusion

This exercise demonstrated the development of a complete verification environment for the 4-bit Synchronous Counter introduced in Chapter 6. A dedicated testbench module was constructed that instantiated the DUT, generated a continuous clock signal, applied a reset sequence sufficient to establish and re-establish a known starting state, and allowed the counter to run through enough clock cycles to exhibit its full count sequence and wrap-around behavior.

The exercise applied, in combination, each of the constructs introduced earlier in this chapter: `initial` blocks for one-time initialization and stimulus sequencing, `always` blocks for continuous clock generation, simulation delays for controlling the timing of the test, and `$monitor` for textual observation of the design's response. The simulation confirmed, through both the printed output and the waveform viewer, that the counter initializes correctly, increments on each rising clock edge, wraps around from `1111` to `0000`, and responds correctly to reset at any point during operation.

The verification methodology demonstrated in this exercise — instantiating a DUT, generating clock and reset signals, applying stimulus over a defined simulation period, and observing the result — forms the foundation for the more advanced verification activities discussed in the chapters that follow, including detailed waveform-based debugging, simulation-driven analysis of RTL bugs, and coverage measurement using IMC.

08

Simulation and Waveform Analysis

System identity, licensing, and workstation preparation

8.1 Compilation

The testbenches developed in Chapter 7 cannot be simulated directly. Before the Cadence Xcelium simulator can execute a single line of stimulus, the Verilog source files describing both the DUT and the testbench must first pass through compilation. Compilation is the stage at which the simulator reads each source file, parses its contents according to the rules of the Verilog language, and checks that the code is syntactically well formed. It is, in effect, the same role played by a compiler in conventional software development, except that the unit of compilation here is a hardware description rather than an executable program.

During compilation, the simulator does not yet build a connected model of the design. It works through each file independently, identifying module declarations, port lists, signal declarations, and procedural blocks, and confirming that every statement obeys Verilog's grammar. This is also the stage at which module discovery takes place: the simulator records every module definition it encounters across the supplied source files, so that later stages can locate and connect them by name. A design that compiles successfully is therefore not yet known to be functionally correct, or even fully connected — it is only known to be syntactically valid and to consist of recognizable module definitions.

Compilation is deliberately the first stage of the flow because it is the least expensive point at which a coding mistake can be caught. A typographical error, a missing keyword, or an incorrectly formed declaration produces an immediate, specific compiler message rather than a confusing or silent failure later during simulation. For this reason, compilation errors should always be resolved completely, and in the order reported, before moving on to elaboration.

Several categories of error are routinely detected at this stage. A missing semicolon at the end of a statement is among the most common, since Verilog uses the semicolon to delimit individual statements within a procedural block:

```verilog
module bad_example(A, B, Y);
    input A, B;
    output Y;

    assign Y = A & B
endmodule
```

The missing semicolon after `assign Y = A \& B` causes the compiler to report a syntax error, frequently pointing not at the line containing the actual mistake but at the line immediately following it, since the parser does not recognize the end of the statement until it encounters the next token. This is a useful detail to

remember when interpreting compiler output: if a reported line appears syntactically correct on its own, the actual fault is often on the line immediately above it.

A second common category is the use of an undeclared signal. Every net or variable referenced inside a module must be declared, either in the port list or in an internal declaration, before it is used:

```verilog
module bad_mux(I0, I1, sel, Y);
    input I0, I1, sel;
    output Y;

    assign Y = sel ? I1 : I2;
endmodule
```

Here, the signal `I2` is referenced in the conditional expression but was never declared as a port or as an internal signal — the intended signal was `I0`. Depending on the simulator's configuration, this may be reported as an undeclared identifier error during compilation, or, in tools that implicitly declare undeclared one-bit nets, it may compile without error and instead produce an incorrect result during simulation, since `I2` would silently default to a net carrying no meaningful value. For this reason, laboratory work in this manual treats any implicitly declared signal as a defect to be corrected, regardless of whether the compiler flags it directly.

A third category involves the module declaration itself, including a mismatch between the ports listed in the module header and the ports referenced inside the body, or a missing `endmodule` keyword at the close of a module. Since Verilog modules do not use indentation to define scope, a missing `endmodule` can cause the compiler to interpret an entire subsequent module as if it were nested inside the previous one, producing a cascade of unrelated-looking errors from a single missing keyword.

Finally, compilation can fail simply because a required source file was never supplied to the simulator. If the testbench instantiates a DUT module that has not been compiled — because its file was omitted from the project, misnamed, or stored in the wrong directory — the simulator will report that the module cannot be found. This category of error is procedural rather than syntactic, and is corrected by reviewing the project's file list rather than the Verilog source itself.

Figure 8.1: Position of compilation within the Cadence simulation flow, showing source files being parsed and checked before elaboration begins.

Compilation, therefore, occupies the first position in the simulation flow: source files in, syntactic confirmation out. It establishes that the supplied Verilog text is well formed and that every module referenced elsewhere in the project has been recognized. It does not yet confirm that those modules are correctly connected to one another, which is the task of the next stage, elaboration.

8.2 Elaboration

Once every source file has compiled successfully, the simulator proceeds to elaboration. Where compilation processes each file independently, elaboration is the stage at which the simulator constructs the actual design hierarchy implied by those files: it resolves which module instantiates which, expands every instance into its place in that hierarchy, connects ports between modules according to the instantiation statements, and resolves any parameters that affect signal widths or behavior.

It is useful to think of elaboration as the point at which the simulator builds a concrete model of the circuit described by the source code. Compilation confirms that the `ripple\_carry\_adder\_4bit` module and the `full\_adder` module are each individually well formed; elaboration is what actually creates four

instances of `full\_adder`, labeled `FA0` through `FA3`, and wires their ports together exactly as specified in the instantiation statements introduced in Chapter 5. The testbench occupies the top of this hierarchy, with the DUT — and, in turn, any modules the DUT itself instantiates — nested beneath it. Elaboration is what links the testbench and the DUT into a single connected design rather than two independently verified but unconnected pieces of Verilog.

Several categories of error are specific to this stage and cannot be detected during compilation, because they depend on how modules are connected rather than on whether any individual module is syntactically valid. A port mismatch occurs when an instantiation connects signals to a module's ports incorrectly, either by supplying the wrong number of connections in positional instantiation, or by misspelling a port name in named instantiation:

```verilog
full_adder FA1 (
    .A(A[1]),
    .B(B[1]),
    .Cinn(c1),
    .Sum(Sum[1]),
    .Cout(c2)
);
```

In this instantiation, the port name `Cinn` does not match any port declared in the `full\_adder` module — the correct name is `Cin`. Named port connections of this kind compile without difficulty, since each line is syntactically valid on its own, but elaboration fails because the simulator cannot match `Cinn` to any port in the module being instantiated. This category of error is corrected by checking the instantiation against the exact port list declared in the module header, rather than against the order in which signals appear in the source.

A related error arises from width mismatches between a port and the signal connected to it. If a four-bit bus is connected to a port declared as a single bit, or vice versa, the elaborator may either truncate the connection, replicate it, or report a width mismatch, depending on the simulator's configuration and the severity settings in use. Because a silently truncated connection can produce a design that elaborates and simulates without any reported error, while still behaving incorrectly, width mismatches are one of the more dangerous classes of elaboration-time defect, and instantiations should always be checked manually against the declared widths in the module header.

A third category involves an instantiated module that does not exist at all — for example, where a top-level testbench references a module name that was renamed, deleted, or never written. Unlike the missing-file case discussed in Section 8.1, this scenario can occur even when every supplied file compiles correctly, if the name used in the instantiation simply does not correspond to any module the simulator has seen. The resulting message identifies the unresolved instance, and the fix is to confirm that the module name in the instantiation statement exactly matches the module name declared with the `module` keyword in its source file.

Figure 8.2: Hierarchy constructed during elaboration, showing the testbench module at the top level, the DUT instance beneath it, and any further instances — such as individual Full Adder stages — nested within the DUT.

A design that elaborates successfully is, at this point, a fully connected model: every instance has been expanded, every port has been connected to a signal of the expected width, and the complete hierarchy from the testbench down to the smallest leaf module exists as a single structure inside the simulator. What remains is to advance that structure through simulated time and observe how its signals behave, which is the task of simulation itself.

8.3 Simulation

Simulation is the stage at which the elaborated design is actually executed. Having confirmed that the source code is syntactically valid and that the design hierarchy is correctly connected, the simulator now advances

through simulated time, evaluating the behavior described by the RTL and the testbench, and producing the signal values that will later be examined in the waveform viewer.

Cadence Xcelium, like essentially all digital logic simulators, is event-driven rather than driven by a fixed time step. This means the simulator does not evaluate every signal in the design at every possible instant. Instead, it maintains a queue of pending events — a signal changing value, a delay expiring, a clock edge occurring — and advances simulated time only as far as the next scheduled event requires. Between events, no computation occurs, because no signal in the design has any reason to change. This is why a simulation containing long, idle delays does not consume proportionally long simulation runtimes: the simulator skips directly from one event to the next rather than stepping through every intervening nanosecond.

The behavior of the testbench during simulation follows directly from the constructs introduced in Chapter 7. `initial` blocks begin executing at simulation time 0 and run through their statements in order, pausing only at explicit delay controls, exactly as described in Section 7.3. `always` blocks, by contrast, remain dormant until the event in their sensitivity list occurs, at which point their body executes once before the block returns to waiting for the next occurrence of that event, as described in Section 7.4. A clock generator constructed using `always \#5 clk = \textasciitilde {}clk;` is, from the simulator's point of view, simply a recurring event: every 5 ns of simulated time, the value of `clk` changes, and that change is itself scheduled as a new event for any logic — inside the DUT or the testbench — that is sensitive to it.

Stimulus applied by the testbench propagates through the design according to this same event-driven principle. When a stimulus statement such as `A = 4'b0011;` executes inside an `initial` block, the simulator schedules an update to the signal `A`. Any combinational logic inside the DUT that depends on `A` — for instance, a continuous assignment such as those introduced in Chapter 5 — is re-evaluated as a consequence of that event, and its output is updated in turn. For a sequential design such as the Synchronous Counter from Chapter 6, a change to an input signal such as `reset` does not immediately change the registered output `count`; instead, the new value of `reset` is simply available the next time the `always @(posedge clk)` block in the DUT executes, which occurs only at the next rising edge of the clock. This distinction — immediate response for combinational logic, clock-synchronized response for sequential logic — is the same distinction introduced conceptually in Table 6.1, and it becomes directly observable once a simulation has been run and its results examined.

```verilog
initial begin
    A = 4'b0011;
    #1;
    $display($time, " A is now %b", A);
end
```

In this short example, the assignment to `A` and the subsequent `\$display` are separated by a 1 ns delay. At simulation time 0, `A` takes the value `0011`; at simulation time 1, the `\$display` statement executes and reports that value. The delay is what allows the simulator to advance from one event to the next in a controlled, observable sequence, rather than executing every statement at the same instant.

It is simulation, not compilation or elaboration, that constitutes verification in the sense introduced in Section 7.1. Compilation and elaboration confirm that a design is well formed and correctly connected; only simulation exercises the design's actual behavior against applied stimulus, and only simulation can reveal whether the DUT's response matches the behavior expected from its specification. A design can compile and elaborate without any reported error and still be functionally incorrect — which is precisely why the laboratory exercises throughout this manual treat a clean compile as a necessary, but never sufficient, condition for considering a design verified.

Figure 8.3: Event-driven progression of simulated time, showing stimulus events in the testbench triggering combinational and sequential responses in the DUT.

The console output produced by `\$monitor` and `\$display`, used throughout the testbenches in Chapter 7, provides one view of this behavior, but it is a strictly textual one: a sequence of printed lines, each tied to a single simulation time, with no direct sense of how signals relate to one another visually or how long a given value was held before changing. For anything beyond the smallest designs, this textual record becomes difficult to interpret efficiently. The standard practice in Cadence-based verification is therefore to record the simulation's signal activity to a waveform database and inspect that database using a waveform viewer such as SimVision, which is the subject of the next section.

8.4 Understanding Waveforms

A waveform viewer displays the value of one or more signals as a function of simulated time, using a graphical convention in which time advances from left to right along a horizontal axis and signal value is represented along the vertical extent of each signal's row. Rather than reading a long sequence of printed `\$monitor` lines, the designer can see, at a glance, exactly when a signal changed, how long it held a given value, and how its transitions relate to the transitions of every other signal displayed alongside it.

The horizontal time axis is labeled in the simulation time units established by the `` `timescale `` directive at the top of the source file, as introduced in Chapter 4. A waveform generated from a testbench using `` `timescale 1ns/1ps `` will display its time axis in nanoseconds, with cursor measurements available to the picosecond precision specified by the second argument of that directive. Reading the time axis correctly is the first step in interpreting any waveform: a transition that appears to occur "immediately" after another may, on closer inspection, be separated by a small but nonzero delay, which is often the detail that distinguishes correct sequential behavior from a subtle timing defect.

Each signal in the display is drawn as a trace whose vertical level indicates its logic value at each instant. For single-bit signals, the trace is drawn as a simple two-level waveform, low for a logic `0` and high for a logic `1`, with a vertical edge marking each transition. A transition from low to high is a rising edge, and a transition from high to low is a falling edge — the same terminology introduced for clock signals in Section 6.2, and the terminology used throughout this manual whenever a clock-sensitive event is described. For multi-bit signals, such as the 4-bit `count` output of the Synchronous Counter, the waveform viewer typically displays the bus as a single trace annotated with its current value in binary, decimal, or hexadecimal, rather than as four separate single-bit traces, since this is normally easier to read for a register whose value changes as a unit.

Two additional logic values appear in Verilog waveforms beyond `0` and `1`, both introduced conceptually in Chapter 4: `X`, representing an unknown or unresolved value, and `Z`, representing a high-impedance or undriven condition. In a waveform display, these are usually rendered in a distinct color or pattern — commonly red or amber for `X`, and a dashed or hollow trace for `Z` — so that they are visually distinct from a deliberate `0` or `1`. An `X` value appearing on an output signal almost always indicates a problem: either the signal has not yet been driven to a known value, or it is the result of a calculation involving another signal that is itself unresolved. A `Z` value appearing on a signal that is expected to be actively driven — rather than on a genuinely tri-stated bus — is equally suspicious and usually points to a missing or incorrect assignment somewhere in the design.

The waveform display makes the relationship between stimulus and response directly visible. Each input signal driven by the testbench's `initial` block appears as a trace whose transitions correspond exactly to the assignment statements in that block, while each output signal produced by the DUT appears as a trace whose transitions are a consequence of those inputs, mediated by the DUT's internal logic. For a combinational design such as the Ripple Carry Adder, the `Sum` and `Cout` traces should change at essentially the same simulated time as the `A`, `B`, and `Cin` traces that drive them, since there is no clock to delay that response. For a sequential design such as the Synchronous Counter, the `count` trace should remain perfectly flat between rising edges of `clk`, and should change only at those edges — a stable, unchanging signal between active edges is exactly the behavior expected of a clocked register, not a sign that the simulation has stalled.

Figure 8.4: Basic waveform display showing two single-bit input signals and a derived combinational output, illustrating rising edges, falling edges, and stable logic levels.

Applying this to the Synchronous Counter from Chapter 6, a correct waveform should show `clk` toggling at a constant period throughout the simulation, `reset` held high for the initial interval described in the testbench from Section 7.8, and `count` remaining at `0000` for as long as `reset` is asserted at a rising edge. Once `reset` is released, `count` should advance by exactly one binary value at each subsequent rising edge of `clk`, holding each value steady until the next edge, and should wrap around from `1111` back to `0000` after sixteen such transitions, exactly as described in Section 6.6.

Figure 8.5: Waveform of the 4-bit Synchronous Counter showing `clk`, `reset`, and `count`, with state transitions occurring only at rising clock edges and wrap-around visible from `1111` to `0000`.

It is this visual correspondence — between the stimulus applied and the response observed, laid out along a shared time axis — that makes the waveform viewer the primary tool for confirming functional correctness in Cadence-based verification. A textual report from `\$monitor` can confirm that a particular value occurred at a particular time, but a waveform makes the entire sequence of transitions, and the timing relationships between them, visible at once. This is also what makes the waveform viewer the primary tool for diagnosing a design that does *not* behave correctly, which is the subject of the remainder of this chapter.

8.5 Common RTL Bugs

Most of the defects encountered while verifying small RTL designs fall into a relatively small number of recurring categories. Recognizing these categories, and knowing how each one tends to appear in simulation and in the waveform viewer, considerably shortens the time required to locate and correct a defect.

Incorrect signal connections. The most frequent class of error is a simple miswiring: a port connected to the wrong signal, or two signals swapped relative to their intended positions. This was illustrated in Section 8.2 with a misspelled port name, but an equally common variant compiles and elaborates without any error at all, because every connection is syntactically and structurally valid — it is simply the wrong valid connection.

```verilog
full_adder FA0 (
    .A(B[0]),
    .B(A[0]),
    .Cin(Cin),
    .Sum(Sum[0]),
    .Cout(c1)
);
```

Here, `A[0]` and `B[0]` have been swapped relative to the ports they are connected to. Because addition is commutative, this particular mistake happens to produce a correct `Sum` output and would not be caught by a quick visual check of the waveform; an error of exactly this kind is much easier to find through a self-checking testbench of the kind introduced in Section 7.7, or by deliberately testing asymmetric operand values, than by inspecting the waveform alone. This is one of the clearest illustrations of why incorrect connections are dangerous: the symptom in simulation can range from an obviously wrong output to no visible symptom whatsoever, depending on the specific logic involved.

Missing reset logic. A sequential design that omits reset handling, or implements it incorrectly, typically shows its symptom directly after the start of simulation: the registered output begins in an unknown `X` state and may remain unknown indefinitely, since nothing in the design ever assigns it a defined starting value.

```verilog
module counter_no_reset(clk, count);
    input clk;
    output reg [3:0] count;

    always @(posedge clk)
        count <= count + 1'b1;
endmodule
```

Because `count` is never explicitly initialized and the module provides no reset input at all, `count` begins simulation as `xxxx` and, since incrementing an unknown value produces another unknown value, remains `xxxx` for the entire simulation. In the waveform viewer, this appears as a trace that never leaves its unknown-value color, even though `clk` continues to toggle normally. The correction is the synchronous reset structure introduced in Section 6.3 and used throughout the counter designs in Chapter 6: an explicit `if (reset)` branch inside the clocked `always` block that forces the register to a known value.

Width mismatches. As discussed in Section 8.2, connecting signals of different widths can silently truncate or zero-extend a value rather than producing an explicit error. A related but distinct mistake occurs entirely within RTL, when an intermediate signal is declared narrower than the value it needs to represent:

```verilog
wire [2:0] sum_wire;
assign sum_wire = A + B;
```

If `A` and `B` are each four bits wide, their sum can require five bits to represent without loss, but `sum\_wire` has been declared with only three bits. The most significant bits of the true sum are silently discarded. In simulation, this appears as a `sum\_wire` value that is correct for small operands but becomes incorrect — and which, because no `X` or `Z` is involved, gives no immediate visual warning in the waveform — once the operands grow large enough for the true sum to exceed three bits. This category of bug is best caught by deliberately including boundary-value test vectors, of the kind discussed in Section 7.6, rather than only typical mid-range values.

Incorrect sensitivity lists. A combinational `always` block must list every signal that the block reads, or the simulator will not re-evaluate the block when one of the omitted signals changes:

```verilog
always @(A)
    Y = A & B;
```

Here, the block is intended to describe a combinational AND function, but its sensitivity list includes only `A`. If `B` changes while `A` remains constant, the block does not re-execute, and `Y` continues to display its previous, now-incorrect value until the next time `A` happens to change. In a waveform, this produces a `Y` trace that appears to lag behind one of its inputs, holding a stale value for an interval where it should have updated immediately. The correction is either to list every relevant signal explicitly, as in `always @(A or B)`, or to use the wildcard form `always @(*)`, which instructs the simulator to infer the complete sensitivity list automatically — the safer and more commonly recommended option for combinational logic.

Misuse of blocking and non-blocking assignments. Chapter 6 introduced the non-blocking assignment operator `<=` as the standard form for clocked sequential logic, since it allows every register triggered by the

same clock edge to be updated using the values that existed before that edge, rather than values already updated earlier in the same procedural block. Using the blocking assignment operator `=` in place of `<=` inside a clocked block can produce results that depend on the order in which statements happen to be written, rather than on the intended hardware behavior:

```verilog
always @(posedge clk) begin
    q1 = d;
    q2 = q1;
end
```

The intention here is almost certainly a two-stage shift register, where `q2` should receive the *previous* value of `q1` at each clock edge, not its newly updated value. Because blocking assignments take effect immediately within the procedural block, `q2` instead receives the same new value as `q1` on the same edge, collapsing the intended two-stage delay into a single stage. In simulation, this appears as `q1` and `q2` updating in lockstep on every clock edge, rather than `q2` trailing `q1` by one cycle as a shift register requires. The correction is to use `<=` for both assignments, consistent with the convention established in every sequential design in Chapter 6.

Incorrect testbench stimulus. Not every defect originates in the DUT. A testbench that applies stimulus too quickly relative to the clock period, or that changes an input at the same simulated instant as a clock edge, can produce a result that depends on simulator-specific event ordering rather than on well-defined design behavior. A stimulus statement that omits a delay before checking a combinational output may sample that output before the design has had any opportunity to respond:

```verilog
A = 1; B = 1;
if (Y !== 1)
    $error("Unexpected Y value");
```

If no delay separates the assignment to `A` and `B` from the check on `Y`, the check executes in the same simulation time step as the stimulus, and depending on simulator event ordering, may observe `Y` before the continuous assignment driving it has been re-evaluated. This produces an intermittent, simulator-dependent false failure that has nothing to do with a defect in the DUT. The correction, consistent with every combinational testbench in this manual, is to insert a short delay — even `\#1` — between applying stimulus and checking the corresponding response.

Uninitialized signals. Finally, a signal that is read before it has ever been assigned a value will simulate as `X`, exactly as discussed under missing reset logic above, but this category extends beyond registers to ordinary testbench variables. A stimulus sequence that conditionally assigns a signal in some branches but not others can leave that signal at `X` for any branch that does not explicitly set it, producing intermittent `X` values in the waveform that depend on which branch of the testbench happened to execute. The general remedy, applicable to both RTL and testbench code, is the same discipline introduced in Section 4.5: every signal should be given a known value through an explicit path in the code, rather than relying on a default that may not exist.

This is not an exhaustive list of every mistake possible in Verilog RTL, but it accounts for the large majority of defects encountered in the laboratory designs covered by this manual. Recognizing the waveform signature associated with each category — a stalled `X`, a lagging output, two signals updating in unexpected lockstep, an intermittent failure tied to event ordering — is what allows a designer to move from "the simulation produced the wrong answer" to "the defect is most likely of this specific kind" within the first few seconds of

looking at a waveform, which is the starting point for the structured debugging process described in the next section.

8.6 Debugging Methodology

Locating the cause of an incorrect simulation result is more efficient when approached as a structured process rather than as an unstructured search through the RTL. The categories introduced in Section 8.5 describe *what* commonly goes wrong; this section describes *how* to proceed once a simulation has produced a result that does not match the expected behavior.

The first step is to observe the failure precisely, rather than reacting to a general impression that "the counter is broken." This means identifying exactly which signal carries an incorrect value, at exactly which simulation time, and exactly how that value differs from what was expected. A vague observation such as "the waveform looks wrong" is far less useful than a specific one such as "`count` remained at `0011` through two additional rising edges of `clk` instead of advancing to `0100` and `0101`." The more precisely the failure is described, the smaller the space of possible causes becomes.

Once the failure has been described precisely, it should be reproduced reliably. If a self-checking testbench of the kind introduced in Section 7.7 is in use, this typically means confirming that the same `\$error` is reported on every run, given the same stimulus. If the testbench relies only on `\$monitor` and visual waveform inspection, reproduction means re-running the simulation and confirming that the same incorrect value appears at the same simulation time. A failure that cannot be reproduced consistently is usually a symptom of the testbench timing issues described in Section 8.5, rather than a genuine defect in the DUT, and should be investigated as such before any RTL changes are made.

With a reproducible failure in hand, the next task is to identify the specific signal responsible for the incorrect behavior, which is not always the signal where the symptom was first noticed. An incorrect `Sum` output, for example, might originate from a fault in the `Sum` logic itself, or it might originate from an incorrect carry signal feeding into that logic from an earlier stage, as in the Ripple Carry Adder from Chapter 5. Tracing the source of the incorrect behavior means working backward through the design hierarchy from the symptom, adding intermediate signals — including internal signals not present at the top-level ports, such as the carry wires `c1` through `c3` — to the waveform viewer until the point at which a value first becomes incorrect can be located precisely.

Once that point has been located, the relevant RTL and testbench code should be inspected directly, with the specific defect categories from Section 8.5 in mind as a checklist: Is this signal connected to the correct port? Is reset logic present and correctly structured? Are bus widths consistent throughout this expression? Does this `always` block's sensitivity list include every signal it reads? Is this a blocking assignment where a non-blocking assignment was intended? Working through these questions against the actual source, rather than guessing at a fix, is what distinguishes disciplined debugging from trial and error.

Only after the cause has been identified with reasonable confidence should the design be modified. The modification should be the minimal change required to address the identified cause — not a broader rewrite undertaken in the hope that it will incidentally fix the problem, since a broader rewrite makes it considerably harder to confirm afterward which specific change was actually responsible for the correction. The simulation should then be re-run from compilation onward, since a change to RTL or testbench source invalidates any previously elaborated design, and the original failure condition should be checked explicitly to confirm that it no longer occurs.

Finally, the correction should be verified more broadly than the single failing condition that prompted it. A fix that resolves one specific test vector but has not been checked against the rest of the stimulus sequence may have addressed only a symptom rather than the underlying defect, or may have introduced a new defect elsewhere in the design. Re-running the complete testbench, rather than only the previously failing portion, is the appropriate final step before considering the design corrected.

1. Observe the failure precisely.
2. Reproduce the failure reliably.

3. Identify the specific signal responsible.
4. Trace the source of the incorrect behavior through the design hierarchy.
5. Inspect the relevant RTL and testbench code against known defect categories.
6. Modify the design with the minimal change required.
7. Re-run the simulation from compilation onward.
8. Verify the correction against the complete stimulus sequence.

Throughout this process, the simulator's own messages — compilation errors, elaboration warnings, and any `\$error` or `\$warning` output from a self-checking testbench — should be read carefully and in full, rather than skimmed past in order to reach the waveform viewer more quickly. A precise error message naming a specific signal, line number, or mismatched value is frequently sufficient on its own to identify the defect category from Section 8.5, without any further investigation being required. Equally, any assumption made about the design's behavior — "this block should only execute on the rising edge," "this bus is four bits wide everywhere it is used" — should be checked directly against the source code rather than taken on faith, since an incorrect assumption of this kind is itself one of the most common reasons debugging takes longer than it should.

Figure 8.6: Structured debugging workflow, from initial observation of a simulation failure through to verified correction of the underlying RTL or testbench defect.

This disciplined approach to debugging — precise observation, reliable reproduction, deliberate tracing, minimal correction, and broad re-verification — is not specific to small laboratory designs. It is the same approach required in larger verification environments, where hundreds or thousands of test vectors may be involved and where a single overlooked assumption can be considerably more expensive to track down than in the designs covered by this manual. The waveform-based techniques introduced in this chapter remain useful at that scale, but they are eventually supplemented by more systematic measures of how thoroughly a design has actually been exercised. That question — not simply whether a design passes the test vectors it has been given, but whether those test vectors are sufficient in the first place — is the subject of coverage analysis, introduced in Chapter 9.

8.7 Laboratory Exercise – Counter Waveform Analysis

Objective

The purpose of this experiment is to analyze simulation waveforms generated from the 4-Bit Synchronous Counter designed in Chapter 6 and to verify whether the circuit behaves in accordance with its specification. Rather than introducing new RTL, this exercise focuses entirely on the post-simulation stage: loading signal data into the waveform viewer, interpreting the time-domain behavior of each signal, and systematically identifying any deviations from expected operation.

By completing this exercise, the reader will develop practical competence in:

- **Waveform interpretation** — reading the time-domain record produced by the simulator and relating observed transitions to the intended hardware behavior.
- **Timing analysis** — verifying that state changes occur at the correct clock edges and that no setup or hold violations are visible at the register level.
- **Signal tracing** — following a value from testbench stimulus through the design hierarchy to its driven output, and confirming that no intermediate connection is broken or incorrectly conditioned.
- **Bug detection** — recognizing waveform symptoms that indicate an error in the RTL or testbench, including stuck signals, unknown states, incorrect reset behavior, and missing transitions.
- **RTL debugging** — applying a systematic diagnostic procedure to locate the source of each identified anomaly and confirming that correction restores the expected waveform.

These skills form the practical core of functional verification. A simulation that completes without error messages does not guarantee correct hardware behavior; only careful waveform inspection can confirm that the design operates as intended under all applied stimulus conditions.

Theory

Simulation produces a time-ordered record of every signal transition that occurs during execution. The waveform viewer presents this record graphically, placing time on the horizontal axis and signal values on the vertical axis. Each signal trace shows when that signal was driven, what value it held, and how long it remained stable before transitioning again. This visual representation allows an engineer to reason about circuit behavior in a way that console messages alone cannot support.

For combinational logic, the relationship between stimulus and response is straightforward: a change on an input produces a change on the affected outputs after a short propagation delay. Verification amounts to confirming that the output value matches the expected function of the current inputs.

Sequential circuits require additional care. The output of a clocked register does not respond to input changes continuously. It samples its input at a specific clock edge and holds the captured value until the next active edge. Waveform analysis of a sequential design must therefore account for the timing relationship between the data signal, the clock, and the registered output. A correct observation is not simply that the right value appears on the output, but that it appears at the right clock edge and remains stable throughout the subsequent clock period.

For a synchronous counter, every state transition is a direct consequence of a rising clock edge. The count output should change only at those edges, and the new value should be exactly one greater than the previous value, modulo the counter width. A waveform that shows transitions at any other time, or that shows incorrect next-state values, indicates an error in either the RTL or the testbench. Waveform analysis is therefore the primary tool for distinguishing between a design that functions correctly and one that merely produces output without violating simulation syntax rules.

Experimental Setup

This experiment uses the design files developed in previous chapters. No new RTL is required. The required components are:

- The `synchronous_counter_4bit` module from Chapter 6, implementing a 4-bit binary up-counter with synchronous reset.
- The `synchronous_counter_4bit_tb` testbench from Chapter 7, which generates the clock, applies reset, and allows the counter to run freely for several cycles before reasserting reset.
- The Cadence simulation environment, configured with both Verilog source files compiled and elaborated without errors.
- The Cadence waveform viewer, used to load and inspect the simulation output database after the simulation has completed.

Before beginning the waveform analysis, confirm that both source files compile without errors and that the simulation runs to completion. The `$finish` statement in the testbench terminates the simulation after all stimulus vectors have been applied. If the simulation exits early with an error, the waveform database may be incomplete, and the exercise cannot proceed correctly.

The following signals must be added to the waveform viewer for this experiment:

- `clk` — the simulation clock generated by the testbench.
- `reset` — the synchronous reset input applied by the testbench.
- `count[3:0]` — the 4-bit counter output produced by the design under test.

It is also useful to expand `count[3:0]` into its individual bit traces `count[3]`, `count[2]`, `count[1]`, and `count[0]` so that the contribution of each bit to the overall state can be observed independently. This separation is particularly helpful when debugging carry propagation or reset faults that affect only a subset of the output bits.

Figure 8.7: Cadence simulation environment with the synchronous counter project loaded, showing the source files, compilation status, and waveform viewer ready to receive signal traces.

Waveform Observation

Once the simulation has completed successfully, the waveform database should be loaded into the waveform viewer. The procedure below describes the sequence of operations required to produce a complete and interpretable waveform display.

1. **Launch the simulation.** From the Cadence simulation environment, compile both source files and run the simulation with `synchronous_counter_4bit_tb` as the top-level module. Confirm that the console reports no elaboration errors and that the simulation completes normally.

2. **Open the waveform viewer.** After the simulation finishes, open the waveform viewer from within the simulation environment. The viewer loads the signal database generated during the simulation run.

3. **Add the relevant signals.** Navigate to the design hierarchy panel and locate the top-level testbench instance. Add `clk`, `reset`, and `count[3:0]` to the waveform display. Expand `count[3:0]` into individual bit traces if the viewer supports hierarchical signal expansion.

4. **Adjust the time axis.** Use the zoom controls to expand the horizontal scale so that individual clock periods are clearly visible. A display range that shows between ten and twenty clock cycles simultaneously provides enough context to observe both reset behavior and several consecutive count transitions without losing sight of the overall stimulus sequence.

5. **Zoom into transitions.** Select specific regions of interest and zoom in further to examine the precise timing relationship between the rising edge of `clk` and the subsequent change on `count[3:0]`. The count output should update on the rising edge, not between edges.

6. **Inspect the reset interval.** Locate the initial period during which `reset` is asserted and confirm that `count[3:0]` remains at `4'b0000` throughout. Then locate the point at which `reset` is deasserted and observe whether the counter begins incrementing on the very next rising clock edge.

Figure 8.8: Waveform viewer display showing `clk`, `reset`, and `count[3:0]` after a complete simulation run of the 4-bit synchronous counter testbench.

A correct waveform will show several distinct phases that correspond directly to the stimulus sequence applied by the testbench. During the initial reset period, `reset` is high and `count[3:0]` holds the value `0000` regardless of clock activity. When `reset` falls, the counter begins incrementing. Each rising edge of `clk` advances the count by one binary step. After the counter passes through all sixteen states and reaches `1111`, the next rising edge returns it to `0000`, producing the expected overflow wrap-around. Later in the simulation, a second reset assertion reloads `0000` and the cycle continues from that point.

Signal Analysis

Waveform observation confirms that signals are present and that transitions occur. Signal analysis goes further: it verifies that each transition occurs at the correct time, that the new value following each transition is the expected value, and that the timing relationships between signals are consistent with the intended hardware behavior.

Clock Periodicity. The first signal to examine is `clk`. Measure the time between two successive rising edges in the waveform viewer. This interval should equal the clock period defined in the testbench, which is 10 ns for the testbench developed in Chapter 7 (produced by the `always #5 clk = ~clk` statement). Confirm that the period is uniform throughout the simulation. An irregular clock period indicates a fault in the testbench clock generator rather than in the design under test.

Rising Edge Identification. Once the clock is confirmed to be correct, locate individual rising edges. These are the moments at which the counter is permitted to update. No transition on `count[3:0]` should occur at any other time. If the counter changes value between rising edges of `clk`, the design is not operating synchronously and an error is present in the RTL.

Reset Behavior. Inspect the interval during which `reset` is asserted. The `count[3:0]` output should remain at `4'b0000` for the entire duration, regardless of how many rising clock edges occur within that interval. The synchronous reset in the RTL is evaluated on the rising edge of `clk`; consequently, the hold at zero is edge-sampled rather than level-sensitive, and the output changes only at clock boundaries. When `reset` falls low, the counter should advance to `0001` on the first subsequent rising edge.

Counter Increment Sequence. After reset is released, the counter should progress through the following binary state sequence, advancing by exactly one step per rising clock edge:

Rising Edge Number	`count[3:0]`
1	0000
2	0001
3	0010
4	0011
5	0100
6	0101
7	0110
8	0111
9	1000
10	1001
11	1010
12	1011
13	1100
14	1101
15	1110
16	1111
17	0000

Edge number 1 in the table above corresponds to the first rising edge after `reset` is deasserted, at which point the count is already `0000` due to the preceding reset interval. Verify each row of this table against the waveform. A single incorrect entry identifies either a broken carry connection between bit positions or an error in the increment expression.

Overflow Wrap-Around. The transition from `1111` to `0000` is the most important individual event to examine. In a 4-bit register, the increment of `1111` by one produces `10000` in five bits; the most significant bit is discarded by the 4-bit width, and the register stores `0000`. This behavior should be visible directly in the waveform as an abrupt return to the zero state on the rising edge that follows `1111`. If the counter instead halts at `1111`, or if it jumps to an unexpected value, the increment expression or the register declaration contains an error.

Stable Output States. Between any two successive rising edges, `count[3:0]` should be perfectly stable. No glitch, partial transition, or momentary unknown value should be visible. A glitch mid-cycle on a specific bit suggests an asynchronous path that is not correctly gated by the clock, which would indicate a structural error in the RTL. In the synchronous counter design used in this exercise, such glitches should not be present because the counter is implemented entirely through a clocked `always` block.

Taken together, these observations confirm that the design satisfies the fundamental requirements of a synchronous 4-bit binary up-counter: it initializes to zero under reset, increments by one on each rising clock edge, holds each value stably between edges, and wraps around from the maximum value to zero without omitting any state.

Bug Identification and Debugging

Simulation waveforms frequently reveal behavioral anomalies that do not generate compilation errors or even simulation warnings. The engineer must recognize these anomalies from their visual signatures and trace each one back to its source in the RTL or testbench. This subsection presents a structured set of realistic debugging scenarios organized by symptom.

The general principle underlying every scenario below is the same: do not modify code in response to a symptom before forming a specific hypothesis about the cause. Random changes to an RTL file may coincidentally correct one problem while introducing another, and the resulting waveform may look correct

Figure 8.9: Systematic debugging workflow for waveform anomalies in synchronous RTL designs. Each anomaly type leads to a defined diagnostic path that narrows the possible causes before any code is modified.

even though the design logic remains flawed. A disciplined hypothesis-test-confirm cycle produces reliable results and is the standard practice in professional verification.

Scenario 1: Counter Not Incrementing. *Symptom.* The `reset` signal deasserts correctly and the `clk` signal toggles as expected, but `count[3:0]` remains at `0000` or at some other fixed value throughout the remainder of the simulation.

Probable Cause. The most common causes in order of likelihood are:

- The `reset` signal is not actually deasserted in simulation time even though it appears to be from a casual inspection. Verify the exact time at which `reset` falls in the waveform viewer and confirm that subsequent rising edges of `clk` occur after that point.
- The sensitivity list of the `always` block does not include `posedge clk`, causing the block to never execute during simulation.
- The increment expression uses a blocking assignment (`count = count + 1`) combined with an incorrect use of the variable, producing a zero result on every evaluation.
- A wire is used for `count` instead of a `reg`, and the synthesis tool or simulator rejects or silently ignores the procedural assignment.

Debugging Procedure. Check the `always` block sensitivity list first. Then verify the port declaration of `count` to confirm it is declared as `output reg`. Add the internal signals of the `always` block to the waveform display if the tool supports it, and confirm that the block executes at each rising edge.

Scenario 2: Counter Stuck at Zero. *Symptom.* The `count[3:0]` output displays `0000` throughout the entire simulation, including during the interval when reset should have been deasserted.

Probable Cause.

- `reset` is never driven low in the testbench. Inspect the `reset` trace and confirm that it actually falls to zero at the expected time. If `reset` remains high for the entire simulation, the counter will correctly hold at zero because the synchronous reset is always evaluated as true.
- The testbench drives `reset` correctly in simulation time, but the delay values prevent the deassertion from occurring before the `$finish` statement terminates the simulation.
- The `reset` port of the design under test is not connected to the `reset` register in the testbench, causing the design to receive a constant high value from the undriven wire.

Debugging Procedure. Examine only the `reset` trace first. If it never transitions to zero, the problem is entirely in the testbench rather than in the design. Verify the testbench `initial` block and check that the delay values allow `reset` to fall before `$finish` is reached. Then check the port connections in the instantiation statement.

Scenario 3: Unknown (`X`) States on Count Output. *Symptom.* The `count[3:0]` trace displays an unknown value, shown as `X` or as a cross-hatched region in the waveform viewer, either at the start of the simulation or at some point during normal counting.

Probable Cause.

- The register was never initialized. In simulation, an uninitialized `reg` starts at `X`. If the testbench does not assert `reset` before the first rising clock edge, the counter attempts to increment an unknown value and propagates `X` through subsequent states.
- A net or register used in the increment expression is itself uninitialized. If any operand of the addition is `X`, the result is also `X`.
- A testbench register used to drive a design input is declared but never assigned an initial value in the `initial` block.

Debugging Procedure. Locate the first clock edge at which an `X` value appears. If it is the very first edge, the initialization sequence in the testbench is the most likely cause. Confirm that `reset` is asserted before the first rising edge, and confirm that `clk` is initialized to a known value in the testbench. If the `X` appears mid-simulation, trace the affected signal back through its driver to the point of origin.

Scenario 4: Incorrect Reset Behavior. *Symptom.* When `reset` is asserted, `count[3:0]` does not return to `0000`. Instead it holds its current value, jumps to an arbitrary state, or produces an `X` output.

Probable Cause.

- The reset logic inside the `always` block is conditioned on the wrong polarity. If the design uses `if (!reset)` rather than `if (reset)`, the reset is active low, but the testbench is driving it active high, producing inverted behavior.
- The reset assignment in the `always` block uses a blocking assignment (`count = 4'b0000`) rather than a non-blocking assignment (`count <= 4'b0000`). This may produce correct results in some simulators but can cause unexpected behavior when the block is executed alongside other procedural code.
- The reset condition is placed in the wrong branch of the `if-else` statement, causing the counter to increment instead of clearing.

Debugging Procedure. Isolate the `reset` and `count` traces and examine the precise clock edge at which `reset` is asserted. Verify whether `count` changes at all on that edge. If it increments when it should clear, check the branch ordering in the `if-else` construct. If `count` does not respond at all, verify reset polarity and confirm that the reset variable in the testbench is connected to the reset port of the design instance.

Scenario 5: Unexpected Mid-Cycle Transitions. *Symptom.* The `count[3:0]` output changes value at a time other than a rising edge of `clk`. This may appear as a brief glitch or as a sustained transition that does not align with any clock event.

Probable Cause.

- The `always` block sensitivity list includes signals other than `posedge clk`, such as `always @(count)` or `always @(*)`. This causes the block to execute combinatorially rather than on clock edges, making the output change whenever any input signal changes.
- A continuous assignment (`assign`) drives a signal that is also driven inside the `always` block, creating a conflict that causes unpredictable transitions.

Debugging Procedure. Review the `always` block sensitivity list. For synchronous sequential logic, the sensitivity list must contain only `posedge clk`, or `posedge clk or negedge reset` if an asynchronous reset is used. Confirm that no continuous assignment drives `count` in parallel with the procedural block.

Scenario 6: Missing Clock Toggles. *Symptom.* The `clk` trace shows a constant high or low value rather than a periodic waveform. The counter consequently never updates.

Probable Cause.

- The `clk` register in the testbench is initialized to `0` but the `always #5 clk = ~clk` statement is missing or contains a syntax error that prevents it from executing.
- The initial block drives `clk` to a value and then falls off the end without spawning the toggle process, leaving the signal static.
- The `$finish` statement in the testbench executes before the first clock edge occurs, terminating the simulation immediately.

Debugging Procedure. Examine the `clk` trace first. If it is static, the testbench clock generator is the problem and the design itself need not be examined at all. Confirm that the `always` block for clock generation exists in the testbench, that it is syntactically correct, and that the initial value of `clk` is explicitly set before the simulation begins.

General Debugging Principle. In every scenario above, the diagnostic procedure follows the same high-level workflow: examine only the affected signal first, identify the clock edge at which the anomaly begins, trace the signal back through its driver, form a specific hypothesis about the RTL or testbench cause, verify the hypothesis by inspecting the source code, make a targeted correction, and rerun the simulation to confirm that the waveform now matches the expected behavior. Waveform analysis is not a process of eliminating code at random; it is a structured investigation in which each step narrows the field of possible causes until only one remains.

Expected Results

A correctly implemented and verified simulation of the 4-bit synchronous counter should produce a waveform exhibiting the following characteristics.

- **Stable clock waveform.** The `clk` signal toggles at a constant 10 ns period throughout the entire simulation. No edge is missing, no period varies, and the waveform begins immediately at time zero.
- **Proper reset behavior.** During the initial reset interval, `count[3:0]` holds `0000` and does not change at any rising clock edge. When `reset` is deasserted, the counter begins incrementing on the first subsequent rising edge without delay. When `reset` is reasserted later in the simulation, the counter returns to `0000` on the next rising edge from whatever value it holds at that moment.
- **Sequential count progression.** After reset is released, the output steps through the full 4-bit binary sequence from `0000` to `1111`, advancing by exactly one on every rising edge of `clk`. No state is repeated out of sequence and no state is skipped.
- **Overflow wrap-around.** On the rising edge that follows `1111`, the output transitions directly to `0000` without pausing at any intermediate value. This confirms that the 4-bit register width is handled correctly and that no carry is propagating into a fifth bit.
- **Absence of undefined states.** No `X` or `Z` value appears on `count[3:0]` at any point during the simulation. All four bits are driven to known binary values from the first reset edge onward.
- **Clock-edge-only transitions.** Every transition visible on `count[3:0]` coincides precisely with a rising edge of `clk`. No glitch, partial transition, or mid-cycle change is visible anywhere in the waveform.

Figure 8.10: Final validated simulation waveform for the 4-bit synchronous counter. The display shows `clk`, `reset`, and `count[3:0]` across the complete testbench stimulus sequence, confirming initialization, sequential increment, overflow wrap-around, and correct response to the second reset assertion.

The waveform in Figure 8.10 constitutes the acceptance criterion for this exercise. A simulation whose output matches this reference in every observable detail has been verified to the level of functional correctness achievable through directed simulation. Any departure from the reference behavior, however small, should be treated as an open finding and debugged to resolution before the design is carried forward to the synthesis stage.

Conclusion

This experiment demonstrated the complete waveform analysis workflow applied to the 4-bit synchronous counter developed in the preceding chapters. Rather than introducing new RTL code, the exercise focused on what the simulation output reveals about the behavior of an already-written design and on how to interpret that output systematically.

The waveform observation phase established the correct procedure for loading simulation results, adding signals to the viewer, and adjusting the display so that timing relationships are clearly visible. The signal analysis phase applied that procedure to verify clock periodicity, reset behavior, binary state progression, overflow wrap-around, and output stability between clock edges. Together these steps confirmed that the counter satisfies its functional specification under the stimulus conditions applied by the testbench.

The debugging section addressed the most common failure modes encountered in practice. Each scenario illustrated how a specific symptom in the waveform can be traced to a specific cause in the RTL or testbench,

and each concluded with a targeted diagnostic procedure rather than a suggestion to edit code at random. This structured approach is applicable to any sequential design, not only to the counter studied here.

Waveform analysis and systematic debugging form the practical foundation upon which all subsequent verification activities are built. Confirming functional correctness through directed simulation, as practiced in this exercise, is the necessary first step before coverage analysis can identify which behaviors remain untested. The following chapter introduces coverage analysis using the Cadence Integrated Metrics Center and explains how coverage-driven verification extends the confidence established by simulation into a measurable, closeable quality metric.

09

Coverage Analysis using IMC

System identity, licensing, and workstation preparation

9.1 Why Simulation Alone is Not Enough

Every design verified in the preceding chapters was confirmed correct by running a simulation and observing that the outputs matched expectations. The Half Adder produced the correct Sum and Carry for all four input combinations. The Ripple Carry Adder produced the correct four-bit result across a selection of operand pairs. The Synchronous Counter incremented on every rising clock edge, reset to zero when instructed, and wrapped around from `1111` to `0000` as required. In each case, the simulation passed, and the design moved forward.

The critical question that was never asked in those exercises is a simple one: did the simulation actually exercise every behavior the design is capable of? Passing a simulation means that the stimulus applied by the testbench produced the expected response from the design under test. It does not mean that every possible behavior of the design was tested, or that every logic path inside the RTL was evaluated at least once. A design that passes every test vector it has been given may still contain errors in logic that the testbench never reached.

Consider the Synchronous Counter from Chapter 6. Its testbench applied a reset pulse at the beginning of the simulation, allowed the counter to run through its full sixteen-state sequence, and then applied a second reset partway through. Suppose a slightly different testbench had been written — one that initialized the counter and allowed it to count, but omitted the second reset assertion entirely. The simulation would still complete without any reported error. The `\$monitor` output would still show the correct incrementing sequence. The waveform would appear correct. Yet an important behavioral condition — reset asserting while the counter is mid-sequence — would never have been exercised. If the reset logic in the RTL contained a subtle defect that only manifested after the counter had already been running, no amount of successful simulation output would reveal it.

This is not a hypothetical concern. It reflects a structural limitation of directed simulation as a verification methodology. Directed simulation applies a fixed, manually constructed sequence of test vectors to the design. The quality of the verification is directly bounded by the quality and completeness of the stimulus. If the stimulus was carefully designed, coverage will be high. If the stimulus was chosen hastily or based only on the most convenient operating conditions, entire portions of the design may go untested, and no tool will automatically flag this gap unless the designer has explicitly arranged for it to be measured.

The implications become more significant as designs grow larger. A 4-bit counter has sixteen states, and it is straightforward to confirm by inspection that a testbench exercises all of them. A finite state machine with thirty states and dozens of conditional transitions is not so easily audited by inspection. A datapath with multiple input registers, arithmetic units, and control signals can be configured in thousands of distinct ways, only a fraction of which can be covered by any practical directed testbench. For designs of this complexity, it is not merely inconvenient to rely on inspection alone — it becomes genuinely unreliable. An engineer who has written a testbench covering several hundred test vectors may have exercised the most common operating conditions thoroughly while leaving an entire class of edge-case behavior untouched.

Coverage analysis addresses this problem by providing a quantitative measurement of how thoroughly the simulation has exercised the design. Rather than asking whether the simulation passed, coverage analysis asks what portion of the design was actually reached by the simulation. This shifts the verification question from a binary outcome — pass or fail — to a continuous metric: how much of the design space has been explored, and what remains unexplored. The answer to that question is what guides the testbench engineer toward the test vectors that are still missing, before those missing scenarios become silicon defects.

The remainder of this chapter introduces the main categories of coverage measurement used in digital verification, explains how each one is collected by the Cadence simulation environment, and describes how the Cadence Integrated Metrics Center is used to inspect and act on the resulting data. Each metric provides a different perspective on the same underlying question: not whether the design passed a given set of tests, but whether those tests were sufficient to give meaningful confidence that the design is correct.

9.2 Code Coverage

The most direct form of coverage measurement is code coverage, which tracks which lines or statements within the RTL source were actually evaluated during simulation. The underlying idea is straightforward: if a line of Verilog was never executed during any simulation run, then the behavior described by that line was never tested. Whatever logic that line represents — a conditional branch, an assignment, an arithmetic operation — it remains unverified, and any defect in it would have gone undetected regardless of how many test vectors were applied to the rest of the design.

Statement coverage, which is the most basic form of code coverage, records whether each individual executable statement in the RTL was reached at least once during the simulation. Each `assign`, each line within a procedural block, and each statement within a conditional branch is tracked separately. After the simulation completes, the coverage tool can report exactly which statements were evaluated and which were not. An uncovered statement is a direct indicator that some part of the design logic was not exercised, and it becomes a prioritized item for testbench improvement.

Line coverage is closely related and is often reported alongside statement coverage. Where statement coverage is concerned with logical execution units, line coverage is concerned with source file line numbers. In practice, for the RTL coding styles used in this manual — where each statement typically occupies its own line — the two metrics tend to agree closely. The distinction becomes more apparent in dense, multi-statement lines or in macro-generated code, neither of which is common in the beginner-level RTL covered by these exercises.

It is important to understand what code coverage does and does not measure. A high code coverage percentage indicates that the simulation visited a large fraction of the source code. It does not indicate that those visits produced the correct result. A statement can be executed many times while producing a wrong output on every occasion, and code coverage will still report it as covered. Coverage measures breadth of execution, not correctness of execution. For that reason, code coverage is a necessary supplement to directed testing, not a replacement for it. The appropriate interpretation is that low code coverage provides strong evidence of insufficient stimulus, while high code coverage is a prerequisite for, but not a guarantee of, thorough verification.

In the context of the Synchronous Counter design used throughout this manual, statement coverage would confirm whether each branch of the clocked `always` block was evaluated. If the testbench never deasserted the reset input, the `else` branch — the line that increments the counter — would show zero coverage, immediately revealing that the normal counting behavior was never tested. Conversely, if reset was never asserted after the first initialization, the `if (reset)` branch might show limited coverage after the initial reset period, depending on how the simulator counts evaluations. These direct, actionable indicators are what make code coverage a valuable first pass in the coverage analysis workflow.

9.3 Branch Coverage

Code coverage at the statement level confirms that a line of RTL was reached, but it does not confirm that every possible outcome of a decision point was exercised. Branch coverage addresses this by tracking, for each conditional construct in the RTL, whether both the true and false outcomes were taken at least once during simulation.

Every `if` statement in a Verilog RTL description defines a decision point with at least two possible outcomes: the branch taken when the condition is true, and the branch taken when the condition is false. In a design of any meaningful complexity, these decision points govern the routing of control flow, the selection of register next-state values, and the behavior of the circuit under different operating conditions. If only one outcome of a given conditional is ever exercised, the logic implementing the other outcome remains unverified, regardless of how many times the statement itself was reached.

The significance of this can be illustrated with the increment logic from the Synchronous Counter. The core of the RTL is a single conditional statement inside a clocked block:

```verilog
always @(posedge clk) begin
    if (reset)
        count <= 4'b0000;
    else
        count <= count + 1'b1;
end
```

This construct defines two branches. The true branch, executed when `reset` is asserted, clears the counter to zero. The false branch, executed during normal operation, increments the count by one. If a testbench applies reset continuously throughout the simulation without ever releasing it, the false branch is never taken, and the increment logic is never tested. Code coverage might still report the `if` statement as executed, because the condition was evaluated. Branch coverage, however, would report the false outcome as not covered, providing an immediate and unambiguous signal that the counting behavior was never observed.

Branch coverage is especially valuable in control-intensive designs, where the majority of the RTL consists of conditional statements, case constructs, and selection logic. A multiplexer selects between inputs based on a control signal; if only one setting of that control signal is ever applied, the other input paths remain unverified. A finite state machine advances through states based on conditional transitions; if several transition conditions are never triggered, the corresponding next-state logic is invisible to the simulation. In both cases, branch coverage provides the measurement needed to identify and close those gaps.

A `case` statement extends this concept to multiple branches simultaneously. Each arm of the case, including the `default` arm if one is present, constitutes a separate branch in the coverage model. For a 2-bit input, a case statement has four possible branches. A testbench that only exercises two of the four input values leaves the other two branches uncovered, and branch coverage will report exactly this. This makes branch coverage a natural tool for verifying that all cases in a case statement have been tested, which is a common requirement in sequential state machine verification.

9.4 Toggle Coverage

Statement and branch coverage describe the behavior of the control flow paths within the RTL. Toggle coverage describes the behavior of the signals themselves. Specifically, it measures whether each signal in the design was observed to transition from logic zero to logic one, and from logic one to logic zero, at least once during the simulation.

The motivation for toggle coverage begins with a simple observation: a signal that never changes its logic value during simulation provides no information about the circuits that depend on it. If a control net is stuck at zero throughout the entire simulation, any logic gated by that net cannot have been exercised in its active

state. The combinational paths that depend on it were never evaluated with that input high. The registered values that it enables or suppresses were never captured under those conditions. Whatever behavior the design exhibits when that signal is asserted remains entirely unobserved.

In a direct-drive laboratory testbench, toggle coverage failures often reveal straightforward omissions. A reset signal that is driven high at the start of the simulation but never asserted again will fail the high-to-low transition check for the false-to-true direction, depending on its polarity. A select input to a multiplexer that is held at a fixed value for the entire simulation will fail toggle coverage on that signal entirely, regardless of how many test vectors were applied to the data inputs. These failures are simple to interpret and simple to correct: the testbench must be extended to drive the missing transitions.

Toggle coverage is also a useful detector of structural problems in the design hierarchy. A signal that is declared and connected within the RTL but that never toggles in simulation may indicate a disconnected net, an incorrectly driven output, or a design condition that is architecturally unreachable given the applied stimulus. In larger designs, toggle coverage analysis is sometimes used as an initial health check during debug bring-up, because a significant number of non-toggling signals in an unexpected portion of the hierarchy can indicate a wiring error or a missing clock enable that would not be visible from inspection alone.

Two specific signals deserve mention in the context of the Synchronous Counter design. The clock input `clk` should toggle for the entire duration of the simulation, because the testbench generates it using a continuous `always` block toggle construct introduced in Chapter 7. If the clock fails to toggle, no register updates can occur, and the entire design is effectively frozen. Toggle coverage on the clock input therefore serves as a basic sanity check on the testbench's clock generator. The reset input `reset`, on the other hand, must be both asserted and deasserted during the simulation to achieve full toggle coverage, which is why the testbench in Chapter 7 applied two distinct reset pulses rather than a single one at initialization. Viewing toggle coverage data after a simulation run can confirm whether this requirement was satisfied, and can flag incomplete testbenches that only drive one polarity of a control signal.

9.5 Functional Coverage

The coverage metrics introduced in the preceding sections — code, branch, and toggle — are all derived automatically from the structure of the RTL source code. They measure properties of the implementation: which statements executed, which decision paths were taken, which nets changed value. This automatic derivation is both a strength and a limitation. It is a strength because it requires no additional effort from the engineer beyond enabling the coverage collection; the tool discovers the measurable quantities from the code itself. It is a limitation because an RTL implementation may not capture all of the behaviors that matter from a specification perspective. A design can achieve one hundred percent code and branch coverage while never having visited a meaningful corner case that the specification explicitly requires to be verified.

Functional coverage addresses this by measuring verification completeness in terms defined by the engineer rather than terms derived from the RTL. Where structural coverage asks whether the implementation was exercised, functional coverage asks whether the specification was exercised. The engineer defines a set of coverage goals — often called cover points — that correspond to meaningful scenarios in the design's intended operating space. The coverage tool tracks how many of those goals were satisfied during simulation, and the engineer's task is to ensure that all of them are reached before the design is released to synthesis.

In the context of the Synchronous Counter, functional coverage might be defined around the values that the counter actually reaches. A testbench that runs the counter for only eight clock cycles after reset would achieve high code and branch coverage — the increment logic and the reset logic would both be exercised — but the counter would only have passed through states `0000` through `0111`. States `1000` through `1111` would remain unvisited. From a structural coverage perspective, this might not register as a problem, because the same increment expression is evaluated identically for all sixteen states. From a functional coverage perspective, it is a clear gap: the full range of counter states was not exercised, and behaviors specific to the upper half of the count range — such as overflow detection logic that only activates near `1111` — would remain untested.

For finite state machine designs, functional coverage is expressed in terms of state visits and transition

activations. Each state in the machine is a cover point, and each conditional transition is another. A simulation that drives the machine through some of its states but never activates certain transitions leaves the corresponding next-state logic unverified. Depending on the machine's complexity, this can be a subtle omission that is entirely invisible from code and branch coverage data, because the statements implementing those transitions may be syntactically unreachable only under specific input sequences that the testbench never produced.

Functional coverage becomes increasingly important as designs grow in complexity beyond the introductory circuits covered in this manual. For a simple counter or adder, the set of meaningful behaviors is small enough that code and branch coverage provide a reasonably complete picture. For a complete processor datapath, a memory controller, or a communications protocol engine, the space of meaningful behavioral scenarios grows to a size that cannot be inferred automatically from the RTL structure. An engineer working on such a design must explicitly define the functional properties that matter — the instruction combinations, the memory access patterns, the protocol state sequences — and measure whether simulation has achieved them. This is the role of functional coverage, and it is the reason that coverage-driven verification in industrial practice extends well beyond the structural metrics that are sufficient for beginner-level designs.

9.6 Using IMC

Cadence provides a dedicated tool for collecting, analyzing, and reporting on coverage data produced by simulation. This tool is the Integrated Metrics Center, referred to throughout the Cadence toolchain as `imc`. IMC is not itself a simulator; it does not execute Verilog RTL or apply stimulus to a design under test. Its role is to receive the coverage database generated by the Xcelium simulator during a coverage-enabled simulation run, aggregate the collected metrics across one or more simulation sessions, and present the results in an organized, navigable format that allows the engineer to identify which coverage goals have been satisfied and which remain open.

To use IMC, coverage collection must first be enabled in the simulator. By default, the Xcelium simulator executes the design and records waveform data without tracking coverage metrics, since coverage instrumentation introduces both simulation overhead and additional database storage. Coverage is enabled by passing appropriate flags to the `xrun` invocation. The `-coverage all` option instructs Xcelium to collect all available coverage types — code, branch, toggle, and any defined functional coverage constructs — during the simulation run. An alternative approach is to enable only specific coverage types using flags such as `-coverage b` for branch or `-coverage t` for toggle, which reduces overhead when only specific metrics are needed.

A complete simulation invocation for the Synchronous Counter design, with full coverage collection enabled, takes the following form:

```csh
xrun -coverage all synchronous_counter_4bit.v synchronous_counter_4bit_tb.v
```

This command compiles and simulates both source files in a single step, which is the same usage introduced in the simulation workflow in Chapter 8. The addition of `-coverage all` causes Xcelium to instrument the compiled design for coverage tracking and to write a coverage database to the local directory upon simulation completion. The database is typically stored under a subdirectory named `cov\_work` or similar, depending on the simulator configuration. The exact database location is reported in the simulation log output.

Once the simulation has completed, IMC can be launched from the same directory to analyze the resulting database:

```csh
imc -load cov_work/scope/worklib
```

The `-load` option points IMC to the coverage database directory generated by the simulator. Alternatively, IMC can be launched interactively without a command-line path argument, in which case the database can be loaded from within the IMC graphical interface. The tool presents a hierarchical view of the design modules alongside the associated coverage data, allowing the engineer to navigate from the top-level testbench module down through each instantiated submodule and inspect the coverage results at whatever level of detail the analysis requires.

The workflow, then, has a natural two-phase structure that builds directly on the simulation workflow established in Chapter 8. In the first phase, the simulation is run with coverage enabled, exercising the design under test with the testbench's stimulus and generating a coverage database as a side effect of that execution. In the second phase, IMC is used to analyze the database and identify which coverage goals were satisfied. These two phases may be iterated multiple times as the testbench is improved, with IMC results from each iteration guiding the next round of stimulus development. This iterative process is the basis of the coverage closure methodology described in Section 9.8.

9.7 Coverage Reports

When IMC loads a coverage database, it presents the collected data through several complementary views. The most immediate is a summary report that lists the overall coverage percentage for each metric type across the entire design. This top-level summary provides a first orientation to the state of the verification effort: it indicates at a glance whether the simulation has achieved broad coverage across the design or whether significant gaps remain. A summary showing ninety-five percent branch coverage and ninety-eight percent toggle coverage on a simple sequential design suggests that the testbench is nearly complete. A summary showing forty percent code coverage on a newly written testbench suggests that a large fraction of the design has never been reached.

Beneath the summary, IMC provides a module-level breakdown that shows the coverage metrics for each individual module in the design hierarchy separately. This hierarchical view is particularly useful when the design under test consists of multiple instantiated submodules, as in the Ripple Carry Adder from Chapter 5, where four individual Full Adder instances are embedded within a top-level module. Coverage that appears adequate at the top level may conceal low coverage in a specific submodule, if that submodule was not sufficiently exercised by the applied stimulus. The module-level report exposes such discrepancies directly, allowing the engineer to direct attention to the specific parts of the design that need additional testing rather than attempting to infer the problem from the aggregate percentage alone.

Within each module, IMC provides a source-annotated view that marks each statement or line in the RTL source with its coverage status. Lines that were executed at least once are typically shown in a neutral or positive annotation. Lines that were never executed are highlighted distinctly, often in a contrasting color in the graphical interface, making them immediately visible during review. This source-level annotation is one of the most practically useful outputs of coverage analysis, because it translates an abstract percentage into a concrete list of specific code regions that must be targeted by additional test vectors.

Branch coverage results are presented as hit and miss counts for each conditional construct. For an `if-else` statement, IMC will report whether the true branch, the false branch, or both were taken during simulation. For a `case` statement, each case arm is reported individually, making it straightforward to identify which specific case values were never exercised. This level of detail is what distinguishes branch coverage from simple statement coverage: not just whether the conditional was reached, but which of its outcomes were observed.

Toggle coverage reports list each signal in the design alongside the transitions observed. A signal is typically reported as fully covered when both the zero-to-one and one-to-zero transitions have been observed at least once. A signal that was held at a constant value throughout the simulation will show both transitions

as uncovered. A signal that transitioned in only one direction will show partial coverage. IMC typically presents this data as a percentage of signals with full toggle coverage within each module, alongside a detailed list of the specific nets that did not toggle, which can be sorted and filtered to prioritize the most significant coverage gaps.

Interpreting coverage percentages requires some engineering judgment. A coverage target of one hundred percent is a common aspiration, but it is not always achievable or meaningful. Some logic within a well-designed RTL description is architecturally unreachable under normal operating conditions: default arms of case statements that exist only to satisfy completeness requirements, reset sequences that can only be activated in ways not modeled by the testbench, or diagnostic paths that are guarded by conditions the simulation environment cannot reproduce. Coverage tools cannot automatically distinguish between logic that is unreachable because the stimulus is insufficient and logic that is unreachable because it was deliberately designed that way. The engineer must review uncovered regions individually and make a judgment about whether each one represents a genuine verification gap or a known unreachable construct. Unreachable constructs can typically be excluded from the coverage model through tool-specific exclusion mechanisms, allowing the reported percentage to reflect only the portions of the design that can and should be exercised.

9.8 Coverage Closure

Coverage closure is the process of iteratively improving verification completeness until the coverage data satisfies the criteria established for the design. It is not a single-pass activity. A first simulation run will almost always reveal coverage gaps, because initial testbenches tend to exercise the most straightforward operating conditions without explicitly targeting boundary cases, unusual input sequences, or infrequently activated control paths. Coverage analysis makes those gaps visible, and the engineer's response is to add stimulus that specifically targets the uncovered regions. The updated testbench is then simulated again, new coverage data is collected, and the results are inspected once more. This loop continues until the coverage metrics reach an acceptable level.

The practical sequence for coverage closure follows directly from the tools and workflows described in preceding sections. After a baseline simulation run with coverage enabled, IMC is used to identify the first set of coverage gaps: uncovered statements, untaken branches, non-toggling signals, or unvisited functional cover points. The engineer reviews these gaps and determines, for each one, whether it represents a testbench deficiency or a known unreachable construct. Testbench deficiencies are addressed by adding new test vectors or modifying existing ones to produce the stimulus conditions that activate the missing behaviors. Known unreachable constructs are documented and excluded from the coverage model. The revised testbench is then run again, and the new coverage results are compared against the previous run to confirm that the targeted gaps have been closed and that no regressions have been introduced.

Each iteration of this loop should be targeted rather than speculative. Adding test vectors randomly in the hope of improving coverage is inefficient and produces testbenches that are difficult to maintain. The more productive approach is to read the IMC report carefully, identify the specific uncovered branch or signal, trace it back to the RTL to understand what stimulus condition would activate it, and write a test vector that directly exercises that condition. For the Synchronous Counter, an uncovered `else` branch in the reset logic immediately suggests that a test must hold `reset` low for multiple clock cycles. An uncovered high-to-low transition on `reset` immediately suggests that a test must assert reset and then deassert it. The coverage data provides the precise information needed to construct the missing test efficiently.

The question of when to stop is a matter of engineering judgment rather than a fixed rule. One hundred percent coverage across all metrics is occasionally achievable for simple designs and is a reasonable target for laboratory exercises. In industrial practice, verification teams often define coverage goals in terms of specific thresholds for each metric type — ninety-five percent branch coverage and ninety percent toggle coverage, for example — together with documented justifications for any remaining uncovered regions. The important discipline is not to choose an arbitrary number and stop, but to ensure that every uncovered region has been reviewed and either targeted for additional testing or explicitly excluded with a documented rationale. A coverage gap that has been reviewed and consciously accepted is a defensible engineering decision. A coverage gap that exists simply because no one looked at it is a verification risk.

For the laboratory exercises that follow, the goal of coverage closure is to achieve complete coverage of the counter's reset behavior, increment behavior, and overflow behavior under the stimulus applied by the testbench. The coverage analysis workflow provides the measurement framework needed to confirm when that goal has been reached, and the IMC reports provide the detailed data needed to identify and close any remaining gaps. The next section applies this workflow to the Synchronous Counter design in a structured laboratory exercise, completing the verification flow from RTL simulation through coverage measurement and closure.

9.9 Laboratory Exercise – Counter Coverage Analysis

9.9.1 Objective

The objective of this laboratory exercise is to apply the coverage analysis workflow described in Sections 9.1 through 9.8 to the 4-bit Synchronous Counter developed in Chapter 6 and verified through simulation in Chapters 7 and 8. The exercise proceeds in two phases. In the first phase, the existing testbench from Chapter 7 is run with coverage collection enabled, and the resulting database is inspected in IMC to identify which behaviors were exercised and which were not. In the second phase, the testbench is improved to address the identified gaps, the simulation is rerun, and coverage closure is confirmed through a second IMC inspection. By the end of the exercise, the reader will have performed a complete coverage-driven verification iteration on a real RTL design using the Cadence toolchain.

9.9.2 Files Required

This exercise uses the Verilog source files created in earlier chapters. No new RTL is required.

- `synchronous\_counter\_4bit.v` — the 4-bit synchronous counter RTL developed in Chapter 6.
- `synchronous\_counter\_4bit\_tb.v` — the testbench developed in Chapter 7, which generates the clock, applies reset, and allows the counter to run for several cycles.

Both files should be present in the working directory used for the simulation exercises in Chapter 8. If either file has been modified since that chapter, the original implementations from Chapters 6 and 7 should be restored before proceeding. The exercise begins from that known baseline and deliberately uses the original testbench first, so that its coverage gaps can be observed directly before any improvement is made.

9.9.3 Procedure

The following procedure describes each step of the coverage analysis workflow in the order it should be performed. Each step should be completed fully before proceeding to the next.

1. **Navigate to the working directory.** Open a terminal session and change to the directory containing the counter source files. Confirm that both required files are present before proceeding:

   ```csh
   cd ~/cadence_test
   ls synchronous_counter_4bit.v synchronous_counter_4bit_tb.v
   ```

   The `ls` command should confirm both files without reporting any errors. If either file is missing, it should be restored from the relevant chapter before continuing.

2. **Run the simulation with coverage collection enabled.** Invoke `xrun` with the `-coverage all` flag to compile, elaborate, and simulate the design while collecting all available coverage metrics. The `-covworkdir` flag specifies the directory into which the coverage database will be written:

```csh
xrun -coverage all -covworkdir cov_work \
     synchronous_counter_4bit.v \
     synchronous_counter_4bit_tb.v
```

The simulator will compile both source files, elaborate the hierarchy with `synchronous\_counter\_4bit\_tb` as the top-level module, run the simulation to completion, and write the coverage database to the `cov\_work` directory upon exit. The console output should conclude with a normal simulation completion message consistent with those observed in Chapter 8. Any compilation or elaboration errors reported at this stage should be resolved before proceeding, using the debugging methodology described in Chapter 8.

3. **Verify that the coverage database was created.** Confirm that the `cov\_work` directory was populated by the simulation run:

```csh
ls -lh cov_work/
```

The directory should contain subdirectories and database files created by Xcelium during the simulation. The presence of these files confirms that coverage instrumentation was active and that the database is available for IMC. If the directory is empty or absent, the simulation invocation should be reviewed to confirm that the `-coverage all` and `-covworkdir` flags were included correctly.

4. **Launch IMC and load the coverage database.** Start the Cadence Integrated Metrics Center from the same working directory, pointing it to the coverage database generated in the previous step:

```csh
imc -load cov_work/scope/worklib
```

IMC will open its graphical interface and load the collected coverage data. The main window presents a hierarchical view of the design on the left and a tabbed report area on the right. A successful load will show the testbench and counter modules listed in the hierarchy panel, each with associated coverage percentages.

Figure 9.1: IMC main window after loading the coverage database generated from the initial counter testbench simulation.

5. **Open the coverage summary report.** In the IMC interface, navigate to the summary view, which presents aggregate coverage percentages for each metric type across the entire design. Record the reported percentages for statement coverage, branch coverage, and toggle coverage. These values represent the baseline coverage achieved by the original testbench from Chapter 7.

Figure 9.2: IMC coverage summary report for the initial testbench run, showing baseline coverage percentages across all metric types.

6. **Inspect statement coverage.** In the module hierarchy, select the `synchronous\_counter\_4bit` module to open its source-annotated coverage view. Each statement in the RTL will be annotated to indicate whether it was executed during the simulation. Confirm that both statements within the `always` block — the reset assignment and the increment assignment — appear as covered. If either statement is marked as not covered, record it as a coverage gap for the analysis step that follows.

7. **Inspect branch coverage.** Remaining in the source annotation view, examine the branch coverage data for the `if (reset)` conditional. IMC will indicate, for each branch, whether the true outcome and the false outcome were both observed during simulation. The true outcome corresponds to the reset being asserted at a rising clock edge. The false outcome corresponds to normal incrementing operation. Note the coverage status of each branch and record any that are reported as not covered.

Figure 9.3: IMC source annotation view for the synchronous counter, showing branch coverage results for the reset conditional. Uncovered branches are highlighted.

8. **Inspect toggle coverage.** Navigate to the toggle coverage report for the `synchronous\_counter\_4bit` module. Examine the toggle status of the `reset` input port and each bit of the `count` output bus. For each signal, IMC reports whether the zero-to-one and one-to-zero transitions were both observed. Record any signals for which one or both transitions are absent.

9. **Record the baseline coverage results.** Before making any changes to the testbench, record the coverage percentages obtained from the initial simulation run. These values will be compared against the results obtained after testbench improvement to demonstrate the effect of targeted stimulus additions.

9.9.4 Coverage Gap Analysis

Inspection of the initial coverage report will typically reveal that the original testbench from Chapter 7 leaves a specific branch condition incompletely exercised. The testbench applies an initial reset pulse at the start of the simulation and then releases reset for an extended period to allow the counter to run freely. It later applies a brief second reset pulse before the simulation ends. Depending on the exact delay values in the stimulus sequence, the second reset assertion may occur at a point in the simulation where the coverage tool has already recorded the branch as hit from the first reset interval, or the tool may report the reset branch as covered because at least one rising clock edge coincided with reset being high during the first pulse.

A more instructive scenario, and one that students may encounter depending on the specific timing of their testbench, arises when the stimulus sequence does not deassert reset before the simulation ends — or when a modified version of the testbench was inadvertently saved without the second reset pulse. In this case, the branch coverage report will show the true branch of the `if (reset)` conditional as covered but will show reduced toggle coverage on the `reset` input, because the signal never completed a low-to-high-to-low cycle in full view of the simulator. Toggle coverage on `reset` would then show the high-to-low transition as unobserved.

Regardless of the precise gap identified, the important skill practiced in this exercise is reading the IMC report carefully enough to connect a coverage metric to a specific stimulus condition. An uncovered branch in the `if (reset)` structure means that a rising clock edge has never been observed while reset was in a particular state. A missing toggle transition on `reset` means that the signal was never driven in the missing direction. Both observations point to the same corrective action: the testbench must be extended to drive the `reset` signal in the missing polarity, at a point in the simulation sequence where at least one clock edge will sample it.

9.9.5 Testbench Improvement

To close the coverage gaps identified in the preceding analysis, the testbench stimulus sequence should be extended to include an explicit second reset assertion after the counter has been running for a sufficient number of cycles. The following modification to the stimulus `initial` block demonstrates how this is accomplished. The additional reset pulse is inserted after the first free-running interval, ensuring that at least one rising clock edge samples `reset` high during the second assertion, and that a rising clock edge also samples it low after release:

```verilog
initial begin
    $monitor($time, " reset=%b count=%b", reset, count);
    reset = 1'b1;
```

```
    #12;
    reset = 1'b0;
    #180;
    reset = 1'b1;      // second reset assertion mid-sequence
    #15;               // hold for at least one full clock period
    reset = 1'b0;
    #50;               // observe several post-reset count steps
    $finish;
end
```

The original testbench from Chapter 7 already includes a second reset pulse, but if the timing values were insufficient to guarantee that a rising edge of `clk` coincided with `reset` being high during that pulse, the branch may have been recorded as partially covered. The revised version above holds the second reset assertion for 15 ns — one and a half clock periods given a 10 ns clock — which guarantees that at least one rising edge samples reset high, satisfying the branch coverage requirement and completing the toggle cycle on the `reset` signal.

The post-release delay of 50 ns allows five additional clock cycles to elapse after the second reset is removed. This ensures that the counter is observed returning to `0000` and then advancing normally from that point, which confirms that the reset behavior is correct not only at initialization but also after the counter has already been running for an extended period — precisely the corner case identified by the coverage gap analysis.

9.9.6 Re-run and Verify

With the testbench updated, the simulation should be rerun with coverage collection enabled. Save the modified testbench and invoke `xrun` again, directing the new coverage database to a separate output directory so that the baseline results are preserved for comparison:

```csh
xrun -coverage all -covworkdir cov_work_v2 \
     synchronous_counter_4bit.v \
     synchronous_counter_4bit_tb.v
```

After the simulation completes, launch IMC and load the updated database:

```csh
imc -load cov_work_v2/scope/worklib
```

Navigate to the same coverage views inspected during the initial analysis — the summary report, the branch coverage annotation for the `always` block, and the toggle coverage report for the `reset` input — and confirm that the previously uncovered items are now reported as covered. The branch previously shown as not taken should now appear as hit. The toggle transitions on `reset` should both be present. The aggregate coverage percentages in the summary report should reflect these improvements.

Figure 9.4: IMC coverage summary report after testbench improvement, showing closure of the branch and toggle coverage gaps identified in the initial analysis.

9.9.7 Observation Table

The table below summarizes the coverage results expected from both simulation runs. The baseline column reflects the coverage achieved by the original testbench, and the improved column reflects the coverage achieved after the second reset pulse was added. Students should record their own observed values in place of the representative figures shown here, as the exact numbers may vary depending on the simulator version and testbench timing.

Coverage Metric	Initial Testbench	Improved Testbench
Statement Coverage	100%	100%
Branch Coverage	50%	100%
Toggle Coverage	75%	100%

Table 9.1: Coverage results for the 4-bit Synchronous Counter before and after testbench improvement. Representative values are shown; students should record actual values from their IMC reports.

The statement coverage figure of one hundred percent in both runs is expected, because the original testbench already exercises both the reset path and the increment path at least once. The branch and toggle improvements visible in the second run are a direct consequence of the targeted stimulus addition described in the preceding section — an illustration of the principle established in Section 9.8 that coverage gaps should be addressed through specific, justified additions to the testbench rather than through broad, unguided changes.

9.9.8 Conclusion

This laboratory exercise demonstrated a complete iteration of the coverage-driven verification workflow applied to the 4-bit Synchronous Counter. The simulation was run with coverage collection enabled using `xrun`, the resulting database was loaded into IMC, and the coverage reports were inspected to identify a specific gap in the branch and toggle coverage of the reset logic. A targeted addition to the testbench stimulus sequence addressed that gap, and a second simulation run confirmed that the identified metrics had been closed.

The exercise reinforces the central observation made in Section 9.1: a simulation that passes is not, by itself, sufficient evidence that a design has been thoroughly verified. The original testbench produced correct waveforms and a clean `\$monitor` output in Chapter 8, yet the coverage analysis revealed that a specific behavioral condition — the reset signal being asserted at a clock edge after the counter had already been running — was exercised incompletely. Coverage measurement made that gap visible in a way that waveform inspection alone could not.

The coverage closure methodology practiced here scales directly to more complex designs. The workflow is the same regardless of the size of the design under test: enable coverage in the simulator, inspect the IMC reports for uncovered regions, trace each gap to the specific stimulus condition that would activate it, add that condition to the testbench, and rerun. What changes with design complexity is the number of iterations required and the sophistication of the stimulus needed to reach deep corner cases — topics that extend into more advanced verification methodologies beyond the scope of this manual. For the foundational VLSI workflow described here, the combination of Xcelium simulation, IMC coverage analysis, and the iterative closure process demonstrated in this exercise provides a complete and practical verification framework for the RTL designs encountered throughout these chapters.

Part IV

LOGIC SYNTHESIS

Synthesis, timing analysis, and logical equivalence checking

10

Introduction to Synthesis

System identity, licensing, and workstation preparation

11

Logic Synthesis using Genus

System identity, licensing, and workstation preparation

11.1 Preparing RTL

11.2 Writing Constraints

11.3 Running Genus

11.4 Generated Outputs

- Netlist
- Timing Reports
- Area Reports
- Power Reports

11.5 Laboratory Exercise

Synthesizing the Counter

12

Timing Analysis

System identity, licensing, and workstation preparation

12.1 Propagation Delay

12.2 Setup Time

12.3 Hold Time

12.4 Clock Frequency

12.5 Critical Paths

12.6 Slack

12.7 Laboratory Exercise

Counter Timing Analysis

13

Logical Equivalence Checking

System identity, licensing, and workstation preparation

13.1 Why Equivalence Checking Matters

13.2 Golden vs Revised Designs

13.3 Conformal Workflow

13.4 Laboratory Exercise

Verifying Counter Equivalence

Part V

PHYSICAL DESIGN

Floorplanning, power planning, placement, clocking, routing, and verification

14

Introduction to Physical Design

System identity, licensing, and workstation preparation

15

Floorplanning

System identity, licensing, and workstation preparation

15.1 Defining the Core Area

15.2 IO Placement

15.3 Macro Placement

15.4 Utilization

15.5 Laboratory Exercise

Counter Floorplan

16

Power Planning

System identity, licensing, and workstation preparation

17

Placement

System identity, licensing, and workstation preparation

17.1 Standard Cell Placement

17.2 Congestion

17.3 Density Optimization

17.4 Laboratory Exercise

Counter Placement

18

Clock Tree Synthesis

System identity, licensing, and workstation preparation

18.1 Clock Distribution

18.2 Clock Buffers

18.3 Clock Skew

18.4 Clock Latency

18.5 Laboratory Exercise

Counter CTS

19

Routing

System identity, licensing, and workstation preparation

19.1 Global Routing

19.2 Detailed Routing

19.3 Routing Resources

19.4 Signal Integrity

19.5 Laboratory Exercise

Counter Routing

20

Physical Verification

System identity, licensing, and workstation preparation

20.1 Design Rule Checking (DRC)

20.2 Layout Versus Schematic (LVS)

20.3 Common Physical Violations

20.4 Laboratory Exercise

Counter Verification

Part VI

TAPEOUT AND BEYOND

GDSII generation, packaging context, and complete flow review

21

GDSII Generation

System identity, licensing, and workstation preparation

22

From Silicon to PCB

System identity, licensing, and workstation preparation

23

Complete Design Flow Review

System identity, licensing, and workstation preparation

23.1 Revisiting the Counter

23.2 RTL → Simulation → Coverage

23.3 Synthesis → Timing → Verification

23.4 Floorplanning → CTS → Routing

23.5 Tapeout

23.6 Final Flow Summary

Verilog → Simulation → IMC → Genus → Conformal → Innovus → GDSII

01

c2ssetup Script for Automating User/Hostname Config

System identity, licensing, and workstation preparation

csh

```csh
#!/bin/csh

echo "======================================="
echo " Available Login Users"
echo "======================================="

awk -F: '$3 >= 1000 && $3 < 65534 && $1 != "nobody" {print $1}' /etc/passwd

echo ""
echo -n "Enter username to rename: "
set OLD_USER = "$<"

id $OLD_USER >& /dev/null
if ( $status != 0 ) then
    echo "Error: User '$OLD_USER' does not exist."
    exit 1
endif

echo ""
echo -n "Enter new username: "
set NEW_USER = "$<"

echo ""
echo -n "Enter display name (Full Name): "
set DISPLAY_NAME = "$<"

echo ""
echo -n "Enter new hostname: "
set NEW_HOST = "$<"

set OLD_GROUP = `id -gn $OLD_USER`

echo ""
echo "======================================="
echo " Renaming User"
echo "======================================="

sudo usermod -l $NEW_USER $OLD_USER
```

```
if ( $status != 0 ) then
    echo "Failed to rename user."
    exit 1
endif

echo ""
echo "========================================"
echo " Updating Home Directory"
echo "========================================"

sudo usermod -d /home/$NEW_USER -m $NEW_USER

if ( $status != 0 ) then
    echo "Failed to update home directory."
    exit 1
endif

echo ""
echo "========================================"
echo " Renaming Primary Group"
echo "========================================"

sudo groupmod -n $NEW_USER $OLD_GROUP

if ( $status != 0 ) then
    echo "Failed to rename group."
    exit 1
endif

echo ""
echo "========================================"
echo " Updating Display Name"
echo "========================================"

sudo usermod -c "$DISPLAY_NAME" $NEW_USER

if ( $status != 0 ) then
    echo "Failed to update display name."
    exit 1
endif

echo ""
echo "========================================"
echo " Fixing Ownership"
echo "========================================"

sudo chown -R ${NEW_USER}:${NEW_USER} /home/$NEW_USER

if ( -d /home/install ) then
    sudo chown -R ${NEW_USER}:${NEW_USER} /home/install
endif

echo ""
echo "========================================"
echo " Updating Hostname"
echo "========================================"

echo "$NEW_HOST" | sudo tee /etc/hostname > /dev/null

sudo sed -i "s/^127\.0\.0\.1.*/127.0.0.1 localhost loghost $NEW_HOST/" /etc/hosts

grep -q "c2s.cdacb.in" /etc/hosts
if ( $status != 0 ) then
```

```
    echo "14.139.1.126 c2s.cdacb.in" | sudo tee -a /etc/hosts > /dev/null
endif

grep -q "cadence_server" /etc/hosts
if ( $status != 0 ) then
    echo "14.139.1.126 cadence_server" | sudo tee -a /etc/hosts > /dev/null
endif

sudo hostnamectl set-hostname "$NEW_HOST"

if ( $status != 0 ) then
    echo "Failed to set hostname."
    exit 1
endif

echo ""
echo "========================================"
echo " Setup Complete"
echo "========================================"

echo "Username     : $NEW_USER"
echo "Display Name : $DISPLAY_NAME"
echo "Hostname     : $NEW_HOST"

echo ""
echo "Please log out and log back in."
echo ""

# For Synopsys tools add to ~/.cshrc if required:
#
# setenv SNPSLMD_LICENSE_FILE 27020@14.139.1.126
# setenv FLEXLM_DIAGNOSTICS 3
```

Save this file as `c2ssetup.sh` and then run the following command to make it executable:

terminal

```
sudo chmod +x c2ssetup.sh
```

Run the file:

terminal

```
./c2ssetup.sh
```

02

Cshrc configuration code

System identity, licensing, and workstation preparation

The following `.cshrc` configuration file must be present in the user environment for proper Cadence tool initialization. Since several Cadence utilities expect a C-Shell environment, the user must first switch from the default shell to C-Shell by executing the command `csh` in the terminal before launching any Cadence tools.

csh

```
#!/bin/csh
setenv CDS_Netlisting_Mode Analog
setenv QRC_ENABLE_QRCRACTION "1"
setenv CDS_AUTO_64BIT ALL
setenv CDS_AHDLCMI_ENABLE YES
setenv EDITOR gedit

setenv LM_LICENSE_FILE 5280@cadence_server
setenv CDS_LIC_FILE $LM_LICENSE_FILE
setenv LANG en_US

############################################################


setenv LIBERATEHOME     /home/install/LIBERATE201


############################################################
####################  CIC     ##############################

setenv CDSHOME      /home/install/IC618
#setenv CDSHOME     /home/install/ICADVM201
setenv ASSURAHOME   /home/install/ASSURA41
setenv QRC_HOME         /home/install/QUANTUS212
setenv PVSHOME      /home/install/PVS222
setenv MMSIMHOME    /home/install/SPECTRE211

############################################################
####################  ASIC    ##############################

setenv IUSHOME      /home/install/INCISIVE152
setenv LECHOME      /home/install/CONFRML211
setenv INNOVUSHOME  /home/install/INNOVUS211
setenv GENUSHOME    /home/install/GENUS211
setenv MODUSHOME    /home/install/MODUS201
```

```
setenv SSVHOME      /home/install/SSV221

####################NEW#################3

setenv JASPERHOME   /home/install/JASPER1909
setenv MVSHOME          /home/install/MVS202
setenv SIGRITYHOME      /home/install/SIGRITY20211
setenv STRATUSHOME  /home/install/STRATUS202
setenv XCELIUMHOME  /home/install/XCELIUM2209
setenv ULTRASIMHOME     /home/install/ULTRASIM181
setenv VMANAGERHOME     /home/install/VMANAGER2009
setenv INTEGRANDHOME    /home/install/INTEGRAND60
setenv JLSHOME          /home/install/JLS201
############################################################
####################  PCB  ################################

setenv SPBHOME      /home/install/SPB174

############################################################
####################  LIBRARIES  ###########################

setenv FOUNDRY      /home/install/FOUNDRY
setenv ANALOG       /home/install/FOUNDRY/analog
setenv STDCELL      /home/install/FOUNDRY/stdcell
setenv FINFET           /home/install/FOUNDRY/cds_ff_mpt_v_1.0
############################################################

setenv DIGITAL      /home/install/FOUNDRY/digital
setenv dig_45nm     $DIGITAL/45nm

############################################################

setenv CDS_INST_DIR     $CDSHOME
setenv CDSHOME      $CDS_INST_DIR
setenv AMSHOME      $IUSHOME
setenv LD_LIBRARY_PATH  $CDS_INST_DIR/tools/lib:$CDS_INST_DIR/tools/dfII/lib
setenv LD_LIBRARY_PATH  $IUSHOME/tools/lib

############################################################

set path = ($CDS_INST_DIR/bin $CDS_INST_DIR/tools/bin $MMSIMHOME/bin $ $IUSHOME/bin $IUSHOME/
    tools.lnx86/bin $IUSHOME/tools/dfII/bin $IUSHOME/tools.lnx86/dfII/bin $IUSHOME/tools.lnx86
    /dfII/bin $IUSHOME/share/bin $ASSURAHOME/bin $ASSURAHOME/tools/bin $ASSURAHOME/tools/dfII/
    bin $ASSURAHOME/tools.lnx86/dfII/bin $ASSURAHOME/tools.lnx86/dfII/bin $ASSURAHOME/share/
    bin $PVSHOME/tools/assura/bin $PVSHOME/tools/bin $PVSHOME/bin $PVSHOME/tools.lnx86/bin $
    PVSHOME/tools/dfII/bin $QRC_HOME/tools/bin $SPBHOME/bin $SPBHOME/tools.lnx86/pcb/bin $
    SPBHOME/tools.lnx86/dfII/bin $SPBHOME/tools.lnx86/dfII/bin $SPBHOME/tools.lnx86/dfII/bin $
    SPBHOME/tools.lnx86/pcb/bin $SPBHOME/tools.lnx86/bin $LECHOME/bin $LECHOME/tools.lnx86/bin
     $LECHOME/tools.lnx86/dfII/bin $LECHOME/tools.lnx86/dfII/bin $LECHOME/tools.lnx86/dfII/bin
     $LECHOME/share/bin $LIBERATEHOME/tools.lnx86/bin $LIBERATEHOME/tools.lnx86/dfII/bin $
    LIBERATEHOME/bin $LIBERATEHOME/tools.lnx86/dfII/bin $LIBERATEHOME/tools.lnx86/dfII/bin $
    GENUSHOME/bin $GENUSHOME/share/bin $GENUSHOME/tools.lnx86/dfII/bin $GENUSHOME/tools.lnx86/
    dfII/bin $GENUSHOME/tools.lnx86/bin $GENUSHOME/tools.lnx86/bin $INNOVUSHOME/bin $
    INNOVUSHOME/tools/bin $INNOVUSHOME/tools/dfII/bin $INNOVUSHOME/share/bin $SSVHOME/bin $
    SSVHOME/tools.lnx86/bin $SSVHOME/tools.lnx86/dfII/bin $SSVHOME/tools.lnx86/dfII/bin $
    SSVHOME/tools.lnx86/dfII/bin $SSVHOME/share/bin $XCELIUMHOME/tools.lnx86/dfII/bin $
    XCELIUMHOME/share/bin $XCELIUMHOME/bin $XCELIUMHOME/tools/bin $XCELIUMHOME/tools/dfII/bin
    $XCELIUMHOME/tools.lnx86/bin $XCELIUMHOME/tools.lnx86/dfII/bin  $MODUSHOME/bin   $
    MODUSHOME/tools.lnx86/bin $MODUSHOME/tools.lnx86/dfII/bin $MODUSHOME/tools.lnx86/bin $
    MODUSHOME/tools.lnx86/dfII/bin $JASPERHOME/bin $JASPERHOME/tools.lnx86/bin $JASPERHOME/
    tools.lnx86/dfII/bin $JASPERHOME/tools.lnx86/dfII/bin $JASPERHOME/tools.lnx86/dfII/bin $
    JASPERHOME/share/bin $MVSHOME/tools.lnx86/dfII/bin $MVSHOME/share/bin $MVSHOME/bin   $
    MVSHOME/tools.lnx86/bin $MVSHOME/tools.lnx86/dfII/bin $MVSHOME/tools.lnx86/bin $MVSHOME/
```

```
    tools.lnx86/dfII/bin $SIGRITYHOME/tools.lnx86/dfII/bin $SIGRITYHOME/share/bin $SIGRITYHOME
    /bin $SIGRITYHOME/tools.lnx86/bin $SIGRITYHOME/tools.lnx86/dfII/bin $SIGRITYHOME/tools.lnx
    86/bin $SIGRITYHOME/tools.lnx86/dfII/bin $STRATUSHOME/bin $STRATUSHOME/tools.lnx86/bin $
    STRATUSHOME/tools.lnx86/dfII/bin $STRATUSHOME/tools.lnx86/dfII/bin $STRATUSHOME/tools.lnx8
    6/dfII/bin $STRATUSHOME/share/bin  $ULTRASIMHOME/tools.lnx86/dfII/bin $ULTRASIMHOME/share/
    bin $ULTRASIMHOME/bin $ULTRASIMHOME/tools.lnx86/bin $ULTRASIMHOME/tools.lnx86/dfII/bin $
    ULTRASIMHOME/tools.lnx86/bin $ULTRASIMHOME/tools.lnx86/dfII/bin $VMANAGERHOME/tools.lnx86/
    dfII/bin $VMANAGERHOME/share/bin $VMANAGERHOME/bin $VMANAGERHOME/tools.lnx86/bin $
    VMANAGERHOME/tools.lnx86/dfII/bin $VMANAGERHOME/tools.lnx86/bin $VMANAGERHOME/tools.lnx86/
    dfII/bin $INTEGRANDHOME/tools.lnx86/dfII/bin $INTEGRANDHOME/share/bin $INTEGRANDHOME/bin $
    INTEGRANDHOME/tools.lnx86/bin $INTEGRANDHOME/tools.lnx86/dfII/bin $INTEGRANDHOME/tools.lnx
    86/bin $INTEGRANDHOME/tools.lnx86/dfII/bin $JLSHOME/tools.lnx86/dfII/bin $JLSHOME/share/
    bin $JLSHOME/bin $JLSHOME/tools.lnx86/bin $JLSHOME/tools.lnx86/dfII/bin $JLSHOME/tools.lnx
    86/bin $JLSHOME/tools.lnx86/dfII/bin $path)


##########################################################

clear
echo ""
echo "Welcome to Cadence tools Suite"
echo ""
```

A standard RHEL installation does not typically create a user-level `.cshrc` file by default, as Bash is generally configured as the primary login shell. Therefore, if no existing `.cshrc` file is present in the user home directory, create one manually using the configuration provided below. If a `.cshrc` file already exists, its contents should be replaced or merged carefully with the required configuration.